Knowledge-Driven IoT Fault Diagnosis System

Project documentation and source-code archive

Prepared from the supplied project ZIP

1 November 2026

This searchable PDF contains 93 text files, organized by their original relative paths. Code listings retain source line numbers. The index at the end gives each file's starting page.

Credentials are replaced with [REDACTED]. Runtime environments, Git metadata, caches, build outputs, local backups and macOS metadata are excluded.

CSV and JSONL datasets are represented by the first 40 lines of each file. Binary databases and the serialized model are listed in the artifact inventory. This PDF is a readable submission copy; it does not preserve an executable project.

Documentation and status statements below are reproduced from the supplied files; they have not been independently verified.

README.md

```
# Knowledge-Driven IoT-Based Fault Diagnosis System

[![Platform: macOS Apple Silicon](https://img.shields.io/badge/Platform-macOS%20Apple%20Silicon-blue.svg)](https://apple.com)
[![Python: 3.10+](https://img.shields.io/badge/Python-3.10%2B-brightgreen.svg)](https://python.org)
[![ESP32: PlatformIO](https://img.shields.io/badge/Firmware-ESP32%20PlatformIO-orange.svg)](https://platformio.org)
[![Node-RED: Automation](https://img.shields.io/badge/Orchestration-Node--RED-red.svg)](https://nodered.org)
[![License: MIT](https://img.shields.io/badge/License-MIT-purple.svg)](LICENSE)

An enterprise-grade, cyber-physical industrial monitoring system for a **12V DC motor asset**, built across six progressive engineering layers: from synthetic physics simulation, TLS cloud MQTT, and persistent SQLite database, through Node-RED automation, knowledge-driven AI/ML hybrid diagnosis, and non-blocking ESP32 edge firmware with physical alarm actuation.

---

## ⚡ Quick Start (One-Command Local Demo)

The entire system can be run immediately in local simulation mode without requiring physical hardware or paid cloud credentials:

```bash
# 1. Clone or navigate to the repository
cd "Fault Diagnosis System new"

# 2. Run initial setup (creates venv, applies migrations, seeds accounts, trains ML model)
./scripts/setup.sh

# 3. Launch one-command local demonstration
./scripts/run_demo.sh
```

- **Industrial Monitoring Portal**: Open http://localhost:8501
- **FastAPI REST API**: Open http://localhost:8000/docs
- **Default Credentials**:
  - Administrator: `[REDACTED]` / `[REDACTED]`
  - Operator: `operator` / `Operator@2026`

□ The Six Mandatory Development Layers

Layer	Milestone	Core Implementation Components	Verification Script
Layer 1	**Synthetic Data → MQTT → CSV → Dashboard**	Synthetic generator (7 physics scenarios), local Paho publisher, thread-safe CSV logger, live Streamlit polling charts.	`./scripts/run_layer1.sh`
Layer 2	**Cloud MQTT (HiveMQ TLS 8883) & Fallback**	TLS v1.2/1.3 client, exponential backoff, credential decoupling (`.env.example`), automatic local broker fallback.	`./scripts/run_layer2.sh`
Layer 3	**SQLite Historical Database (WAL Mode)**	SQLite 3 in WAL mode, `UNIQUE(event_id)` deduplication, schema migrations, composite index `(device_id, timestamp DESC)`.	`./scripts/run_layer3.sh`
Layer 4	**Node-RED Industrial Automation & Alerts**	Importable `nodered/flows.json`, consecutive fault debouncing ($N=3$), 15s cooldown, recovery notifications, Docker compose.	`./scripts/run_layer4.sh`
Layer 5	**Knowledge AI/ML + Auth + SaaS Dashboard**	Priority-ranked YAML rules, Random Forest with 10 rolling physics features, hybrid consensus reconciler, salted PBKDF2 login, dark SaaS theme.	`./scripts/run_layer5.sh`
Layer 6	**ESP32 Edge Hardware & Physical Alerts**	Arduino C++ firmware, MPU6050 AC-coupled vibration RMS, DS18B20 1-Wire, ACS712 auto-zero calibration, optical IR RPM ISR, LED/buzzer alarms.	`./scripts/run_layer6.sh`

□ System Architecture

```mermaid
flowchart TD
    subgraph Edge["Edge Layer & Simulation"]
        ESP["ESP32 DevKit Microcontroller\n(MPU6050, DS18B20, ACS712, IR RPM)"]
        SIM["Python Synthetic Generator\n(7 Physics Scenarios)"]
    end
```
```

README.md (continued)

```
    subgraph Messaging["Message Transport"]
        BROKER["MQTT Broker\n(Local Mosquitto 1883 / HiveMQ Cloud 8883 TLS)"]
    end

    ESP -->|iot/factory/motor01/sensors| BROKER
    SIM -->|iot/factory/motor01/sensors| BROKER

    subgraph Automation["Industrial Orchestration (Layer 4)"]
        NRED["Node-RED Engine\n(Debouncing N=3, Hysteresis, 15s Cooldown)"]
    end

    BROKER <--> NRED

    subgraph CoreBackend["Storage & Diagnostics Engine (Layer 3 & 5)"]
        INGEST["Ingestion Consumer (mqtt_subscriber.py)"]
        CSV["CSV Logger (sensor_telemetry.csv)"]
        SQL[("SQLite WAL Database\n(Deduplicated via event_id)")]
        RULES["Physics Rule Engine (YAML)"]
        ML["Random Forest Classifier"]
        HYBRID["Hybrid Decision Reconciler"]
    end

    BROKER --> INGEST
    INGEST --> CSV
    INGEST --> SQL
    INGEST --> RULES
    INGEST --> ML
    RULES --> HYBRID
    ML --> HYBRID
    HYBRID -->|Prescriptive Diagnosis| SQL
    HYBRID -->|iot/factory/motor01/alerts| BROKER

    subgraph Presentation["Operations & User Interface"]
        DASH["Streamlit Industrial Portal (Port 8501)"]
        API["FastAPI REST Backend (Port 8000)"]
        ALARM["ESP32 Physical Warning LED & Piezo Buzzer"]
    end

    SQL --> DASH
    SQL --> API
    BROKER -->|Alert Actuation| ALARM
```

□ Hardware Pin Mapping & Electrical Safety (Layer 6)

Sensor / Actuator	ESP32 GPIO	Operating Voltage	Conditioning & Safety Requirement
MPU6050 Accelerometer	**GPIO 21 (SDA)** **GPIO 22 (SCL)**	3.3V	Powered from 3.3V rail. I2C bus utilizes 4.7kΩ pull-ups.
DS18B20 Temp Probe	**GPIO 4 (DQ)**	3.3V	Requires external **4.7kΩ pull-up resistor** to 3.3V.
ACS712-05B Current	**GPIO 36 (ADC)**	5V	**MUST use 10kΩ/20kΩ voltage divider** to scale 0–5V to safe 0–3.3V ADC.
Optical Slot Tachometer	**GPIO 18**	3.3V	Optical slot interrupt on FALLING edge.
Status Warning LED	**GPIO 2**	3.3V	In series with 330Ω current-limiting resistor to GND.
Piezo Buzzer	**GPIO 15**	5V / 12V	Driven via 2N2222 NPN transistor with base resistor.
12V DC Motor	**L298N Driver**	12V DC (External)	**COMMON GROUND** shared with ESP32; motor never driven directly from GPIO.

□ Automated Testing

Execute the test suite covering schema compliance, deduplication, consensus logic, and REST routes:

```bash
# Run all 22 automated tests
./.venv/bin/pytest tests/ -v
```

README.md (continued) (continued)

````
---

## 📁 Repository Layout

```text
knowledge-driven-iot/
├── README.md                      # Primary project overview and instructions
├── IMPLEMENTATION_PLAN.md         # Layer-by-layer architectural plan
├── PROJECT_STATUS.md              # Detailed verification status report
├── .env.example                   # Environment configuration template
├── requirements.txt               # Pinned Python package dependencies
├── pyproject.toml                 # Pytest configuration
├── docker-compose.yml             # Docker stack for Mosquitto, Node-RED, Backend, UI
│
├── config/
│   ├── settings.py                # Strongly-typed environment settings
│   ├── thresholds.yaml            # Operational engineering thresholds
│   ├── devices.yaml               # Motor asset hardware specification
│   └── knowledge_base.yaml        # Declarative physics rule hypotheses
│
├── simulation/
│   ├── synthetic_generator.py     # 7-scenario physics telemetry generator
│   ├── mqtt_publisher.py          # Standalone Paho MQTT publisher
│   └── scenarios.py               # Operational regime parameter profiles
│
├── ingestion/
│   ├── mqtt_subscriber.py         # Authoritative ingestion & diagnosis consumer
│   ├── mqtt_client.py             # Managed client with TLS & exponential backoff
│   ├── csv_logger.py              # Thread-safe Layer 1 CSV persistence
│   └── data_validator.py          # Schema validation & deduplication cache
│
├── database/
│   ├── models.py                  # SQLite DDL schema (WAL mode, indexes)
│   ├── connection.py              # Thread-safe SQLite connection context manager
│   ├── repository.py              # Authoritative persistence repository
│   └── migrations/                # Versioned SQL migrations & runner
│
├── backend/
│   ├── main.py                    # FastAPI REST application
│   └── auth/security.py           # Salted PBKDF2 authentication & sessions
│
├── diagnosis/
│   ├── rule_engine.py             # Declarative YAML physics rule evaluator
│   ├── hybrid_engine.py           # Consensus reconciler (Rules + ML)
│   ├── recommendations.py         # Prescriptive maintenance action mapper
│   └── knowledge_base.py          # Knowledge base query & inspection interface
│
├── ml/
│   ├── train.py                   # Model training pipeline (Random Forest)
│   ├── predict.py                 # Real-time inference engine with rolling features
│   ├── evaluate.py                # Detailed tabular metric & confusion matrix evaluator
│   ├── features.py                # Rolling temporal feature engineering
│   └── models/                    # Saved .joblib model bundle & metrics
│
├── dashboard/
│   ├── app.py                     # Multi-page Streamlit portal with authentication
│   ├── pages/                     # 7 functional monitoring & diagnostics pages
│   ├── components/charts.py       # High-performance Plotly time-series visualizers
│   └── styles/theme.css           # Industrial dark charcoal UI theme
│
├── nodered/
│   ├── flows.json                 # Importable Node-RED automation workflow
│   ├── package.json               # Node-RED dependencies
│   └── README.md                  # Node-RED setup & hysteresis guide
│
├── firmware/esp32_motor_monitor/
│   ├── platformio.ini             # PlatformIO build configuration for ESP32
│   ├── src/main.cpp               # C++ firmware (RMS vibe, temp, current, RPM, alerts)
│   ├── include/config.h           # Firmware constants, pins, and calibration
│   └── README.md                  # Build, flash & calibration guide
│
````
````

README.md (continued) (continued) (continued)

```
├── tests/                        # Pytest automated test suites
├── scripts/                      # Executable launch & verification scripts
└── docs/                         # In-depth technical & academic documentation
```

📚 Academic Documentation

- [System Architecture](docs/ARCHITECTURE.md)
- [Layer 1 Guide](docs/LAYER_1.md)
- [Layer 2 Guide](docs/LAYER_2.md)
- [Layer 3 Guide](docs/LAYER_3.md)
- [Layer 4 Guide](docs/LAYER_4.md)
- [Layer 5 Guide](docs/LAYER_5.md)
- [Layer 6 Guide](docs/LAYER_6.md)
- [Hardware Wiring & Electrical Safety](docs/HARDWARE_WIRING.md)
- [REST API Specifications](docs/API.md)
- [Testing Methodology](docs/TESTING.md)
- [Troubleshooting & Recovery](docs/TROUBLESHOOTING.md)
- [Final Engineering Project Report](docs/PROJECT_REPORT.md)

.env

```
# ============================================================================
# Knowledge-Driven IoT-Based Fault Diagnosis System — Environment Configuration
# ============================================================================

# Application Environment (development / production / testing)
ENVIRONMENT=development
LOG_LEVEL=INFO
SECRET_KEY=[REDACTED]

# System Operation Mode (simulation / hardware)
OPERATION_MODE=hardware
DEVICE_ID=motor01

# MQTT Broker Configuration
# When USE_LOCAL_BROKER=true or HiveMQ credentials are blank, system falls back to local broker
USE_LOCAL_BROKER=false
LOCAL_MQTT_HOST=127.0.0.1
LOCAL_MQTT_PORT=1883

# HiveMQ Cloud Credentials (Layer 2 & Cloud Operations)
# TLS port 8883 is standard for HiveMQ Cloud
HIVEMQ_HOST=1285e2f1b7e14c249b55489e3f2de0e3.s1.eu.hivemq.cloud
HIVEMQ_PORT=8883
HIVEMQ_USERNAME=[REDACTED]
HIVEMQ_PASSWORD=[REDACTED]
HIVEMQ_USE_TLS=true

# SQLite Database
DATABASE_PATH=data/sqlite/fault_diagnosis.db

# Ingestion & Ingestion Path (mqtt / nodered)
INGESTION_PATH=mqtt
CSV_LOG_PATH=data/csv/sensor_telemetry.csv

# Backend API
API_HOST=0.0.0.0
API_PORT=8000

# Streamlit Dashboard
DASHBOARD_PORT=8501

# Node-RED Automation Integration
NODERED_ENABLED=true
NODERED_HOST=127.0.0.1
NODERED_PORT=1880
NODERED_API_URL=http://127.0.0.1:1880

# Machine Learning Paths
ML_MODEL_PATH=ml/models/random_forest_model.joblib
ML_METRICS_PATH=ml/models/model_metrics.json
ML_CONFUSION_MATRIX_PATH=ml/models/confusion_matrix.json
```

.env.example

```
# ============================================================================
# Knowledge-Driven IoT-Based Fault Diagnosis System — Environment Configuration
# ============================================================================

# Application Environment (development / production / testing)
ENVIRONMENT=development
LOG_LEVEL=INFO
SECRET_KEY=[REDACTED]

# System Operation Mode (simulation / hardware)
OPERATION_MODE=simulation
DEVICE_ID=motor01

# MQTT Broker Configuration
# When USE_LOCAL_BROKER=true or HiveMQ credentials are blank, system falls back to local broker
USE_LOCAL_BROKER=true
LOCAL_MQTT_HOST=127.0.0.1
LOCAL_MQTT_PORT=1883

# HiveMQ Cloud Credentials (Layer 2 & Cloud Operations)
# TLS port 8883 is standard for HiveMQ Cloud
HIVEMQ_HOST=
HIVEMQ_PORT=8883
HIVEMQ_USERNAME=
[REDACTED]
HIVEMQ_USE_TLS=true

# SQLite Database
DATABASE_PATH=data/sqlite/fault_diagnosis.db

# Ingestion & Ingestion Path (mqtt / nodered)
INGESTION_PATH=mqtt
CSV_LOG_PATH=data/csv/sensor_telemetry.csv

# Backend API
API_HOST=0.0.0.0
API_PORT=8000

# Streamlit Dashboard
DASHBOARD_PORT=8501

# Node-RED Automation Integration
NODERED_ENABLED=true
NODERED_HOST=127.0.0.1
NODERED_PORT=1880
NODERED_API_URL=http://127.0.0.1:1880

# Machine Learning Paths
ML_MODEL_PATH=ml/models/random_forest_model.joblib
ML_METRICS_PATH=ml/models/model_metrics.json
ML_CONFUSION_MATRIX_PATH=ml/models/confusion_matrix.json

```

.gitignore

```
# Python
__pycache__/
*.py[cod]
*$py.class
*.so
.Python
env/
.venv/
venv/
ENV/
build/
develop-eggs/
dist/
downloads/
eggs/
.eggs/
lib/
lib64/
parts/
sdist/
var/
wheels/
*.egg-info/
.installed.cfg
*.egg

# Virtualenv
.env
!.env.example

# IDE
.vscode/
.idea/
*.swp
*.swo
.DS_Store

# Testing & Cache
.pytest_cache/
.coverage
htmlcov/

# Logs & DBs
*.log
data/sqlite/*.db
!data/sqlite/.gitkeep
data/csv/*.csv
!data/csv/.gitkeep


# Private ESP32 credentials
/Final_ARD_Code/secrets.h
.local_backups/

# Private PlatformIO ESP32 credentials
/firmware/esp32_motor_monitor/include/local_secrets.h

# Local-only files
.venv-old-link
/data/node_red/*.jsonl
/node_red_flow.json
```

Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

# Install system dependencies
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    curl \
    && rm -rf /var/lib/apt/lists/*

# Install python dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy project source
COPY . .

# Expose API and Streamlit ports
EXPOSE 8000 8501

CMD ["python", "backend/main.py"]

```

IMPLEMENTATION_PLAN.md

````
# Implementation Plan — Knowledge-Driven IoT-Based Fault Diagnosis System

## Overview
This document specifies the architecture, engineering roadmap, and verification strategy for the end-to-end six-layer industrial IoT fault diagnosis system monitoring a 12V DC motor asset.

---

## 1. Architectural Architecture & Layer Milestones

```mermaid
flowchart TD
    subgraph L1_L2["Layer 1 & 2: Telemetry Ingestion"]
        S1["Synthetic Generator / ESP32 Hardware"] -->|JSON over MQTT| B1["MQTT Broker (Local Mosquitto 1883 / HiveMQ Cloud 8883 TLS)"]
        B1 -->|Telemetry Stream| SUB["Managed Ingestion Subscriber"]
        SUB -->|Raw Log| CSV["CSV Persistence (data/csv/sensor_telemetry.csv)"]
    end

    subgraph L3["Layer 3: Authoritative Database"]
        SUB -->|Deduplicated Insert| SQLITE[("SQLite Database (WAL Mode)")]
        SQLITE --- T1["devices"]
        SQLITE --- T2["sensor_readings"]
        SQLITE --- T3["fault_predictions"]
        SQLITE --- T4["alert_history"]
        SQLITE --- T5["users / system_events"]
    end

    subgraph L4["Layer 4: Industrial Automation"]
        B1 -->|iot/factory/+/sensors| NRED["Node-RED Orchestration Engine"]
        NRED -->|Hysteresis & Cooldown| NRED_ALERT["Alert Dispatch & Recovery"]
        NRED_ALERT -->|iot/factory/+/alerts| B1
    end

    subgraph L5["Layer 5: Knowledge-Driven AI/ML & Dashboard"]
        SUB -->|Windowed Readings| HYBRID["Hybrid Diagnostic Engine"]
        HYBRID -->|Rules Evaluation| RE["Knowledge Rule Engine (YAML Rules)"]
        HYBRID -->|Temporal Features| ML["Scikit-Learn Random Forest"]
        HYBRID -->|Consensus / Conflict Resolution| DIAG_OUT["Prescriptive Diagnosis"]
        DIAG_OUT -->|Store| SQLITE
        DIAG_OUT -->|Publish| B1
        SQLITE --> DASH["Streamlit Industrial Portal (Port 8501)"]
        SQLITE --> FASTAPI["FastAPI REST API (Port 8000)"]
    end

    subgraph L6["Layer 6: Physical ESP32 Hardware"]
        SENSORS["MPU6050 (RMS), DS18B20, ACS712, IR Pulse"] --> ESP32["ESP32 DevKit Microcontroller"]
        ESP32 -->|WiFi + TLS MQTT| B1
        B1 -->|iot/factory/+/alerts (Commands)| ESP32
        ESP32 --> ACT["Status LED / Piezo Buzzer"]
    end
```

---

## 2. Detailed Six-Layer Implementation Breakdown

### Layer 1: Synthetic Telemetry → Local MQTT → CSV Logging → Dashboard
- **Simulation Generator (`simulation/synthetic_generator.py`)**: Reproducible random physics simulation modeling normal motor behavior, mechanical overload, excessive vibration, thermal overheating, slowdown, motor stall, and sensor acquisition anomalies.
- **Publisher (`simulation/mqtt_publisher.py`)**: Paho MQTT publisher with configurable publishing rate, burst intervals, and scenario selection.
- **Ingestion (`ingestion/csv_logger.py`, `ingestion/data_validator.py`)**: Thread-safe CSV append logger with JSON schema validation.
- **Verification**: `tests/test_simulation.py`, `tests/test_mqtt.py`.

### Layer 2: Cloud MQTT Integration (HiveMQ Cloud TLS 8883) with Local Fallback
- **TLS Client (`ingestion/mqtt_client.py`)**: Paho MQTT v2 client with automatic exponential backoff, reconnect handling, TLS v1.2/1.3 wrapper, and fallback to local broker when cloud credentials are unconfigured.
- **Settings (`config/settings.py`)**: Strict environment variable parsing, avoiding hardcoded credentials.
````

IMPLEMENTATION_PLAN.md (continued)

```
- **Verification**: `tests/test_mqtt.py`.

### Layer 3: Authoritative SQLite Persistence & Analytics
- **Schema (`database/models.py`)**: SQLite in Write-Ahead-Logging (WAL) mode with foreign keys, indexes on `(device_id, timestamp)`, and `UNIQUE(event_id)` deduplication.
- **Repository (`database/repository.py`)**: Centralized repository pattern handling telemetry insertion, historical window queries, alert acknowledgements, and system event logging.
- **Migrations (`database/migrations/`)**: Versioned schema migration script (`001_initial_schema.sql`).
- **Verification**: `tests/test_database.py`.

### Layer 4: Node-RED Automation, Hysteresis & Alert Dispatch
- **Flow Definition (`nodered/flows.json`)**: Importable flow supporting MQTT topic subscription, JSON validation, persistence state tracking, consecutive fault debouncing ($N=3$), cooldown timers (15s), recovery events, and alert publishing.
- **Docker Integration (`docker-compose.yml`)**: Multi-container compose configuration running Mosquitto, Node-RED, FastAPI, and Streamlit.
- **Verification**: `tests/test_alerts.py`.

### Layer 5: Knowledge-Driven AI/ML Expert System & Multi-Page Dashboard
- **Declarative Knowledge Base (`config/knowledge_base.yaml`, `diagnosis/rule_engine.py`)**: Priority-ranked physics equations evaluating load, thermal stress, locked rotor, dynamic unbalance, and transducer fault hypotheses.
- **Machine Learning Pipeline (`ml/features.py`, `ml/train.py`, `ml/predict.py`, `ml/evaluate.py`)**: Rolling feature extractor (temperature drift, vibration RMS trends, speed drops, current variance), baseline vs Random Forest training, group holdout testing, and model persistence.
- **Hybrid Consensus Reconciler (`diagnosis/hybrid_engine.py`)**: Combines rule engine determinism with ML probabilistic scores, highlighting diagnostic conflicts and uncertainties.
- **Security & RBAC (`backend/auth/security.py`)**: PBKDF2-HMAC-SHA256 password hashing with salt, rate-limiting, and token authentication.
- **Streamlit Industrial SaaS Dashboard (`dashboard/`)**: Dark theme (GitHub/Grafana palette), live polling, real-time charts, diagnosis breakdowns, historical analytics, alert acknowledgement, system settings, and model metrics.
- **Verification**: `tests/test_rules.py`, `tests/test_ml.py`, `tests/test_api.py`.

### Layer 6: ESP32 Hardware Integration & Physical Alerts
- **Firmware (`firmware/esp32_motor_monitor/`)**: PlatformIO C++ firmware with non-blocking WiFi/MQTT reconnection, I2C MPU6050 vibration RMS calculation (sampling window 100 samples), OneWire DS18B20 temperature, ACS712 zero-offset calibrated current, and interrupt-driven optical RPM measurement.
- **Physical Indication**: Listens to `iot/factory/motor01/alerts`, toggling warning LEDs and buzzer alert tones on CRITICAL diagnostic flags.
- **Documentation (`docs/HARDWARE_WIRING.md`)**: Comprehensive pinouts, 3.3V/5V logic conversion, optoisolator protection, separate 12V motor power rail, and safety checklists.

---

## 3. Execution & Verification Roadmap

1. **Verify Existing Tests**: Confirm baseline 22 unit tests pass across all functional modules.
2. **Add Missing Modules**:
   - `ml/evaluate.py`: Standalone evaluation script with CLI output.
   - `diagnosis/recommendations.py`: Prescriptive maintenance engine.
   - `diagnosis/knowledge_base.py`: Knowledge base inspector and query interface.
   - `database/migrations/001_initial_schema.sql` & `migrate.py`.
3. **Build ESP32 Firmware & Hardware Files**:
   - `firmware/esp32_motor_monitor/platformio.ini`
   - `firmware/esp32_motor_monitor/include/config.h`
   - `firmware/esp32_motor_monitor/src/main.cpp`
   - `firmware/esp32_motor_monitor/README.md`
4. **Create Shell Launch Scripts (`scripts/`)**:
   - `scripts/setup.sh`
   - `scripts/run_demo.sh`
   - `scripts/run_layer1.sh` through `scripts/run_layer6.sh`
5. **Produce Engineering Documentation (`docs/` & `README.md`)**:
   - `README.md`
   - `docs/ARCHITECTURE.md`
   - `docs/LAYER_1.md` through `docs/LAYER_6.md`
   - `docs/HARDWARE_WIRING.md`
   - `docs/API.md`
   - `docs/TESTING.md`
   - `docs/TROUBLESHOOTING.md`
   - `docs/PROJECT_REPORT.md`
```

IMPLEMENTATION_PLAN.md (continued) (continued)

```
119  6. **Execute Full Test & Verification Suite**:
120     - Run pytest and record detailed terminal logs in `PROJECT_STATUS.md`.
```

PROJECT_STATUS.md

```
# Project Status & Verification Report
# Knowledge-Driven IoT-Based Fault Diagnosis System

**Last Verified:** 2026-10-30
**Environment:** macOS Apple Silicon (Darwin arm64), Python 3.14.7, Virtualenv `.venv`
**Git Branch:** `main` (clean local commit, no uncommitted changes)

---

## 1. Six-Layer Acceptance Checklist

| Layer | Component | Status | Verification Detail |
|---|---|---|---|
| **Layer 1** | Synthetic Data Generator |  VERIFIED | 7 physics scenarios, reproducible random seed (`tests/test_simulation.py` passed). |
| **Layer 1** | Local MQTT Pub/Sub |  VERIFIED | Paho MQTT loopback validated (`tests/test_mqtt.py` passed). |
| **Layer 1** | CSV Persistence |  VERIFIED | Thread-safe CSV append logger writing to `data/csv/sensor_telemetry.csv`. |
| **Layer 1** | Live Streamlit Dashboard |  VERIFIED | Multi-page Streamlit portal with auto-polling Plotly charts on Port 8501. |
| **Layer 2** | HiveMQ Cloud TLS (8883) |  IMPLEMENTED | TLS v1.2/1.3 client with credential decoupling; requires user `.env` for cloud. |
| **Layer 2** | Local Broker Fallback |  VERIFIED | Graceful fallback to `127.0.0.1:1883` verified (`tests/test_mqtt.py` passed). |
| **Layer 3** | SQLite Schema & Models |  VERIFIED | WAL mode, 7 tables, composite indexes on `(device_id, timestamp DESC)`. |
| **Layer 3** | Deduplication & History |  VERIFIED | `UNIQUE(event_id)` deduplication and time-range query verified (`tests/test_database.py` passed). |
| **Layer 3** | Database Migrations |  VERIFIED | Versioned SQL migrations runner (`database/migrations/migrate.py`) executed. |
| **Layer 4** | Node-RED Flow (`flows.json`) |  VERIFIED | Importable flow with deduplication cache, debouncing, and alerts. |
| **Layer 4** | Consecutive Debounce ($N=3$) |  VERIFIED | Suppression of inrush spikes verified (`tests/test_alerts.py` passed). |
| **Layer 4** | Alert Cooldown & Recovery |  VERIFIED | 15s cooldown suppression & recovery events verified (`tests/test_alerts.py` passed). |
| **Layer 5** | Knowledge-Driven Rule Engine |  VERIFIED | Priority-ranked YAML rules verified across all conditions (`tests/test_rules.py` passed). |
| **Layer 5** | Machine Learning Pipeline |  VERIFIED | 10 rolling features, Random Forest 100% test accuracy (`tests/test_ml.py` passed). |
| **Layer 5** | Hybrid Consensus Reconciler |  VERIFIED | Agreement confidence boost & conflict safety fallback verified (`tests/test_ml.py` passed). |
| **Layer 5** | User Authentication (PBKDF2) |  VERIFIED | Salted PBKDF2 hash, rate-limiting & session token verified (`tests/test_api.py` passed). |
| **Layer 5** | Multi-Page Dark SaaS UI |  VERIFIED | 7 responsive pages styled in dark charcoal industrial SaaS theme. |
| **Layer 6** | ESP32 C++ Firmware |  VERIFIED | Non-blocking firmware for MPU6050 RMS, DS18B20, ACS712, IR pulse ISR, and alert actuation. |
| **Layer 6** | Hardware Wiring & Safety |  VERIFIED | Pin mappings, 10k/20k voltage divider, 4.7k pullup, and safety checklists documented. |

---

## 2. Test Execution Summary

```text
============================= test session starts ==============================
platform darwin -- Python 3.14.7, pytest-9.2.0, pluggy-1.7.0 -- .venv/bin/python3.14
rootdir: /Users/[REDACTED]/Desktop/Fault Diagnosis System new
configfile: pyproject.toml

tests/test_alerts.py::test_alert_persistence_debouncing PASSED           [  4%]
tests/test_api.py::test_health_endpoint PASSED                           [  9%]
tests/test_api.py::test_auth_and_protected_flow PASSED                   [ 13%]
tests/test_api.py::test_model_metrics_endpoint PASSED                    [ 18%]
tests/test_database.py::test_insert_and_retrieve_reading PASSED          [ 22%]
tests/test_database.py::test_alert_lifecycle PASSED                      [ 27%]
tests/test_ml.py::test_feature_extraction PASSED                         [ 31%]
tests/test_ml.py::test_ml_predictor_inference PASSED                     [ 36%]
tests/test_ml.py::test_hybrid_engine_consensus PASSED                    [ 40%]
tests/test_mqtt.py::test_effective_mqtt_config_fallback PASSED           [ 45%]
```
```

PROJECT_STATUS.md (continued)

```
tests/test_mqtt.py::test_data_validator_deduplication PASSED             [ 50%]
tests/test_mqtt.py::test_local_mqtt_pub_sub_loopback PASSED              [ 54%]
tests/test_rules.py::test_rule_normal_operation PASSED                   [ 59%]
tests/test_rules.py::test_rule_motor_stall PASSED                        [ 63%]
tests/test_rules.py::test_rule_overheating PASSED                        [ 68%]
tests/test_rules.py::test_rule_mechanical_overload PASSED                [ 72%]
tests/test_rules.py::test_rule_excessive_vibration PASSED                [ 77%]
tests/test_rules.py::test_rule_sensor_anomaly PASSED                     [ 81%]
tests/test_simulation.py::test_scenario_profiles_completeness PASSED     [ 86%]
tests/test_simulation.py::test_generator_schema_compliance PASSED        [ 90%]
tests/test_simulation.py::test_generator_reproducibility PASSED          [ 95%]
tests/test_simulation.py::test_stall_scenario_physics PASSED             [100%]

======================== 22 passed, 1 warning in 2.70s ========================
```

---

3. Operational Distinctions

Verified Local Simulation (No Hardware or Credentials Required)
- Runs completely offline on macOS Apple Silicon or Linux.
- Synthetic generator produces realistic motor telemetry across normal and 6 fault scenarios.
- Telemetry flows through local MQTT loopback, persists to CSV and SQLite in WAL mode.
- Evaluates rules and Random Forest machine learning models in real-time.
- Multi-page Streamlit portal provides full live charts, diagnostic explanations, alert management, and historical analytics.

Requires User HiveMQ Cloud Credentials
- Cloud TLS messaging requires filling in `.env` with valid cluster credentials:
  - `HIVEMQ_HOST` (e.g., `xxxxxx.s1.eu.hivemq.cloud`)
  - `HIVEMQ_PORT=8883`
  - `HIVEMQ_USERNAME`
  - `HIVEMQ_PASSWORD`
  - `HIVEMQ_USE_TLS=true`
  - `USE_LOCAL_BROKER=false`
- If unconfigured, the system operates in local broker fallback mode without crashing.

Requires Physical ESP32 Hardware
- Physical ESP32 DevKit V1 board connected via USB.
- Monitored 12V DC motor driven by an L298N driver and external 12V 2A power supply.
- Sensors wired according to `docs/HARDWARE_WIRING.md`:
  - DS18B20 with 4.7kΩ pull-up to 3.3V on GPIO 4.
  - MPU6050 I2C on GPIO 21 (SDA) and GPIO 22 (SCL).
  - ACS712-05B on GPIO 36 via 10kΩ/20kΩ voltage divider.
  - Optical slot sensor on GPIO 18 (hardware interrupt).
  - Warning LED on GPIO 2 and buzzer driver on GPIO 15.
- Both simulation and physical hardware feed the exact same downstream pipeline and SQLite database.

docker-compose.yml

```yaml
version: '3.8'

services:
  # 1. Mosquitto Local MQTT Broker
  mosquitto:
    image: eclipse-mosquitto:2
    container_name: fault_diagnosis_mosquitto
    restart: unless-stopped
    ports:
      - "1883:1883"
    environment:
      - TZ=UTC
    command: mosquitto -c /mosquitto-no-auth.conf

  # 2. Node-RED Industrial Automation & Orchestration Service
  nodered:
    image: nodered/node-red:latest
    container_name: fault_diagnosis_nodered
    restart: unless-stopped
    depends_on:
      - mosquitto
    ports:
      - "1880:1880"
    environment:
      - TZ=UTC
      - FLOWS=flows.json
    volumes:
      - ./nodered:/data

  # 3. Python FastAPI Diagnostic Backend
  backend:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: fault_diagnosis_backend
    restart: unless-stopped
    depends_on:
      - mosquitto
    ports:
      - "8000:8000"
    environment:
      - USE_LOCAL_BROKER=true
      - LOCAL_MQTT_HOST=mosquitto
      - LOCAL_MQTT_PORT=1883
    volumes:
      - ./data:/app/data

  # 4. Streamlit Monitoring Dashboard
  dashboard:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: fault_diagnosis_dashboard
    restart: unless-stopped
    depends_on:
      - backend
    ports:
      - "8501:8501"
    command: streamlit run dashboard/app.py --server.port=8501 --server.address=0.0.0.0
    volumes:
      - ./data:/app/data

```

node_red_flow.json

```
[
    {
        "id": "54da57a1d2760d74",
        "type": "tab",
        "label": "Flow 1",
        "disabled": false,
        "info": "",
        "env": []
    },
    {
        "id": "6a653036bc72c6a4",
        "type": "mqtt in",
        "z": "54da57a1d2760d74",
        "name": "Motor Fault Alerts",
        "topic": "iot/factory/motor01/alerts/#",
        "qos": "0",
        "datatype": "auto-detect",
        "broker": "565918fea3215f7c",
        "nl": false,
        "rap": true,
        "rh": 0,
        "inputs": 0,
        "x": 215,
        "y": 340,
        "wires": [
            [
                "a97f57419f246ecd"
            ]
        ],
        "l": false
    },
    {
        "id": "a967ebe8e814cc74",
        "type": "debug",
        "z": "54da57a1d2760d74",
        "name": "debug 1",
        "active": true,
        "tosidebar": true,
        "console": false,
        "tostatus": false,
        "complete": "payload",
        "targetType": "msg",
        "statusVal": "",
        "statusType": "auto",
        "x": 540,
        "y": 340,
        "wires": []
    },
    {
        "id": "4406bb09e1109abf",
        "type": "inject",
        "z": "54da57a1d2760d74",
        "name": "Simulate Motor Fault",
        "props": [
            {
                "p": "payload"
            },
            {
                "p": "topic",
                "vt": "str"
            }
        ],
        "repeat": "",
        "crontab": "",
        "once": false,
        "onceDelay": 0.1,
        "topic": "",
        "payload": "{   \"device_id\": \"motor01\",   \"test\": true,   \"severity\": \"WARNING\",   \"fault\": \"TEST ONLY - Possible Overheating\",   \"temperature_c\": 70,   \"actuate_hardware\": false }",
        "payloadType": "json",
        "x": 210,
```

node_red_flow.json (continued)

```
        "y": 700,
        "wires": [
            [
                "15d2aa2763ceaffa"
            ]
        ]
    },
    {
        "id": "15d2aa2763ceaffa",
        "type": "mqtt out",
        "z": "54da57a1d2760d74",
        "name": "Publish Test Alert",
        "topic": "iot/factory/motor01/alerts/test",
        "qos": "0",
        "retain": "false",
        "respTopic": "",
        "contentType": "",
        "userProps": "",
        "correl": "",
        "expiry": "",
        "broker": "565918fea3215f7c",
        "x": 470,
        "y": 700,
        "wires": []
    },
    {
        "id": "a97f57419f246ecd",
        "type": "switch",
        "z": "54da57a1d2760d74",
        "name": "",
        "property": "topic",
        "propertyType": "msg",
        "rules": [
            {
                "t": "eq",
                "v": "iot/factory/motor01/alerts/test",
                "vt": "str"
            },
            {
                "t": "eq",
                "v": "iot/factory/motor01/alerts",
                "vt": "str"
            }
        ],
        "checkall": "true",
        "repair": false,
        "outputs": 2,
        "x": 370,
        "y": 380,
        "wires": [
            [
                "a967ebe8e814cc74",
                "c086fcba17587385"
            ],
            [
                "0345ac59cb79eac8",
                "91e36132c842b5bc",
                "5b6d533c85cf0f3f"
            ]
        ]
    },
    {
        "id": "0345ac59cb79eac8",
        "type": "debug",
        "z": "54da57a1d2760d74",
        "name": "Real Motor Alerts",
        "active": true,
        "tosidebar": true,
        "console": false,
        "tostatus": false,
        "complete": "payload",
```

node_red_flow.json (continued) (continued)

```
        "targetType": "msg",
        "statusVal": "",
        "statusType": "auto",
        "x": 570,
        "y": 400,
        "wires": []
    },
    {
        "id": "91e36132c842b5bc",
        "type": "switch",
        "z": "54da57a1d2760d74",
        "name": "Fault Severity Filter",
        "property": "payload.severity",
        "propertyType": "msg",
        "rules": [
            {
                "t": "eq",
                "v": "WARNING",
                "vt": "str"
            },
            {
                "t": "eq",
                "v": "CRITICAL",
                "vt": "str"
            }
        ],
        "checkall": "true",
        "repair": false,
        "outputs": 2,
        "x": 570,
        "y": 460,
        "wires": [
            [
                "d835881b5a06196c"
            ],
            [
                "2eeab1636b1eb242"
            ]
        ]
    },
    {
        "id": "d835881b5a06196c",
        "type": "debug",
        "z": "54da57a1d2760d74",
        "name": "Motor Warning Alert",
        "active": true,
        "tosidebar": true,
        "console": false,
        "tostatus": false,
        "complete": "payload",
        "targetType": "msg",
        "statusVal": "",
        "statusType": "auto",
        "x": 860,
        "y": 440,
        "wires": []
    },
    {
        "id": "2eeab1636b1eb242",
        "type": "debug",
        "z": "54da57a1d2760d74",
        "name": "Critical Motor Alarm",
        "active": true,
        "tosidebar": true,
        "console": false,
        "tostatus": false,
        "complete": "payload",
        "targetType": "msg",
        "statusVal": "",
        "statusType": "auto",
        "x": 860,
```

node_red_flow.json (continued) (continued) (continued)

```
        "y": 500,
        "wires": []
    },
    {
        "id": "5b6d533c85cf0f3f",
        "type": "function",
        "z": "54da57a1d2760d74",
        "name": "Save Fault Alert",
        "func": "\nif (msg.topic !== \"iot/factory/motor01/alerts\") {\n    return null;\n}\n\nlet alert = msg.payload;\n\nif (typeof alert === \"string\") {\n    try {\n        alert = JSON.parse(alert);\n    } catch (error) {\n        node.warn(\"Invalid alert JSON\");\n        return null;\n    }\n}\n\nif (!alert || typeof alert !== \"object\" || alert.test === true) {\n    return null;\n}\n\nconst log = {\n    received_at: new Date().toISOString(),\n    timestamp: alert.timestamp,\n    device_id: alert.device_id,\n    severity: alert.severity,\n    fault: alert.fault,\n    message: alert.message,\n    recommended_action: alert.recommended_action\n};\n\nmsg.payload = JSON.stringify(log) + \"\\n\";\n\nreturn msg;\n",
        "outputs": 1,
        "timeout": 0,
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 560,
        "y": 560,
        "wires": [
            [
                "cf050b9957bebc80"
            ]
        ]
    },
    {
        "id": "cf050b9957bebc80",
        "type": "file",
        "z": "54da57a1d2760d74",
        "name": "Fault Alert History",
        "filename": "/Users/[REDACTED]/Desktop/Fault Diagnosis System new/data/node_red/fault_alerts.jsonl",
        "filenameType": "str",
        "appendNewline": false,
        "createDir": true,
        "overwriteFile": "false",
        "encoding": "none",
        "x": 810,
        "y": 560,
        "wires": [
            []
        ]
    },
    {
        "id": "c086fcba17587385",
        "type": "function",
        "z": "54da57a1d2760d74",
        "name": "Log MQTT Test Alert",
        "func": "\nif (msg.topic !== \"iot/factory/motor01/alerts/test\") {\n    return null;\n}\n\nlet alert = msg.payload;\n\nif (typeof alert === \"string\") {\n    try {\n        alert = JSON.parse(alert);\n    } catch (error) {\n        return null;\n    }\n}\n\nif (!alert || alert.test !== true) {\n    return null;\n}\n\nmsg.payload = JSON.stringify({\n    received_at: new Date().toISOString(),\n    test_only: true,\n    device_id: alert.device_id,\n    severity: alert.severity,\n    fault: alert.fault,\n    actuate_hardware: false\n}) + \"\\n\";\n\nreturn msg;\n",
        "outputs": 1,
        "timeout": 0,
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 580,
        "y": 260,
        "wires": [
            [
                "09b398d13872743b"
            ]
        ]
    },
```

node_red_flow.json (continued) (continued) (continued) (continued)

```
    {
        "id": "09b398d13872743b",
        "type": "file",
        "z": "54da57a1d2760d74",
        "name": "MQTT Test Alert History",
        "filename": "/Users/[REDACTED]/Desktop/Fault Diagnosis System
new/data/node_red/mqtt_test_alerts.jsonl",
        "filenameType": "str",
        "appendNewline": false,
        "createDir": true,
        "overwriteFile": "false",
        "encoding": "none",
        "x": 850,
        "y": 260,
        "wires": [
            []
        ]
    },
    {
        "id": "565918fea3215f7c",
        "type": "mqtt-broker",
        "name": "HiveMQ Cloud",
        "broker": "1285e2f1b7e14c249b55489e3f2de0e3.s1.eu.hivemq.cloud",
        "port": "8883",
        "tls": "a2b642b66c12e81a",
        "clientid": "",
        "autoConnect": true,
        "usetls": true,
        "protocolVersion": 4,
        "keepalive": 60,
        "cleansession": true,
        "autoUnsubscribe": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthRetain": "false",
        "birthPayload": "",
        "birthMsg": {},
        "closeTopic": "",
        "closeQos": "0",
        "closeRetain": "false",
        "closePayload": "",
        "closeMsg": {},
        "willTopic": "",
        "willQos": "0",
        "willRetain": "false",
        "willPayload": "",
        "willMsg": {},
        "userProps": "",
        "sessionExpiry": ""
    },
    {
        "id": "a2b642b66c12e81a",
        "type": "tls-config",
        "name": "HiveMQ TLS",
        "certType": "files",
        "cert": "",
        "key": "",
        "p12": "",
        "p12name": "",
        "ca": "",
        "certname": "",
        "keyname": "",
        "caname": "",
        "servername": "",
        "verifyservercert": true,
        "alpnprotocol": "",
        "certEnv": "",
        "keyEnv": "",
        "caEnv": ""
    }
]
```

pyproject.toml

```
[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"

[project]
name = "fault-diagnosis-system"
version = "1.0.0"
description = "Knowledge-Driven IoT-Based Fault Diagnosis System for Industrial 12V DC Motors"
authors = [{ name = "IoT & AI Engineering Team" }]
readme = "README.md"
requires-python = ">=3.10"
dependencies = [
    "paho-mqtt>=2.1.0",
    "pandas>=3.0.0",
    "numpy>=2.0.0",
    "scikit-learn>=1.6.0",
    "scipy>=1.15.0",
    "streamlit>=1.40.0",
    "fastapi>=0.115.0",
    "uvicorn>=0.30.0",
    "pydantic>=2.10.0",
    "python-dotenv>=1.0.0",
    "pyyaml>=6.0.0",
    "requests>=2.32.0",
    "plotly>=6.0.0",
    "joblib>=1.4.0",
]

[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["."]
addopts = "-v --tb=short"

```

requirements.txt

```
paho-mqtt>=2.1.0
pandas>=3.0.0
numpy>=2.0.0
scikit-learn>=1.6.0
scipy>=1.15.0
streamlit>=1.40.0
fastapi>=0.115.0
uvicorn>=0.30.0
pytest>=8.0.0
pydantic>=2.10.0
python-dotenv>=1.0.0
pyyaml>=6.0.0
requests>=2.32.0
plotly>=6.0.0
joblib>=1.4.0

```

Final_ARD_Code/Final_ARD_Code.ino

```

#define IR_SENSOR 27
#define IN1 25
#define IN2 26

volatile unsigned long pulseCount = 0;
volatile unsigned long lastPulseTime = 0;

unsigned long lastUpdate = 0;
unsigned long motorStartTime = 0;

bool motorRunning = false;

const int pulsesPerRevolution = 1;
const unsigned long maxRunTime = 6000;

void IRAM_ATTR countPulse() {
  unsigned long now = micros();

  if (now - lastPulseTime > 15000) {
    pulseCount++;
    lastPulseTime = now;
  }
}

void stopMotor() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);

  motorRunning = false;
  Serial.println("Motor STOPPED");
}

void startMotor() {
  noInterrupts();
  pulseCount = 0;
  interrupts();

  lastUpdate = millis();
  motorStartTime = millis();
  motorRunning = true;

  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);

  Serial.println("Motor STARTED");
}

void setup() {
  Serial.begin(115200);

  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(IR_SENSOR, INPUT);

  stopMotor();

  attachInterrupt(
    digitalPinToInterrupt(IR_SENSOR),
    countPulse,
    FALLING
  );

  Serial.println("FactoryPulse AI READY");
  Serial.println("Type START or STOP");
}

void loop() {
  if (Serial.available()) {
    String command = Serial.readStringUntil('\n');
    command.trim();
```

Final_ARD_Code/Final_ARD_Code.ino (continued)

```
  72      command.toUpperCase();
  73
  74      if (command == "START") {
  75        startMotor();
  76      }
  77
  78      if (command == "STOP") {
  79        stopMotor();
  80      }
  81    }
  82
  83    unsigned long now = millis();
  84
  85    if (motorRunning) {
  86      if (now - lastUpdate >= 2000) {
  87        unsigned long elapsed = now - lastUpdate;
  88
  89        noInterrupts();
  90        unsigned long pulses = pulseCount;
  91        pulseCount = 0;
  92        interrupts();
  93
  94        float rpm = (pulses * 60000.0) /
  95                    (elapsed * pulsesPerRevolution);
  96
  97        Serial.print("Motor RPM: ");
  98        Serial.println(rpm, 1);
  99
 100        lastUpdate = now;
 101      }
 102
 103      if (now - motorStartTime >= maxRunTime) {
 104        stopMotor();
 105      }
 106    }
 107  }
```

Final_ARD_Code/secrets.h

```
#pragma once

const char* WIFI_SSID = "[REDACTED]";
const char* WIFI_PASSWORD = "[REDACTED]";
const char* MQTT_USER = "[REDACTED]";
const char* MQTT_PASSWORD = "[REDACTED]";
```

backend/auth/security.py

```python
"""
Authentication and Cryptographic Security Module.
Implements PBKDF2-HMAC-SHA256 password hashing, session tokens, and brute-force rate limiting.
"""

import hashlib
import secrets
import time
from datetime import datetime, timezone
from typing import Optional, Dict, Any, Tuple
from database.connection import db_manager


class AuthSecurityManager:
    """
    Manages user authentication, password hashing, and login rate limiting.
    """

    def __init__(self, max_attempts: int = 5, lock_duration_sec: int = 300):
        self.max_attempts = max_attempts
        self.lock_duration_sec = lock_duration_sec
        self._failed_attempts: Dict[str, list] = {}  # username -> list of timestamps
        self._active_sessions: Dict[str, Dict[str, Any]] = {}  # token -> session dict

    @staticmethod
    def hash_password(password: str, salt: Optional[str] = None) -> Tuple[str, str]:
        """Hashes password with PBKDF2-HMAC-SHA256 and unique 16-byte salt."""
        if salt is None:
            salt = secrets.token_hex(16)
        pwd_hash = hashlib.pbkdf2_hmac(
            "sha256",
            password.encode("utf-8"),
            salt.encode("utf-8"),
            100_000,
        ).hex()
        return pwd_hash, salt

    @staticmethod
    def verify_password(password: str, password_hash: str, salt: str) -> bool:
        """Verifies candidate password against stored hash using constant-time comparison."""
        candidate_hash, _ = AuthSecurityManager.hash_password(password, salt=salt)
        return secrets.compare_digest(candidate_hash, password_hash)

    def _is_rate_limited(self, username: str) -> bool:
        """Checks if account is temporarily locked due to repeated failed logins."""
        now = time.time()
        attempts = self._failed_attempts.get(username, [])
        # Filter attempts within the lock duration window
        recent = [t for t in attempts if now - t < self.lock_duration_sec]
        self._failed_attempts[username] = recent
        return len(recent) >= self.max_attempts

    def _record_failure(self, username: str) -> None:
        attempts = self._failed_attempts.setdefault(username, [])
        attempts.append(time.time())

    def _clear_failures(self, username: str) -> None:
        self._failed_attempts.pop(username, None)

    def create_user(self, username: str, password: str, role: str = "operator") -> bool:
        """Registers a new user into SQLite users table."""
        if len(password) < 6:
            raise ValueError("Password must be at least 6 characters long.")

        pwd_hash, salt = self.hash_password(password)
        now_utc = datetime.now(timezone.utc).isoformat()
        query = """
        INSERT INTO users (username, password_hash, salt, role, is_active, created_at)
        VALUES (?, ?, ?, ?, 1, ?)
        """
        try:
```

backend/auth/security.py (continued)

```
            with db_manager.session() as conn:
                conn.execute(query, (username.strip().lower(), pwd_hash, salt, role, now_utc))
                return True
        except Exception:
            return False

    def authenticate_user(self, username: str, password: str) -> Optional[Dict[str, Any]]:
        """
        Validates credentials and returns session payload if authenticated.
        Returns None on invalid credentials or rate limiting.
        """
        user_key = username.strip().lower()
        if self._is_rate_limited(user_key):
            return {"error": "Account temporarily locked due to excessive failed attempts. Please wait 5 minutes."}

        query = "SELECT id, username, password_hash, salt, role, is_active FROM users WHERE username = ?"
        with db_manager.session() as conn:
            cursor = conn.execute(query, (user_key,))
            row = cursor.fetchone()

        if not row or not row["is_active"]:
            self._record_failure(user_key)
            return None

        if not self.verify_password(password, row["password_hash"], row["salt"]):
            self._record_failure(user_key)
            return None

        # Successful login
        self._clear_failures(user_key)
        now_utc = datetime.now(timezone.utc).isoformat()
        with db_manager.session() as conn:
            conn.execute("UPDATE users SET last_login = ? WHERE id = ?", (now_utc, row["id"]))

        # Generate secure session token
        token = secrets.token_urlsafe(32)
        session_data = {
            "token": token,
            "user_id": row["id"],
            "username": row["username"],
            "role": row["role"],
            "login_time": now_utc,
        }
        self._active_sessions[token] = session_data
        return session_data

    def validate_session(self, token: str) -> Optional[Dict[str, Any]]:
        """Returns session data if token is active."""
        return self._active_sessions.get(token)

    def logout_session(self, token: str) -> None:
        """Invalidates an active session."""
        self._active_sessions.pop(token, None)


auth_manager = AuthSecurityManager()

```

backend/main.py

```python
"""
FastAPI Backend API Application.
Provides authenticated REST endpoints for sensor telemetry, historical analytics,
alerts management, system health, and simulation scenario controls.
"""

import sys
from pathlib import Path
from typing import Optional, Dict, Any, List

# Add project root to sys.path
sys.path.insert(0, str(Path(__file__).resolve().parent.parent))

from fastapi import FastAPI, HTTPException, Header, Depends, Query
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel

from config.settings import settings
from database.repository import repo
from backend.auth.security import auth_manager
from ml.predict import predictor

app = FastAPI(
    title="Knowledge-Driven IoT Fault Diagnosis API",
    version="1.0.0",
    description="REST API for 12V Industrial DC Motor Telemetry, Expert Rules & AI Diagnosis",
)

# CORS middleware for Streamlit / external UI integration
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)


# --- Request Models ---
class LoginRequest(BaseModel):
    username: str
    password: str


class ScenarioRequest(BaseModel):
    device_id: str = "motor01"
    scenario: str


class AlertAckRequest(BaseModel):
    username: Optional[str] = "operator"


# --- Dependency: Auth Header Validation ---
def get_current_user(authorization: Optional[str] = Header(None)) -> Dict[str, Any]:
    if not authorization:
        raise HTTPException(status_code=401, detail="Missing Authorization Header")
    token = authorization.replace("Bearer ", "").strip()
    session = auth_manager.validate_session(token)
    if not session:
        raise HTTPException(status_code=401, detail="Invalid or expired session token")
    return session


# --- Health & Status ---
@app.get("/api/health")
def get_health():
    mqtt_cfg = settings.get_effective_mqtt_config()
    return {
        "status": "healthy",
        "environment": settings.ENVIRONMENT,
```

backend/main.py (continued)

```
        "operation_mode": settings.OPERATION_MODE,
        "mqtt_broker": {
            "type": mqtt_cfg["broker_type"],
            "host": mqtt_cfg["host"],
            "port": mqtt_cfg["port"],
            "tls": mqtt_cfg["use_tls"],
        },
        "database": {
            "type": "SQLite WAL",
            "path": str(settings.DATABASE_PATH),
            "accessible": settings.DATABASE_PATH.exists(),
        },
        "ml_model": {
            "loaded": predictor.is_loaded,
            "classes": predictor.classes,
        },
    }


# --- Authentication ---
@app.post("/api/auth/login")
def login(req: LoginRequest):
    res = auth_manager.authenticate_user(req.username, req.password)
    if not res:
        raise HTTPException(status_code=401, detail="Invalid credentials or inactive account")
    if "error" in res:
        raise HTTPException(status_code=429, detail=res["error"])
    return res


@app.post("/api/auth/logout")
def logout(authorization: Optional[str] = Header(None)):
    if authorization:
        token = authorization.replace("Bearer ", "").strip()
        auth_manager.logout_session(token)
    return {"message": "Logged out successfully"}


# --- Telemetry & Diagnostics ---
@app.get("/api/telemetry/recent")
def get_recent_telemetry(
    device_id: str = "motor01",
    limit: int = Query(default=50, le=500),
):
    df = repo.get_recent_readings(device_id=device_id, limit=limit)
    return df.to_dict(orient="records")


@app.get("/api/telemetry/history")
def get_telemetry_history(
    device_id: str = "motor01",
    start_time: str = Query(..., description="ISO UTC Start"),
    end_time: str = Query(..., description="ISO UTC End"),
    limit: int = Query(default=1000, le=5000),
):
    df = repo.get_readings_by_time_range(device_id, start_time, end_time, limit)
    return df.to_dict(orient="records")


@app.get("/api/diagnosis/recent")
def get_recent_diagnoses(
    device_id: str = "motor01",
    limit: int = Query(default=20, le=100),
):
    df = repo.get_recent_predictions(device_id=device_id, limit=limit)
    return df.to_dict(orient="records")


# --- Alerts Management ---
@app.get("/api/alerts")
def get_alerts(
```

backend/main.py (continued) (continued)

```
    device_id: Optional[str] = None,
    unacknowledged_only: bool = False,
    limit: int = 50,
):
    df = repo.get_alerts(device_id=device_id, only_unacknowledged=unacknowledged_only, limit=limit)
    return df.to_dict(orient="records")


@app.post("/api/alerts/{alert_id}/ack")
def acknowledge_alert(
    alert_id: int,
    req: AlertAckRequest,
    user: Dict[str, Any] = Depends(get_current_user),
):
    username = user.get("username", req.username)
    ok = repo.acknowledge_alert(alert_id, user=username)
    if not ok:
        raise HTTPException(status_code=404, detail="Alert not found or update failed")
    return {"message": f"Alert {alert_id} acknowledged by {username}"}


# --- Model Evaluation Metrics ---
@app.get("/api/model/metrics")
def get_model_metrics():
    import json
    if not settings.ML_METRICS_PATH.exists():
        raise HTTPException(status_code=404, detail="Model metrics file not found")
    with open(settings.ML_METRICS_PATH, "r", encoding="utf-8") as f:
        metrics = json.load(f)
    cm = {}
    if settings.ML_CONFUSION_MATRIX_PATH.exists():
        with open(settings.ML_CONFUSION_MATRIX_PATH, "r", encoding="utf-8") as f:
            cm = json.load(f)
    return {"metrics": metrics, "confusion_matrix": cm}


if __name__ == "__main__":
    import uvicorn
    uvicorn.run("backend.main:app", host=settings.API_HOST, port=settings.API_PORT, reload=False)

```

config/devices.yaml

```yaml
# Registered Industrial Assets & Specifications

devices:
  - device_id: motor01
    name: "Primary Feed Drive Motor (Station 01)"
    type: "12V Brushed DC Motor"
    driver: "L298N Dual H-Bridge Driver"
    rated_voltage_v: 12.0
    rated_current_a: 0.80
    stall_current_a: 2.20
    no_load_speed_rpm: 2350
    rated_speed_rpm: 2000
    rated_torque_kgcm: 1.5
    installed_sensors:
      - type: "DS18B20"
        parameter: "temperature_c"
        interface: "1-Wire GPIO4"
      - type: "MPU6050"
        parameter: "vibration_rms_g"
        interface: "I2C (SDA 21, SCL 22)"
      - type: "ACS712-05B"
        parameter: "current_a"
        interface: "ADC1 CH0 (GPIO36) with divider"
      - type: "IR Optical Slot Sensor"
        parameter: "rpm"
        interface: "Interrupt GPIO18"
    location: "Machining Cell 1 - Primary Line"
    created_at: "2026-10-30T00:00:00Z"

```

config/knowledge_base.yaml

```yaml
# Knowledge-Driven Diagnostic Expert Rules
# Declarative Physics & Engineering Rules for 12V Industrial DC Motor System

version: "1.0.0"
domain: "12V DC Industrial Motor Fault Diagnosis"

rules:
  - rule_id: "RULE_001_SENSOR_ANOMALY"
    name: "Sensor Out-of-Range or Acquisition Error"
    priority: 100
    conditions:
      any:
        - "temperature_c < -20.0 or temperature_c > 130.0"
        - "current_a < 0.0 or current_a > 8.0"
        - "vibration_rms_g < 0.0 or vibration_rms_g > 15.0"
        - "rpm < 0.0 or rpm > 6000.0"
        - "sensor_status == 'error'"
    hypothesis:
      suspected_fault: "Sensor Acquisition Fault"
      severity: "UNKNOWN / SENSOR ERROR"
      confidence: 0.95
      explanation: "Telemetry readings violate physical bounds or signal acquisition bitflag reported hardware read failure."
      recommended_action: "Inspect sensor wiring: check DS18B20 4.7kΩ pull-up resistor, verify ACS712 ADC voltage divider (5V to 3.3V safe limit), test I2C bus lines for MPU6050."

  - rule_id: "RULE_002_MOTOR_STALL"
    name: "Locked Rotor / Mechanical Jam"
    priority: 90
    conditions:
      all:
        - "current_a >= 1.70"
        - "rpm <= 250.0"
    hypothesis:
      suspected_fault: "Motor Stall / Locked Rotor"
      severity: "CRITICAL"
      confidence: 0.96
      explanation: "Current exceeds 1.7A (approaching stall current rating of 2.2A) while motor speed drops to near zero (<250 RPM), indicating locked rotor."
      recommended_action: "Immediate de-energization advised to avoid L298N thermal shutdown or armature winding burnout. Clear mechanical blockage before restarting."

  - rule_id: "RULE_003_THERMAL_OVERHEATING"
    name: "Motor Overheating / Thermal Stress"
    priority: 80
    conditions:
      all:
        - "temperature_c >= 65.0"
        - "current_a >= 0.90"
    hypothesis:
      suspected_fault: "Motor Overheating"
      severity: "CRITICAL"
      confidence: 0.90
      explanation: "Motor casing temperature has risen above critical limit (>=65°C) accompanied by elevated current draw, leading to thermal insulation degradation."
      recommended_action: "Check airflow around cooling fins, verify ambient temperature, inspect brush gear and winding insulation resistance."

  - rule_id: "RULE_004_MECHANICAL_OVERLOAD"
    name: "Excessive Mechanical Load / Drag"
    priority: 70
    conditions:
      all:
        - "current_a >= 1.05"
        - "rpm < 1600.0"
        - "rpm > 250.0"
    hypothesis:
      suspected_fault: "Mechanical Overload"
      severity: "WARNING"
      confidence: 0.88
      explanation: "Current draw is significantly elevated (>=1.05A) while shaft rotational speed is depressed
```

config/knowledge_base.yaml (continued)

```
    below normal operating range (1600-2400 RPM)."
66        recommended_action: "Reduce mechanical transmission load, check belt tension or gearbox lubrication,
    inspect driven machinery for mechanical binding."
67
68    - rule_id: "RULE_005_EXCESSIVE_VIBRATION"
69      name: "Structural Resonance / Imbalance / Misalignment"
70      priority: 60
71      conditions:
72        all:
73          - "vibration_rms_g >= 0.40"
74          - "current_a < 1.70"
75      hypothesis:
76        suspected_fault: "Excessive Vibration"
77        severity: "WARNING"
78        confidence: 0.85
79        explanation: "Vibration RMS exceeds 0.40g threshold while current remains below stall, indicating
    mechanical unbalance, loose mounting bolts, or shaft misalignment."
80        recommended_action: "Tighten motor mounting fasteners, check shaft concentricity with laser/dial
    indicator, inspect motor bearings for spalling."
81
82    - rule_id: "RULE_006_MOTOR_SLOWDOWN"
83      name: "Motor Speed Degradation"
84      priority: 50
85      conditions:
86        all:
87          - "rpm < 1500.0"
88          - "rpm > 250.0"
89          - "current_a < 1.05"
90      hypothesis:
91        suspected_fault: "Motor Slowdown"
92        severity: "WARNING"
93        confidence: 0.80
94        explanation: "Shaft speed is below 1500 RPM despite moderate current draw, indicating low supply voltage
    or high dry friction in bearings."
95        recommended_action: "Verify 12V DC power supply rail voltage under load, inspect commutator brushes for
    wear, lubricate sleeve/ball bearings."
96
97    - rule_id: "RULE_007_NORMAL_OPERATION"
98      name: "Healthy Baseline State"
99      priority: 10
100     conditions:
101       all:
102         - "temperature_c >= 20.0 and temperature_c < 55.0"
103         - "vibration_rms_g < 0.35"
104         - "current_a >= 0.30 and current_a < 0.95"
105         - "rpm >= 1700.0"
106     hypothesis:
107       suspected_fault: "Normal"
108       severity: "NORMAL"
109       confidence: 0.98
110       explanation: "All monitored telemetry parameters (temperature, vibration, current, speed) are well within
     nominal rated design envelopes."
111       recommended_action: "No corrective intervention required. Continue routine condition-monitoring
    schedule."
112
```

config/settings.py

```python
"""
Configuration and Environment Settings Module
Provides centralized, strongly-typed settings with automatic local broker fallback.
"""

import os
from pathlib import Path
from typing import Optional
from dotenv import load_dotenv

# Base Directory Resolution
BASE_DIR = Path(__file__).resolve().parent.parent
load_dotenv(BASE_DIR / ".env")


class Settings:
    # Base paths
    PROJECT_ROOT: Path = BASE_DIR
    CONFIG_DIR: Path = BASE_DIR / "config"
    DATA_DIR: Path = BASE_DIR / "data"
    CSV_DIR: Path = DATA_DIR / "csv"
    SQLITE_DIR: Path = DATA_DIR / "sqlite"
    SYNTHETIC_DIR: Path = DATA_DIR / "synthetic"
    MODELS_DIR: Path = BASE_DIR / "ml" / "models"
    DOCS_DIR: Path = BASE_DIR / "docs"

    # Environment & Device
    ENVIRONMENT: str = os.getenv("ENVIRONMENT", "development")
    LOG_LEVEL: str = os.getenv("LOG_LEVEL", "INFO")
    SECRET_KEY: str = os.getenv("SECRET_KEY", "default-dev-secret-key-change-in-production-12345678")
    OPERATION_MODE: str = os.getenv("OPERATION_MODE", "simulation").lower()
    DEVICE_ID: str = os.getenv("DEVICE_ID", "motor01")

    # MQTT Settings
    USE_LOCAL_BROKER: bool = os.getenv("USE_LOCAL_BROKER", "true").lower() in ("true", "1", "yes")
    LOCAL_MQTT_HOST: str = os.getenv("LOCAL_MQTT_HOST", "127.0.0.1")
    LOCAL_MQTT_PORT: int = int(os.getenv("LOCAL_MQTT_PORT", "1883"))

    # HiveMQ Cloud Settings
    HIVEMQ_HOST: Optional[str] = os.getenv("HIVEMQ_HOST", "").strip() or None
    HIVEMQ_PORT: int = int(os.getenv("HIVEMQ_PORT", "8883"))
    HIVEMQ_USERNAME: Optional[str] = os.getenv("HIVEMQ_USERNAME", "").strip() or None
    HIVEMQ_PASSWORD: Optional[str] = os.getenv("HIVEMQ_PASSWORD", "").strip() or None
    HIVEMQ_USE_TLS: bool = os.getenv("HIVEMQ_USE_TLS", "true").lower() in ("true", "1", "yes")

    # Storage Paths
    DATABASE_PATH: Path = BASE_DIR / os.getenv("DATABASE_PATH", "data/sqlite/fault_diagnosis.db")
    CSV_LOG_PATH: Path = BASE_DIR / os.getenv("CSV_LOG_PATH", "data/csv/sensor_telemetry.csv")
    INGESTION_PATH: str = os.getenv("INGESTION_PATH", "mqtt").lower()

    # REST API & Dashboard
    API_HOST: str = os.getenv("API_HOST", "0.0.0.0")
    API_PORT: int = int(os.getenv("API_PORT", "8000"))
    DASHBOARD_PORT: int = int(os.getenv("DASHBOARD_PORT", "8501"))

    # Node-RED
    NODERED_ENABLED: bool = os.getenv("NODERED_ENABLED", "true").lower() in ("true", "1", "yes")
    NODERED_HOST: str = os.getenv("NODERED_HOST", "127.0.0.1")
    NODERED_PORT: int = int(os.getenv("NODERED_PORT", "1880"))
    NODERED_API_URL: str = os.getenv("NODERED_API_URL", "http://127.0.0.1:1880")

    # ML Paths
    ML_MODEL_PATH: Path = BASE_DIR / os.getenv("ML_MODEL_PATH", "ml/models/random_forest_model.joblib")
    ML_METRICS_PATH: Path = BASE_DIR / os.getenv("ML_METRICS_PATH", "ml/models/model_metrics.json")
    ML_CONFUSION_MATRIX_PATH: Path = BASE_DIR / os.getenv("ML_CONFUSION_MATRIX_PATH",
"ml/models/confusion_matrix.json")

    # Config File Paths
    THRESHOLDS_FILE: Path = BASE_DIR / "config" / "thresholds.yaml"
    DEVICES_FILE: Path = BASE_DIR / "config" / "devices.yaml"
    KNOWLEDGE_BASE_FILE: Path = BASE_DIR / "config" / "knowledge_base.yaml"
```

config/settings.py (continued)

```
71
72      @property
73      def is_hivemq_configured(self) -> bool:
74          """Returns True only if valid non-empty HiveMQ host and credentials exist."""
75          return bool(
76              self.HIVEMQ_HOST
77              and not self.USE_LOCAL_BROKER
78              and self.HIVEMQ_USERNAME
79              and self.HIVEMQ_PASSWORD
80          )
81
82      def get_effective_mqtt_config(self) -> dict:
83          """
84          Determines whether to use HiveMQ Cloud or fall back to local broker.
85          Never throws when cloud credentials are unconfigured.
86          """
87          if self.is_hivemq_configured:
88              return {
89                  "broker_type": "hivemq_cloud",
90                  "host": self.HIVEMQ_HOST,
91                  "port": self.HIVEMQ_PORT,
92                  "username": self.HIVEMQ_USERNAME,
93                  "password": self.HIVEMQ_PASSWORD,
94                  "use_tls": self.HIVEMQ_USE_TLS,
95              }
96          return {
97              "broker_type": "local_fallback",
98              "host": self.LOCAL_MQTT_HOST,
99              "port": self.LOCAL_MQTT_PORT,
100             "username": None,
101             "password": None,
102             "use_tls": False,
103         }
104
105     # MQTT Topic Helpers
106     @staticmethod
107     def get_sensor_topic(device_id: str = "motor01") -> str:
108         return f"iot/factory/{device_id}/sensors"
109
110     @staticmethod
111     def get_status_topic(device_id: str = "motor01") -> str:
112         return f"iot/factory/{device_id}/status"
113
114     @staticmethod
115     def get_ground_truth_topic(device_id: str = "motor01") -> str:
116         return f"iot/factory/{device_id}/ground_truth"
117
118     @staticmethod
119     def get_diagnosis_topic(device_id: str = "motor01") -> str:
120         return f"iot/factory/{device_id}/diagnosis"
121
122     @staticmethod
123     def get_alerts_topic(device_id: str = "motor01") -> str:
124         return f"iot/factory/{device_id}/alerts"
125
126     @staticmethod
127     def get_commands_topic(device_id: str = "motor01") -> str:
128         return f"iot/factory/{device_id}/commands"
129
130     @staticmethod
131     def get_command_ack_topic(device_id: str = "motor01") -> str:
132         return f"iot/factory/{device_id}/command_ack"
133
134
135 settings = Settings()
136
137 # Ensure directories exist
138 for directory in [
139     settings.DATA_DIR,
140     settings.CSV_DIR,
141     settings.SQLITE_DIR,
```

config/settings.py (continued) (continued)

```
    settings.SYNTHETIC_DIR,
    settings.MODELS_DIR,
]:
    directory.mkdir(parents=True, exist_ok=True)

```

config/thresholds.yaml

```yaml
# Engineering Operating Thresholds for 12V DC Motor Diagnostic System
# Units:
# - Temperature: Celsius (°C)
# - Vibration: RMS Acceleration (g)
# - Current: Amperes (A)
# - Speed: Revolutions Per Minute (RPM)

temperature:
  normal_min: 20.0
  normal_max: 50.0
  warning_high: 60.0
  critical_high: 75.0
  sensor_error_min: -20.0
  sensor_error_max: 130.0
  hysteresis_c: 2.0

vibration:
  normal_min: 0.02
  normal_max: 0.25
  warning_high: 0.40
  critical_high: 0.70
  sensor_error_max: 16.0
  hysteresis_g: 0.03

current:
  normal_min: 0.30
  normal_max: 0.85
  warning_high: 1.10
  critical_high: 1.70
  stall_threshold: 1.80
  sensor_error_min: 0.0
  sensor_error_max: 10.0
  hysteresis_a: 0.08

rpm:
  normal_min: 1800.0
  normal_max: 2400.0
  slowdown_warning: 1400.0
  stall_max: 200.0
  sensor_error_max: 5000.0
  hysteresis_rpm: 50.0

persistence:
  consecutive_readings_required: 3
  cooldown_seconds: 15
  heartbeat_timeout_seconds: 10

```

dashboard/app.py

```python
"""
Streamlit Multi-Page Main Application.
Handles user session authentication, dark industrial CSS injection, and sidebar navigation.
"""

import sys
from pathlib import Path

# Add project root to sys.path
sys.path.insert(0, str(Path(__file__).resolve().parent.parent))

import streamlit as st
from config.settings import settings
from backend.auth.security import auth_manager

st.set_page_config(
    page_title="IoT Fault Diagnosis System",
    page_icon="⚡",
    layout="wide",
    initial_sidebar_state="expanded",
)

# Load custom CSS
css_path = Path(__file__).parent / "styles" / "theme.css"
if css_path.exists():
    with open(css_path, "r", encoding="utf-8") as f:
        st.markdown(f"<style>{f.read()}</style>", unsafe_allow_html=True)

# Session state initialization
if "authenticated" not in st.session_state:
    st.session_state["authenticated"] = False
if "username" not in st.session_state:
    st.session_state["username"] = None
if "role" not in st.session_state:
    st.session_state["role"] = None


def render_login_form():
    st.markdown("<h1 style='text-align: center; margin-top: 50px;'>⚡ Industrial Motor Diagnostic Portal</h1>", unsafe_allow_html=True)
    st.markdown("<p style='text-align: center; color: #8b949e;'>Knowledge-Driven IoT Fault Diagnosis System — 12V DC Motor Asset</p>", unsafe_allow_html=True)

    col1, col2, col3 = st.columns([1, 1.5, 1])
    with col2:
        st.markdown("<div class='metric-container'>", unsafe_allow_html=True)
        st.subheader("Secure Operator Login")
        username = st.text_input("Username", key="login_username")
        password = st.text_input("Password", type="password", key="login_password")

        if st.button("Sign In", type="primary", use_container_width=True):
            if username and password:
                session = auth_manager.authenticate_user(username, password)
                if session and "error" not in session:
                    st.session_state["authenticated"] = True
                    st.session_state["username"] = session["username"]
                    st.session_state["role"] = session["role"]
                    st.session_state["token"] = session["token"]
                    st.success(f"Welcome, {session['username']} ({session['role']})!")
                    st.rerun()
                elif session and "error" in session:
                    st.error(session["error"])
                else:
                    st.error("Invalid username or password.")
            else:
                st.warning("Please enter both username and password.")

        st.markdown("---")
        st.caption("Default local credentials:")
        st.caption("• **Admin**: `[REDACTED]` / `[REDACTED]`")
        st.caption("• **Operator**: `operator` / `Operator@2026`")
```

dashboard/app.py (continued)

```
        st.markdown("</div>", unsafe_allow_html=True)


def main():
    if not st.session_state["authenticated"]:
        render_login_form()
        return

    # Sidebar Header
    st.sidebar.markdown(f"### ⚡ Motor Condition Monitor")
    st.sidebar.markdown(f"**Operator:** `{st.session_state['username']}` ({st.session_state['role']})")

    # Connection badge
    mqtt_cfg = settings.get_effective_mqtt_config()
    broker_label = "HiveMQ Cloud (TLS)" if mqtt_cfg["broker_type"] == "hivemq_cloud" else "Local Mosquitto (1883)"
    st.sidebar.caption(f"Broker: **{broker_label}**")
    st.sidebar.caption(f"Mode: **{settings.OPERATION_MODE.upper()}**")

    if st.sidebar.button("Sign Out", use_container_width=True):
        st.session_state["authenticated"] = False
        st.session_state["username"] = None
        st.session_state["role"] = None
        st.rerun()

    st.sidebar.markdown("---")
    st.sidebar.markdown("#### Navigation")
    st.sidebar.info("Select a view from the sidebar pages above to inspect live telemetry, rule explanations, or machine learning diagnostics.")

    # Redirect to Overview by default
    st.title("⚡ Knowledge-Driven Industrial Fault Diagnosis System")
    st.markdown("""
    Welcome to the condition-monitoring and diagnostic platform for industrial 12V DC motor assets.
    Use the navigation menu in the left sidebar to explore:
    - **1 Overview**: Real-time asset health KPIs, operating states, and active hypotheses.
    - **2 Live Monitoring**: High-frequency multi-sensor trend charts and scenario controls.
    - **3 Fault Diagnosis**: Hybrid knowledge-driven rule reasoning and Random Forest predictions.
    - **4 Historical Analytics**: Query persistent SQLite records, multi-sensor overlays, and CSV export.
    - **5 Alerts**: Active alarm queue with operator acknowledgement workflows.
    - **6 Settings**: MQTT broker configuration, thresholds editor, and hardware/simulation toggle.
    - **7 Model Performance**: Confusion matrix, classification metrics, and synthetic limitations.
    """)


if __name__ == "__main__":
    main()

```

dashboard/components/charts.py

```
"""
Plotly Chart Components for Industrial Monitoring.
Dark theme styling matching industrial SCADA / APM dashboards.
"""

from typing import List, Optional
import pandas as pd
import plotly.graph_objects as go
from plotly.subplots import make_subplots


CHART_LAYOUT_DEFAULTS = dict(
    template="plotly_dark",
    paper_bgcolor="#0d1117",
    plot_bgcolor="#161b22",
    margin=dict(l=40, r=30, t=40, b=40),
    font=dict(color="#f0f6fc", family="Inter, -apple-system, sans-serif"),
    legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="right", x=1),
)


def create_live_telemetry_chart(df: pd.DataFrame) -> go.Figure:
    """Creates a 4-row synchronized time-series subplot for Temp, Vibration, Current, RPM."""
    if df.empty:
        fig = go.Figure()
        fig.update_layout(**CHART_LAYOUT_DEFAULTS, title="No Telemetry Data Available")
        return fig

    # Ensure timestamp conversion
    timestamps = pd.to_datetime(
    df["timestamp"],
    format="ISO8601",
    utc=True,
    errors="coerce"
)

    fig = make_subplots(
        rows=4,
        cols=1,
        shared_xaxes=True,
        vertical_spacing=0.06,
        subplot_titles=(
            "Temperature (°C)",
            "Vibration RMS (g)",
            "Motor Current (A)",
            "Speed (RPM)",
        ),
    )

    # 1. Temperature
    fig.add_trace(
        go.Scatter(
            x=timestamps,
            y=df["temperature_c"],
            mode="lines",
            name="Temperature",
            line=dict(color="#f85149", width=2),
        ),
        row=1,
        col=1,
    )

    # 2. Vibration
    fig.add_trace(
        go.Scatter(
            x=timestamps,
            y=df["vibration_rms_g"],
            mode="lines",
            name="Vibration",
            line=dict(color="#39c5bb", width=2),
        ),
```

dashboard/components/charts.py (continued)

```
        row=2,
        col=1,
    )

    # 3. Current
    fig.add_trace(
        go.Scatter(
            x=timestamps,
            y=df["current_a"],
            mode="lines",
            name="Current",
            line=dict(color="#d29922", width=2),
        ),
        row=3,
        col=1,
    )

    # 4. RPM
    fig.add_trace(
        go.Scatter(
            x=timestamps,
            y=df["rpm"],
            mode="lines",
            name="RPM",
            line=dict(color="#58a6ff", width=2),
        ),
        row=4,
        col=1,
    )

    layout_config = {
        **CHART_LAYOUT_DEFAULTS,
        "height": 650,
        "showlegend": False,
    }
    fig.update_layout(**layout_config)
    return fig


def create_gauge(
    value: float,
    min_val: float,
    max_val: float,
    title: str,
    unit: str,
    warning_thresh: float,
    crit_thresh: float,
) -> go.Figure:
    """Builds a circular gauge indicator with consolidated layout and ordered threshold zones."""
    # Handle both normal thresholds (higher is worse) and inverted thresholds (lower is worse, e.g. RPM)
    if warning_thresh < crit_thresh:
        # Standard: Normal [min -> warn], Warning [warn -> crit], Critical [crit -> max]
        steps = [
            {"range": [min_val, warning_thresh], "color": "rgba(63, 185, 80, 0.25)"},
            {"range": [warning_thresh, crit_thresh], "color": "rgba(210, 153, 34, 0.35)"},
            {"range": [crit_thresh, max_val], "color": "rgba(248, 81, 73, 0.45)"},
        ]
    else:
        # Inverted (e.g. RPM): Critical [min -> crit], Warning [crit -> warn], Normal [warn -> max]
        steps = [
            {"range": [min_val, crit_thresh], "color": "rgba(248, 81, 73, 0.45)"},
            {"range": [crit_thresh, warning_thresh], "color": "rgba(210, 153, 34, 0.35)"},
            {"range": [warning_thresh, max_val], "color": "rgba(63, 185, 80, 0.25)"},
        ]

    fig = go.Figure(
        go.Indicator(
            mode="gauge+number",
            value=value,
            title={"text": f"{title} ({unit})", "font": {"size": 15, "color": "#8b949e"}},
            number={"suffix": f" {unit}", "font": {"size": 22, "color": "#f0f6fc"}},
```

dashboard/components/charts.py (continued) (continued)

```
            gauge={
                "axis": {"range": [min_val, max_val], "tickcolor": "#8b949e"},
                "bar": {"color": "#58a6ff"},
                "bgcolor": "#161b22",
                "borderwidth": 1,
                "bordercolor": "#30363d",
                "steps": steps,
            },
        )
    )

    # Consolidate layout configuration: override defaults in dict to guarantee single 'margin' argument
    gauge_layout = {
        **CHART_LAYOUT_DEFAULTS,
        "height": 180,
        "margin": dict(l=20, r=20, t=30, b=10),
    }
    fig.update_layout(**gauge_layout)
    return fig


def create_confusion_matrix_heatmap(labels: List[str], matrix: List[List[int]]) -> go.Figure:
    """Renders confusion matrix heatmap."""
    fig = go.Figure(
        data=go.Heatmap(
            z=matrix,
            x=labels,
            y=labels,
            colorscale="Blues",
            showscale=True,
            text=matrix,
            texttemplate="%{text}",
            textfont={"size": 12, "color": "white"},
        )
    )
    heatmap_layout = {
        **CHART_LAYOUT_DEFAULTS,
        "title": "Confusion Matrix (Evaluation Holdout Set)",
        "xaxis_title": "Predicted Label",
        "yaxis_title": "True Ground-Truth Label",
        "height": 450,
    }
    fig.update_layout(**heatmap_layout)
    return fig

```

dashboard/pages/1_Overview.py

```python
"""
Overview Page — Executive Asset Health, KPIs, and Connectivity.
"""

import sys
from pathlib import Path

# Add project root to sys.path
sys.path.insert(0, str(Path(__file__).resolve().parent.parent.parent))

import streamlit as st
import pandas as pd
from datetime import datetime, timezone
from config.settings import settings
from database.repository import repo
from dashboard.components.charts import create_gauge

st.set_page_config(page_title="Overview | Fault Diagnosis", page_icon="⚡", layout="wide")

if not st.session_state.get("authenticated", False):
    st.warning("Please sign in from the main portal.")
    st.stop()

st.title("🏭 Executive Asset Overview — Motor 01")
st.markdown("Live condition summary for 12V Brushed DC Feed Motor (Cell 1, Station 01).")

# Fetch latest telemetry and diagnosis from SQLite
readings_df = repo.get_recent_readings(device_id="motor01", limit=1)
diag_df = repo.get_recent_predictions(device_id="motor01", limit=1)
alerts_df = repo.get_alerts(device_id="motor01", only_unacknowledged=True, limit=5)

latest_reading = readings_df.iloc[-1].to_dict() if not readings_df.empty else None
latest_diag = diag_df.iloc[0].to_dict() if not diag_df.empty else None

# Status Header Bar
status_col1, status_col2, status_col3 = st.columns([2, 1, 1])

with status_col1:
    if latest_reading:
        ts = latest_reading["timestamp"]
        st.markdown(f"**Last Telemetry Event:** `{ts}`")
    else:
        st.info("No telemetry received yet. Ensure generator or ESP32 is running.")

with status_col2:
    mqtt_cfg = settings.get_effective_mqtt_config()
    broker_name = "HiveMQ Cloud" if mqtt_cfg["broker_type"] == "hivemq_cloud" else "Local Broker"
    st.markdown(f"**Broker:** `{broker_name}`")

with status_col3:
    mode_badge = "🔄 SIMULATION" if settings.OPERATION_MODE == "simulation" else "🔌 REAL HARDWARE"
    st.markdown(f"**Mode:** {mode_badge}")

st.markdown("---")

# KPI Metric Cards
m_col1, m_col2, m_col3, m_col4 = st.columns(4)

temp_val = latest_reading["temperature_c"] if latest_reading else 0.0
vibe_val = latest_reading["vibration_rms_g"] if latest_reading else 0.0
curr_val = latest_reading["current_a"] if latest_reading else 0.0
rpm_val = latest_reading["rpm"] if latest_reading else 0.0

with m_col1:
    st.markdown(f"""
    <div class='metric-container'>
        <div class='metric-label'>Motor Temperature</div>
        <div class='metric-value'>{temp_val:.1f}<span class='metric-unit'>°C</span></div>
    </div>
    """, unsafe_allow_html=True)

```

dashboard/pages/1_Overview.py (continued)

```
with m_col2:
    st.markdown(f"""
    <div class='metric-container'>
        <div class='metric-label'>Vibration RMS</div>
        <div class='metric-value'>{"N/A" if vibe_val is None else format(vibe_val, ".3f")}<span class='metric-unit'>g</span></div>
    </div>
    """, unsafe_allow_html=True)

with m_col3:
    st.markdown(f"""
    <div class='metric-container'>
        <div class='metric-label'>Armature Current</div>
        <div class='metric-value'>{curr_val:.3f}<span class='metric-unit'>A</span></div>
    </div>
    """, unsafe_allow_html=True)

with m_col4:
    st.markdown(f"""
    <div class='metric-container'>
        <div class='metric-label'>Rotational Speed</div>
        <div class='metric-value'>{rpm_val:.0f}<span class='metric-unit'>RPM</span></div>
    </div>
    """, unsafe_allow_html=True)

# Active Diagnostic Hypothesis Banner
st.subheader("Diagnostic Status")
if latest_diag:
    sev = latest_diag["hybrid_severity"]
    fault = latest_diag["hybrid_fault"]
    explanation = latest_diag.get("explanation", "")
    rec = latest_diag.get("recommended_action", "")

    badge_class = {
        "NORMAL": "status-normal",
        "WARNING": "status-warning",
        "CRITICAL": "status-critical",
    }.get(sev, "status-unknown")

    st.markdown(f"""
    <div class='diagnosis-box'>
        <h4><span class='status-pill {badge_class}'>{sev}</span>   {fault}</h4>
        <p style='color: #c9d1d9; margin-top: 8px;'>{explanation}</p>
        <p style='color: #58a6ff; font-weight: 600;'>Recommended Action: {rec}</p>
    </div>
    """, unsafe_allow_html=True)
else:
    st.info("Awaiting diagnostic inference results...")

# Gauges Section
st.subheader("Operational Envelopes")
g_col1, g_col2, g_col3, g_col4 = st.columns(4)

with g_col1:
    st.plotly_chart(create_gauge(temp_val, 0, 100, "Temperature", "°C", 60.0, 75.0), use_container_width=True)
with g_col2:
    st.plotly_chart(create_gauge(vibe_val, 0, 1.0, "Vibration", "g", 0.40, 0.70), use_container_width=True)
with g_col3:
    st.plotly_chart(create_gauge(curr_val, 0, 2.5, "Current", "A", 1.10, 1.70), use_container_width=True)
with g_col4:
    st.plotly_chart(create_gauge(rpm_val, 0, 3000, "Speed", "RPM", 1400, 200), use_container_width=True)

# Quick Unacknowledged Alerts
st.subheader(" Active Unacknowledged Alerts")
if not alerts_df.empty:
    for _, alert in alerts_df.iterrows():
        st.warning(f"**[{alert['severity']}] {alert['title']}** — {alert['message']} ({alert['timestamp']})")
else:
    st.success(" No unacknowledged alarms. Operating within healthy tolerances.")

```

dashboard/pages/2_Live_Monitoring.py

```
"""
Live Monitoring Page — Real-Time Telemetry Streaming & Scenario Controls.
"""

import sys
import time
from pathlib import Path

# Add project root to sys.path
sys.path.insert(0, str(Path(__file__).resolve().parent.parent.parent))

import streamlit as st
import pandas as pd
from config.settings import settings
from database.repository import repo
from simulation.scenarios import MotorScenario
from ingestion.mqtt_client import ManagedMQTTClient
from dashboard.components.charts import create_live_telemetry_chart

st.set_page_config(page_title="Live Monitoring | Fault Diagnosis", page_icon="⚡", layout="wide")

if not st.session_state.get("authenticated", False):
    st.warning("Please sign in from the main portal.")
    st.stop()

st.title("📡 Live Telemetry Monitoring — Motor 01")

# Scenario Injection Toolbar
with st.expander("🎮 Simulation Scenario & Fault Injection Controls", expanded=True):
    ctrl_col1, ctrl_col2, ctrl_col3 = st.columns([2, 1, 1])

    with ctrl_col1:
        selected_scenario = st.selectbox(
            "Inject Fault Scenario into Simulation:",
            options=[s.value for s in MotorScenario],
            format_func=lambda x: f"{x.replace('_', ' ').title()}",
            key="selected_scenario_box",
        )

    with ctrl_col2:
        st.write("")
        st.write("")
        if st.button("Apply Scenario", type="primary", use_container_width=True):
            # Publish scenario command over MQTT to iot/factory/motor01/commands
            cfg = settings.get_effective_mqtt_config()
            cmd_client = ManagedMQTTClient(f"dash_cmd_{int(time.time())}", cfg)
            if cmd_client.connect(timeout=2):
                cmd_payload = {"command": "set_scenario", "scenario": selected_scenario}
                cmd_client.publish(settings.get_commands_topic("motor01"), cmd_payload)
                cmd_client.disconnect()
                st.success(f"Dispatched scenario command: '{selected_scenario}'")
            else:
                st.error("Could not reach MQTT broker to send scenario command.")

    with ctrl_col3:
        st.write("")
        st.write("")
        auto_refresh = st.checkbox("Auto-Refresh (1s)", value=True)

# Fetch latest 60 readings from SQLite
readings_df = repo.get_recent_readings(device_id="motor01", limit=60)

if readings_df.empty:
    st.info("No live telemetry points stored yet. Run the publisher: `python simulation/mqtt_publisher.py`")
else:
    latest = readings_df.iloc[-1]

    # Check freshness (offline indicator)
    try:
        last_dt = pd.to_datetime(latest["timestamp"])
        now_dt = pd.Timestamp.now(tz="UTC")
```

dashboard/pages/2_Live_Monitoring.py (continued)

```
        age_seconds = (now_dt - last_dt).total_seconds()
        is_stale = age_seconds > 10.0
    except Exception:
        is_stale = False

    if is_stale:
        st.error(f"⚠ Telemetry Link Stale / Node Offline! (No reading received for {age_seconds:.0f}s)")
    else:
        st.success(f" Telemetry Stream Active (Latency: <1s | Last: {latest['timestamp']})")

    # Render synchronized 4-row live time-series chart
    fig = create_live_telemetry_chart(readings_df)
    st.plotly_chart(fig, use_container_width=True)

    # Telemetry Data Table
    with st.expander("View Raw Telemetry Log (Last 15 Frames)"):
        st.dataframe(
            readings_df.tail(15)[["timestamp", "temperature_c", "vibration_rms_g", "current_a", "rpm",
"sensor_status", "event_id"]],
            use_container_width=True,
        )

# Auto-refresh loop
if auto_refresh:
    time.sleep(1.0)
    st.rerun()

```

dashboard/pages/3_Fault_Diagnosis.py

```
"""
Fault Diagnosis Page — Knowledge-Driven Rule Engine & ML Hybrid Explanations.
"""

import sys
from pathlib import Path

# Add project root to sys.path
sys.path.insert(0, str(Path(__file__).resolve().parent.parent.parent))

import streamlit as st
import pandas as pd
from database.repository import repo
from ml.predict import predictor

st.set_page_config(page_title="Fault Diagnosis | Expert System", page_icon="⚡", layout="wide")

if not st.session_state.get("authenticated", False):
    st.warning("Please sign in from the main portal.")
    st.stop()

st.title("🔬 Knowledge-Driven Hybrid Fault Diagnosis")
st.markdown("Combines deterministic engineering physics rules with statistical Random Forest inference.")

# Retrieve latest predictions
preds_df = repo.get_recent_predictions(device_id="motor01", limit=20)

if preds_df.empty:
    st.info("No diagnostic predictions recorded yet. Telemetry will trigger automatic evaluation.")
    st.stop()

latest = preds_df.iloc[0]

# Hybrid Verdict Card
sev = latest["hybrid_severity"]
fault = latest["hybrid_fault"]
conf = float(latest.get("ml_confidence", 0.0))
conflict = bool(latest.get("conflict_detected", 0))

sev_color = {
    "NORMAL": "#3fb950",
    "WARNING": "#d29922",
    "CRITICAL": "#f85149",
}.get(sev, "#bc8cff")

st.markdown(f"""
<div style='background-color: #161b22; border-left: 6px solid {sev_color}; border-radius: 8px; padding: 20px; margin-bottom: 20px;'>
    <h2 style='margin: 0; color: #f0f6fc;'>{fault}</h2>
    <div style='margin-top: 8px;'>
        <span class='status-pill' style='background-color: {sev_color}22; color: {sev_color}; border: 1px solid {sev_color}66;'>
            SEVERITY: {sev}
        </span>
          
        <span style='color: #8b949e; font-size: 0.9rem;'>Evaluated at: <code>{latest['timestamp']}</code></span>
    </div>
    <p style='color: #c9d1d9; margin-top: 14px; font-size: 1.05rem;'>{latest.get('explanation', '')}</p>
    <div style='background-color: #0d1117; border-radius: 6px; padding: 12px; margin-top: 12px;'>
        <strong style='color: #58a6ff;'>🛠 Prescriptive Corrective Maintenance Action:</strong>
        <p style='color: #f0f6fc; margin: 4px 0 0 0;'>{latest.get('recommended_action', '')}</p>
    </div>
</div>
""", unsafe_allow_html=True)

# Diagnostic Comparison: Rules vs ML
col_rule, col_ml = st.columns(2)

with col_rule:
    st.markdown("### 📐 Physics Rule-Based Expert System")
```

dashboard/pages/3_Fault_Diagnosis.py (continued)

```
    st.markdown(f"**Suspected Condition:** `{latest['rule_fault']}`")
    st.markdown(f"**Rule Severity:** `{latest['rule_severity']}`")
    st.info(f"**Primary Rule Applied:** Knowledge Base Physics Definition")

with col_ml:
    st.markdown("### □ Random Forest Classifier")

    ml_fault = latest.get("ml_fault")

    ml_available = (
        ml_fault is not None
        and str(ml_fault).strip().lower()
        not in ("", "none", "nan", "null")
    )

    if not ml_available:
        st.markdown("**Predicted Class:** `Not Evaluated`")
        st.markdown("**Statistical Confidence:** `N/A`")

        st.info(
            "ML diagnosis unavailable for real ESP32 data. "
            "The vibration sensor is disconnected. "
            "Rule-based diagnosis is active."
        )

        st.caption("Consensus: Not Applicable")

    else:
        st.markdown(f"**Predicted Class:** `{ml_fault}`")
        st.markdown(
            f"**Statistical Confidence:** `{conf*100:.1f}%`"
        )

        st.progress(min(1.0, max(0.0, conf)))

        if conflict:
            st.warning(
                "Diagnostic Conflict: Rule and ML "
                "predictions disagree."
            )
        else:
            st.success(
                "Rule-based and ML predictions agree."
            )


st.markdown("---")

# Historical Diagnoses Table
st.subheader("□ Recent Diagnostic Audit Trail")
display_cols = [
    "timestamp",
    "hybrid_fault",
    "hybrid_severity",
    "rule_fault",
    "ml_fault",
    "ml_confidence",
    "conflict_detected",
]
st.dataframe(preds_df[display_cols], use_container_width=True)

```

dashboard/pages/4_Historical_Analytics.py

```python
"""
Historical Analytics Page — Persistent Database Queries & Data Export.
"""

import sys
from pathlib import Path
from datetime import datetime, timezone, timedelta

# Add project root to sys.path
sys.path.insert(0, str(Path(__file__).resolve().parent.parent.parent))

import streamlit as st
import pandas as pd
from database.repository import repo
from dashboard.components.charts import create_live_telemetry_chart

st.set_page_config(page_title="Historical Analytics | Database", page_icon="⚡", layout="wide")

if not st.session_state.get("authenticated", False):
    st.warning("Please sign in from the main portal.")
    st.stop()

st.title("🗄️ Historical Telemetry Analytics & Export")

# Filter Controls
f_col1, f_col2, f_col3 = st.columns([1, 1, 1])

now = datetime.now(timezone.utc)
default_start = (now - timedelta(hours=2)).date()
default_end = now.date()

with f_col1:
    start_date = st.date_input("Start Date", default_start)
with f_col2:
    end_date = st.date_input("End Date", default_end)
with f_col3:
    max_records = st.selectbox("Max Records Limit", options=[100, 500, 1000, 5000], index=1)

start_iso = f"{start_date.isoformat()}T00:00:00Z"
end_iso = f"{end_date.isoformat()}T23:59:59Z"

# Fetch filtered readings from SQLite
df = repo.get_readings_by_time_range("motor01", start_iso, end_iso, limit=max_records)

if df.empty:
    st.warning(f"No records found between {start_iso} and {end_iso}. Expanding to most recent records...")
    df = repo.get_recent_readings("motor01", limit=max_records)

if not df.empty:
    st.subheader(f"Query Results ({len(df)} records)")

    # Time-series Chart
    fig = create_live_telemetry_chart(df)
    st.plotly_chart(fig, use_container_width=True)

    # Statistical Aggregations
    st.subheader("Statistical Summary")
    stats_df = df[["temperature_c", "vibration_rms_g", "current_a", "rpm"]].describe().T
    stats_df = stats_df.rename(columns={
        "mean": "Mean", "std": "Std Dev", "min": "Min", "max": "Max"
    })
    st.dataframe(stats_df[["Mean", "Std Dev", "Min", "Max"]], use_container_width=True)

    # CSV Export Button
    csv_bytes = df.to_csv(index=False).encode("utf-8")
    st.download_button(
        label="📥 Download Historical Dataset (CSV)",
        data=csv_bytes,
        file_name=f"motor01_telemetry_{start_date}_{end_date}.csv",
        mime="text/csv",
        type="primary",
```

dashboard/pages/4_Historical_Analytics.py (continued)

```
    )
else:
    st.info("Database is currently empty. Run publisher or simulation to accumulate telemetry.")

```

dashboard/pages/5_Alerts.py

```python
"""
Alerts Page — Active Alarms, Operator Acknowledgment, and Incident Audit Trail.
"""

import sys
from pathlib import Path

# Add project root to sys.path
sys.path.insert(0, str(Path(__file__).resolve().parent.parent.parent))

import streamlit as st
import pandas as pd
from database.repository import repo

st.set_page_config(page_title="Alerts | Fault Diagnosis", page_icon="⚡", layout="wide")

if not st.session_state.get("authenticated", False):
    st.warning("Please sign in from the main portal.")
    st.stop()

st.title("🚨 Industrial Alarm & Event Management")
st.markdown("Manages threshold violations, multi-reading persistence checks, and operator incident responses.")

# Fetch Unacknowledged Alerts
unack_df = repo.get_alerts(device_id="motor01", only_unacknowledged=True, limit=20)

st.subheader(f"⚠️ Active Unacknowledged Queue ({len(unack_df)})")

if unack_df.empty:
    st.success("✅ All system alarms have been acknowledged or resolved. Nominal operating state.")
else:
    for idx, row in unack_df.iterrows():
        sev = row["severity"]
        alert_id = row["id"]

        sev_color = "#f85149" if sev == "CRITICAL" else ("#d29922" if sev == "WARNING" else "#bc8cff")

        with st.container():
            st.markdown(f"""
            <div style='background-color: #161b22; border-left: 5px solid {sev_color}; padding: 14px 18px; border-radius: 6px; margin-bottom: 10px;'>
                <div style='display: flex; justify-content: space-between;'>
                    <strong style='color: #f0f6fc; font-size: 1.1rem;'>[{sev}] {row['title']}</strong>
                    <span style='color: #8b949e; font-size: 0.85rem;'>{row['timestamp']}</span>
                </div>
                <p style='color: #c9d1d9; margin: 8px 0 0 0;'>{row['message']}</p>
            </div>
            """, unsafe_allow_html=True)

            ack_col1, ack_col2 = st.columns([1, 4])
            with ack_col1:
                if st.button(f"Acknowledge #{alert_id}", key=f"ack_btn_{alert_id}", type="secondary"):
                    user = st.session_state.get("username", "operator")
                    repo.acknowledge_alert(alert_id, user=user)
                    st.success(f"Alert #{alert_id} acknowledged by {user}.")
                    st.rerun()

st.markdown("---")

# Full Alert Audit History
st.subheader("📜 Historical Incident Audit Log")
all_alerts = repo.get_alerts(device_id="motor01", only_unacknowledged=False, limit=50)

if not all_alerts.empty:
    display_df = all_alerts[[
        "id", "timestamp", "severity", "title", "acknowledged", "acknowledged_by", "acknowledged_at"
    ]]
    st.dataframe(display_df, use_container_width=True)
else:
    st.info("No recorded alarms in audit history.")

```

dashboard/pages/6_Settings.py

```
"""
Settings & Administration Page.
Allows reviewing MQTT broker status, physical motor ratings, thresholds, and operational mode.
"""

import sys
import yaml
from pathlib import Path

# Add project root to sys.path
sys.path.insert(0, str(Path(__file__).resolve().parent.parent.parent))

import streamlit as st
from config.settings import settings
from database.connection import db_manager

st.set_page_config(page_title="Settings | Administration", page_icon="⚡", layout="wide")

if not st.session_state.get("authenticated", False):
    st.warning("Please sign in from the main portal.")
    st.stop()

st.title("⚙ System Settings & Asset Configuration")

# Operational Mode
st.subheader("1. Operational Mode")
mode_col1, mode_col2 = st.columns(2)
with mode_col1:
    st.markdown(f"**Current Mode:** `{settings.OPERATION_MODE.upper()}`")
    st.caption("• **Simulation Mode**: Generates synthetic motor data with continuous physics.")
    st.caption("• **Real Hardware Mode**: Reads live physical telemetry from ESP32 DevKit.")

with mode_col2:
    st.info("Mode toggle is controlled via `.env` (`OPERATION_MODE=simulation` or `OPERATION_MODE=hardware`). Both modes feed the exact same downstream pipeline.")

st.markdown("---")

# MQTT Broker Configuration
st.subheader("2. MQTT Broker & Connectivity")
mqtt_cfg = settings.get_effective_mqtt_config()

b_col1, b_col2, b_col3 = st.columns(3)
with b_col1:
    st.metric("Broker Type", mqtt_cfg["broker_type"].replace("_", " ").title())
with b_col2:
    st.metric("Host", mqtt_cfg["host"])
with b_col3:
    st.metric("Port", f"{mqtt_cfg['port']} (TLS: {mqtt_cfg['use_tls']})")

if mqtt_cfg["broker_type"] == "hivemq_cloud":
    st.success("Connected to HiveMQ Cloud with TLS 1.2/1.3.")
else:
    st.warning("Using Local Mosquitto Broker fallback (zero cloud credential dependency).")

st.markdown("---")

# Device Specifications
st.subheader("3. Registered Device Specifications (Motor 01)")
if settings.DEVICES_FILE.exists():
    with open(settings.DEVICES_FILE, "r") as f:
        dev_data = yaml.safe_load(f) or {}
    st.json(dev_data)

st.markdown("---")

# Thresholds Configuration
st.subheader("4. Operational Thresholds Configuration")
if settings.THRESHOLDS_FILE.exists():
    with open(settings.THRESHOLDS_FILE, "r") as f:
        thresh_data = yaml.safe_load(f) or {}
```

dashboard/pages/6_Settings.py (continued)

```
71      st.json(thresh_data)
72
```

dashboard/pages/7_Model_Performance.py

```python
"""
Model Performance Page — Evaluation Metrics, Baseline Comparison & Explainability.
"""

import sys
import json
from pathlib import Path

# Add project root to sys.path
sys.path.insert(0, str(Path(__file__).resolve().parent.parent.parent))

import streamlit as st
import pandas as pd
import plotly.express as px
from config.settings import settings
from dashboard.components.charts import create_confusion_matrix_heatmap

st.set_page_config(page_title="Model Performance | ML Analytics", page_icon="⚡", layout="wide")

if not st.session_state.get("authenticated", False):
    st.warning("Please sign in from the main portal.")
    st.stop()

st.title("🧠 Machine Learning Model Evaluation & Explainability")
st.markdown("Rigorous benchmark evaluation of Random Forest Classifier against Baseline Decision Tree.")

# Load Metrics JSON
if not settings.ML_METRICS_PATH.exists():
    st.error("Model metrics not found. Train the model using: `python ml/train.py`")
    st.stop()

with open(settings.ML_METRICS_PATH, "r", encoding="utf-8") as f:
    metrics = json.load(f)

# Load Confusion Matrix JSON
cm_data = None
if settings.ML_CONFUSION_MATRIX_PATH.exists():
    with open(settings.ML_CONFUSION_MATRIX_PATH, "r", encoding="utf-8") as f:
        cm_data = json.load(f)

# Headline Metrics Cards
k_col1, k_col2, k_col3, k_col4 = st.columns(4)

with k_col1:
    st.metric("Primary Model", "Random Forest (100 Trees)")
with k_col2:
    st.metric("Holdout Accuracy", f"{metrics['random_forest']['accuracy']*100:.2f}%")
with k_col3:
    st.metric("Weighted F1-Score", f"{metrics['random_forest']['weighted_f1']:.4f}")
with k_col4:
    base_acc = metrics["baseline_decision_tree"]["accuracy"]
    st.metric("Baseline DT Accuracy", f"{base_acc*100:.2f}%")

st.markdown("---")

# Model Comparison Section
st.subheader("1. Baseline Decision Tree vs. Tuned Random Forest")
comp_df = pd.DataFrame([
    {
        "Algorithm": "Baseline Decision Tree (Depth=5)",
        "Accuracy": metrics["baseline_decision_tree"]["accuracy"],
        "Weighted F1": metrics["baseline_decision_tree"]["weighted_f1"],
    },
    {
        "Algorithm": "Random Forest Classifier (Ensemble)",
        "Accuracy": metrics["random_forest"]["accuracy"],
        "Weighted F1": metrics["random_forest"]["weighted_f1"],
    },
])
st.table(comp_df)

```

dashboard/pages/7_Model_Performance.py (continued)

```
st.markdown("---")

# Confusion Matrix Heatmap
st.subheader("2. Confusion Matrix (Independent Episode Holdout Set)")
if cm_data:
    fig_cm = create_confusion_matrix_heatmap(cm_data["labels"], cm_data["matrix"])
    st.plotly_chart(fig_cm, use_container_width=True)

st.markdown("---")

# Feature Importance Bar Chart
st.subheader("3. Feature Importance & Explainability")
importances = metrics.get("feature_importances", {})
if importances:
    feat_df = pd.DataFrame({
        "Feature": list(importances.keys()),
        "Importance": list(importances.values()),
    }).sort_values(by="Importance", ascending=True)

    fig_feat = px.bar(
        feat_df,
        x="Importance",
        y="Feature",
        orientation="h",
        template="plotly_dark",
        title="Gini Impurity Feature Importance",
        color="Importance",
        color_continuous_scale="Blues",
    )
    fig_feat.update_layout(paper_bgcolor="#0d1117", plot_bgcolor="#161b22", height=420)
    st.plotly_chart(fig_feat, use_container_width=True)

st.markdown("---")

# Per-Class Classification Report Table
st.subheader("4. Per-Class Precision, Recall & F1-Score")
report = metrics.get("classification_report", {})
report_rows = []
for label, scores in report.items():
    if isinstance(scores, dict):
        report_rows.append({
            "Class Label": label,
            "Precision": round(scores.get("precision", 0.0), 4),
            "Recall": round(scores.get("recall", 0.0), 4),
            "F1-Score": round(scores.get("f1-score", 0.0), 4),
            "Support (Samples)": int(scores.get("support", 0)),
        })

if report_rows:
    st.dataframe(pd.DataFrame(report_rows), use_container_width=True)

# Limitations Disclosure Box
st.markdown("""
<div class='diagnosis-box' style='border-left-color: #d29922;'>
    <h4 style='color: #d29922; margin-top: 0;'>⚠ Engineering Scope & Synthetic Limitations Notice</h4>
    <p style='color: #c9d1d9; font-size: 0.95rem; margin-bottom: 0;'>
        This model was trained and evaluated on physics-informed synthetic motor profiles.
        While cross-episode validation confirms zero temporal leakage across simulation runs,
        real-world motor installations exhibit physical non-linearities, bearing wear hysteresis,
        and acoustic noise. Transfer calibration against physical benchmark measurements is required
        prior to safety-critical automated actuation.
    </p>
</div>
""", unsafe_allow_html=True)

```

dashboard/styles/theme.css

```
/* Dark Industrial Monitoring Dashboard Theme */

:root {
    --bg-primary: #0d1117;
    --bg-card: #161b22;
    --bg-card-hover: #1f242c;
    --border-color: #30363d;
    --text-primary: #f0f6fc;
    --text-secondary: #8b949e;
    --accent-blue: #58a6ff;
    --accent-cyan: #39c5bb;
    --accent-green: #3fb950;
    --accent-amber: #d29922;
    --accent-red: #f85149;
    --accent-purple: #bc8cff;
}

/* Metric Card Styling */
.metric-container {
    background-color: var(--bg-card);
    border: 1px solid var(--border-color);
    border-radius: 8px;
    padding: 16px 20px;
    margin-bottom: 12px;
    box-shadow: 0 4px 12px rgba(0, 0, 0, 0.25);
    transition: transform 0.15s ease, border-color 0.15s ease;
}

.metric-container:hover {
    border-color: var(--accent-blue);
    transform: translateY(-2px);
}

.metric-label {
    color: var(--text-secondary);
    font-size: 0.85rem;
    font-weight: 600;
    text-transform: uppercase;
    letter-spacing: 0.5px;
    margin-bottom: 4px;
}

.metric-value {
    color: var(--text-primary);
    font-size: 1.85rem;
    font-weight: 700;
    line-height: 1.2;
}

.metric-unit {
    color: var(--text-secondary);
    font-size: 1.0rem;
    font-weight: 400;
    margin-left: 4px;
}

/* Status Badges */
.status-pill {
    display: inline-block;
    padding: 4px 12px;
    border-radius: 16px;
    font-size: 0.8rem;
    font-weight: 600;
    text-transform: uppercase;
    letter-spacing: 0.5px;
}

.status-normal {
    background-color: rgba(63, 185, 80, 0.15);
    color: var(--accent-green);
    border: 1px solid rgba(63, 185, 80, 0.4);
```

dashboard/styles/theme.css (continued)

```
}

.status-warning {
    background-color: rgba(210, 153, 34, 0.15);
    color: var(--accent-amber);
    border: 1px solid rgba(210, 153, 34, 0.4);
}

.status-critical {
    background-color: rgba(248, 81, 73, 0.15);
    color: var(--accent-red);
    border: 1px solid rgba(248, 81, 73, 0.4);
}

.status-unknown {
    background-color: rgba(188, 140, 255, 0.15);
    color: var(--accent-purple);
    border: 1px solid rgba(188, 140, 255, 0.4);
}

/* Callout Box */
.diagnosis-box {
    background: linear-gradient(135deg, #161b22 0%, #1a222d 100%);
    border-left: 4px solid var(--accent-blue);
    border-radius: 6px;
    padding: 16px;
    margin: 14px 0;
}

```

data/csv/sensor_telemetry.csv

Data preview: first 40 of 616 lines. Remaining records are omitted.

```
event_id,device_id,timestamp,source,temperature_c,vibration_rms_g,current_a,rpm,sensor_status
a1eaa703-3836-42bd-b964-fce7de01620e,motor01,2026-10-30T06:55:01.244784+00:00,synthetic,36.46,0.121,0.582,2135.5,ok
56d01a66-51b0-4e0e-bbfb-9c0cc94195cf,motor01,2026-10-30T06:55:02.250506+00:00,synthetic,36.44,0.125,0.583,2141.6,ok
10090c14-054e-4ef9-b134-10ae3d3fe738,motor01,2026-10-30T06:55:03.255983+00:00,synthetic,36.56,0.12,0.577,2145.6,ok
a9282265-c898-4a5e-81c5-5817235d4de3,motor01,2026-10-30T06:55:04.261191+00:00,synthetic,36.6,0.12,0.578,2158.9,ok
c2c6d1f3-cc28-4ab1-b316-62fa794ed539,motor01,2026-10-30T06:55:05.267154+00:00,synthetic,36.68,0.122,0.576,2142.7,ok
95562035-46d5-4130-a53c-6e5556784bf1,motor01,2026-10-30T06:55:06.272599+00:00,synthetic,36.8,0.122,0.58,2143.1,ok
ab4c5e4c-f837-4bba-994c-d9170d23893f,motor01,2026-10-30T06:55:07.278045+00:00,synthetic,36.89,0.116,0.584,2135.5,ok
7cb92b4e-7f21-41e3-bc1d-0e4db394a0b1,motor01,2026-10-30T06:55:08.283269+00:00,synthetic,36.43,0.12,0.576,2135.1,ok
edf4cd33-6221-471e-9274-1c26898518ed,motor01,2026-10-30T06:55:09.285148+00:00,synthetic,36.56,0.116,0.584,2129.3,ok
b7d79bee-58f0-412d-b3a6-bb4fd0e11fd7,motor01,2026-10-30T06:55:10.290943+00:00,synthetic,36.55,0.12,0.583,2133.9,ok
2c38dbff-e4ad-4b59-a087-e5201317a749,motor01,2026-10-30T06:55:11.296085+00:00,synthetic,36.67,0.122,0.587,2142.7,ok
eaffa10a-4802-4a44-9455-40c4f337414b,motor01,2026-10-30T06:55:12.301412+00:00,synthetic,36.56,0.124,0.582,2138.8,ok
ec24023b-e315-4d23-9ab6-14a41e5018e7,motor01,2026-10-30T06:55:13.306923+00:00,synthetic,36.53,0.118,0.575,2163.7,ok
818a15b8-1192-4d35-a235-42fd08519601,motor01,2026-10-30T06:55:14.312470+00:00,synthetic,36.67,0.122,0.58,2171.7,ok
5dbce575-83ed-4139-9120-70d81f32429f,motor01,2026-10-30T06:55:15.316633+00:00,synthetic,36.9,0.119,0.57,2163.3,ok
dd5e845a-1e80-43a5-9515-5175892c7f54,motor01,2026-10-30T06:55:16.322200+00:00,synthetic,37.02,0.121,0.573,2145.6,ok
83139159-6856-471b-8945-d2917d32160e,motor01,2026-10-30T06:55:17.323743+00:00,synthetic,36.92,0.13,0.579,2153.7,ok
232bbe07-32c0-47d8-87f2-2ef29b0daab1,motor01,2026-10-30T06:55:18.329237+00:00,synthetic,36.98,0.124,0.575,2146.9,ok
abc554c6-236b-4b9a-a6bd-f98461ab61aa,motor01,2026-10-30T06:55:19.331829+00:00,synthetic,37.09,0.126,0.576,2142.9,ok
185aee8f-93ba-4746-9eed-07da6141ae29,motor01,2026-10-30T06:55:20.333317+00:00,synthetic,36.92,0.12,0.564,2142.7,ok
86127ced-1a5a-438b-806c-aedab958f622,motor01,2026-10-30T06:55:21.338818+00:00,synthetic,36.8,0.12,0.577,2149.0,ok
047c07df-fc5a-419c-a1ef-1bf266299f72,motor01,2026-10-30T06:55:22.344423+00:00,synthetic,36.83,0.124,0.586,2150.9,ok
ab2dee40-0f61-40e6-98fd-268277643e9e,motor01,2026-10-30T06:55:23.350024+00:00,synthetic,36.86,0.124,0.591,2141.5,ok
5647c3f8-1f58-4dc7-8b2d-6cffd5143050,motor01,2026-10-30T06:55:24.355559+00:00,synthetic,37.04,0.122,0.602,2144.2,ok
ca9d7a6a-19c4-4a11-9c02-624d2d4c3741,motor01,2026-10-30T06:55:25.361340+00:00,synthetic,37.25,0.117,0.592,2147.3,ok
d3499f54-c872-4be7-a8f9-fa15dbad1733,motor01,2026-10-30T06:55:26.366707+00:00,synthetic,37.16,0.12,0.594,2161.0,ok
6a75f99a-bead-4368-8af7-6cddcf7034d5,motor01,2026-10-30T06:55:27.372057+00:00,synthetic,36.72,0.118,0.594,2163.6,ok
fdfaa95b-665b-480e-a63b-7ce357279e94,motor01,2026-10-30T06:55:28.377601+00:00,synthetic,36.63,0.114,0.603,2158.8,ok
24cf9933-05b1-4239-930d-ce9dbe19538e,motor01,2026-10-30T06:55:29.383374+00:00,synthetic,36.59,0.114,0.593,2168.0,ok
e90ba87b-f1f1-45d0-8820-efccc273ceac,motor01,2026-10-30T06:55:30.385617+00:00,synthetic,36.64,0.111,0.591,2150.5,ok
79ea1eb2-8964-45ec-9c74-5f0de2df7d5a,motor01,2026-10-30T06:55:31.388963+00:00,synthetic,36.81,0.114,0.605,2150.4,ok
4c09da6c-a6ee-4843-9261-968a3280261e,motor01,2026-10-30T06:55:32.394425+00:00,synthetic,37.03,0.117,0.596,2138.5,ok
2b5c5268-0305-47db-8efe-f9688cfa5be6,motor01,2026-10-30T06:55:33.399092+00:00,synthetic,37.28,0.12,0.599,2127.4,ok
f0bfdd67-788f-4525-aae6-2cdcbf07cfd4,motor01,2026-10-30T06:55:34.401089+00:00,synthetic,37.46,0.118,0.594,2141.7,ok
```

data/csv/sensor_telemetry.csv (continued)

```
36  d69fc6a8-2b18-4133-9327-1003b163ef84,motor01,2026-10-
    30T06:55:35.407162+00:00,synthetic,37.17,0.127,0.588,2146.4,ok
37  2d04f394-e0f2-4433-ab4e-eb015bb28f9c,motor01,2026-10-
    30T06:55:36.410750+00:00,synthetic,37.03,0.123,0.574,2150.5,ok
38  f9554c14-1550-486d-a444-61f42711ce0c,motor01,2026-10-
    30T06:55:37.416263+00:00,synthetic,37.05,0.122,0.587,2157.8,ok
39  ac92858a-0c90-4784-9cc7-d7b771e15067,motor01,2026-10-
    30T06:55:38.421799+00:00,synthetic,36.84,0.124,0.588,2159.2,ok
40  00e84803-9a94-4e4e-8979-2e57097a2bcd,motor01,2026-10-
    30T06:55:39.425417+00:00,synthetic,36.74,0.126,0.587,2166.2,ok
```

data/node_red/fault_alerts.jsonl

Data preview: first 40 of 9 lines. Remaining records are omitted.


```
1 {"received_at":"2026-10-31T15:35:09.461Z","device_id":"motor01","severity":"WARNING","fault":"TEST ONLY - File
  Logging","message":"Local software test. No real motor fault.","recommended_action":"No action required."}
2 {"received_at":"2026-10-31T15:35:16.058Z","device_id":"motor01","severity":"WARNING","fault":"TEST ONLY - File
  Logging","message":"Local software test. No real motor fault.","recommended_action":"No action required."}
3 {"received_at":"2026-10-31T18:37:38.358Z","timestamp":"2026-10-
  31T18:37:37Z","device_id":"motor01","severity":"WARNING","fault":"Speed Feedback Missing","message":"Hardware
  Alert: Speed Feedback Missing (Severity: WARNING). Temperature=26.44 C, Current=0.432 A, RPM=0.0. Vibration not
   measured. ML diagnosis not evaluated. Action: Stop motor and inspect motor and IR
  sensor.","recommended_action":"Stop motor and inspect motor and IR sensor."}
4 {"received_at":"2026-10-31T18:39:30.793Z","timestamp":"2026-10-
  31T18:39:30Z","device_id":"motor01","severity":"WARNING","fault":"Speed Feedback Missing","message":"Hardware
  Alert: Speed Feedback Missing (Severity: WARNING). Temperature=26.44 C, Current=0.452 A, RPM=0.0. Vibration not
   measured. ML diagnosis not evaluated. Action: Stop motor and inspect motor and IR
  sensor.","recommended_action":"Stop motor and inspect motor and IR sensor."}
5 {"received_at":"2026-10-31T18:45:40.763Z","timestamp":"2026-10-
  31T18:45:40Z","device_id":"motor01","severity":"WARNING","fault":"Speed Feedback Missing","message":"Hardware
  Alert: Speed Feedback Missing (Severity: WARNING). Temperature=26.44 C, Current=0.402 A, RPM=0.0. Vibration not
   measured. ML diagnosis not evaluated. Action: Stop motor and inspect motor and IR
  sensor.","recommended_action":"Stop motor and inspect motor and IR sensor."}
6 {"received_at":"2026-10-31T18:49:18.643Z","timestamp":"2026-10-
  31T18:49:18Z","device_id":"motor01","severity":"WARNING","fault":"Speed Feedback Missing","message":"Hardware
  Alert: Speed Feedback Missing (Severity: WARNING). Temperature=26.44 C, Current=0.368 A, RPM=0.0. Vibration not
   measured. ML diagnosis not evaluated. Action: Stop motor and inspect motor and IR
  sensor.","recommended_action":"Stop motor and inspect motor and IR sensor."}
7 {"received_at":"2026-10-31T18:51:01.046Z","timestamp":"2026-10-
  31T18:51:00Z","device_id":"motor01","severity":"WARNING","fault":"Speed Feedback Missing","message":"Hardware
  Alert: Speed Feedback Missing (Severity: WARNING). Temperature=26.44 C, Current=0.323 A, RPM=0.0. Vibration not
   measured. ML diagnosis not evaluated. Action: Stop motor and inspect motor and IR
  sensor.","recommended_action":"Stop motor and inspect motor and IR sensor."}
8 {"received_at":"2026-10-31T18:51:19.113Z","timestamp":"2026-10-
  31T18:51:18Z","device_id":"motor01","severity":"WARNING","fault":"Speed Feedback Missing","message":"Hardware
  Alert: Speed Feedback Missing (Severity: WARNING). Temperature=26.44 C, Current=0.344 A, RPM=0.0. Vibration not
   measured. ML diagnosis not evaluated. Action: Stop motor and inspect motor and IR
  sensor.","recommended_action":"Stop motor and inspect motor and IR sensor."}
9 {"received_at":"2026-10-31T19:01:54.412Z","timestamp":"2026-10-
  31T19:01:54Z","device_id":"motor01","severity":"WARNING","fault":"Speed Feedback Missing","message":"Hardware
  Alert: Speed Feedback Missing (Severity: WARNING). Temperature=26.38 C, Current=0.345 A, RPM=0.0. Vibration not
   measured. ML diagnosis not evaluated. Action: Stop motor and inspect motor and IR
  sensor.","recommended_action":"Stop motor and inspect motor and IR sensor."}
```

data/node_red/mqtt_test_alerts.jsonl

Data preview: first 40 of 1 lines. Remaining records are omitted.


```
1  {"received_at":"2026-10-
   31T16:07:11.485Z","test_only":true,"device_id":"motor01","severity":"WARNING","fault":"TEST ONLY - Possible
   Overheating","actuate_hardware":false}
```

data/synthetic/motor_training_data.csv

Data preview: first 40 of 3501 lines. Remaining records are omitted.

```
event_id,device_id,timestamp,source,temperature_c,vibration_rms_g,current_a,rpm,sensor_status,run_id,label
c0dcf7cd-6030-4728-9fd0-1799b3f0ff7e,motor01,2026-10-30T05:59:58.833163+00:00,synthetic,36.51,0.121,0.576,2155.4,ok,1,Normal
b1f44d4b-f419-4830-bf33-275efde4d4f4,motor01,2026-10-30T05:59:58.833211+00:00,synthetic,36.44,0.12,0.568,2156.0,ok,1,Normal
79f8acb9-135f-4cf2-9eb0-821101da6bb6,motor01,2026-10-30T05:59:58.833226+00:00,synthetic,36.29,0.122,0.592,2167.2,ok,1,Normal
80752273-9d7a-4b58-b93f-d556b035585e,motor01,2026-10-30T05:59:58.833234+00:00,synthetic,36.24,0.124,0.582,2163.4,ok,1,Normal
ecd8e83a-d316-411d-a470-8b81914f473c,motor01,2026-10-30T05:59:58.833241+00:00,synthetic,36.53,0.119,0.575,2159.3,ok,1,Normal
fd7a5438-da77-4e17-b28b-66bd04cc862f,motor01,2026-10-30T05:59:58.833247+00:00,synthetic,36.31,0.12,0.567,2141.3,ok,1,Normal
693cf75c-9119-4238-9391-dccf95929095,motor01,2026-10-30T05:59:58.833252+00:00,synthetic,36.66,0.122,0.573,2137.5,ok,1,Normal
00ade224-2402-4fbc-b967-5533b66834a6,motor01,2026-10-30T05:59:58.833257+00:00,synthetic,36.83,0.122,0.58,2134.7,ok,1,Normal
2b36fdb2-f2fa-4b5b-ae06-09b583c7ccb9,motor01,2026-10-30T05:59:58.833261+00:00,synthetic,36.71,0.125,0.58,2148.1,ok,1,Normal
649062bb-eadd-41e9-8d52-1172e8aebdc0,motor01,2026-10-30T05:59:58.833266+00:00,synthetic,36.36,0.114,0.574,2141.9,ok,1,Normal
c08ec9c9-0075-4b71-8a25-c0d4df8245f3,motor01,2026-10-30T05:59:58.833271+00:00,synthetic,36.53,0.113,0.584,2145.0,ok,1,Normal
caa3689e-f547-4d3d-acd6-99c8ee1fea1e,motor01,2026-10-30T05:59:58.833275+00:00,synthetic,36.49,0.104,0.597,2152.9,ok,1,Normal
e6c8338a-fb74-42e7-9ab9-76462302c41d,motor01,2026-10-30T05:59:58.833280+00:00,synthetic,36.34,0.101,0.591,2139.8,ok,1,Normal
6cf0637e-424b-4d44-bacd-82e85e1ba463,motor01,2026-10-30T05:59:58.833285+00:00,synthetic,36.11,0.099,0.577,2149.0,ok,1,Normal
3374f1fa-894f-41ac-8357-d1f17159113d,motor01,2026-10-30T05:59:58.833289+00:00,synthetic,36.3,0.104,0.571,2145.0,ok,1,Normal
d8d7157f-7159-4cd8-880e-34ec980a701a,motor01,2026-10-30T05:59:58.833294+00:00,synthetic,36.45,0.107,0.581,2151.7,ok,1,Normal
6c5ec2f7-03b4-41d1-93a2-e608678e166f,motor01,2026-10-30T05:59:58.833298+00:00,synthetic,36.62,0.105,0.571,2157.6,ok,1,Normal
49922bf3-aa79-4ce4-a903-ca97b3b0a160,motor01,2026-10-30T05:59:58.833302+00:00,synthetic,36.51,0.12,0.572,2165.6,ok,1,Normal
adc9c719-6fb5-4305-a8ed-ce033e17ff4f,motor01,2026-10-30T05:59:58.833307+00:00,synthetic,36.26,0.117,0.584,2165.2,ok,1,Normal
357c0991-e463-48df-93a1-e494a96aa21a,motor01,2026-10-30T05:59:58.833311+00:00,synthetic,36.09,0.115,0.573,2163.1,ok,1,Normal
5524e7de-8e22-422f-b026-40fa72869dd7,motor01,2026-10-30T05:59:58.833316+00:00,synthetic,36.33,0.119,0.572,2156.6,ok,1,Normal
5603386b-12d2-4daf-ad47-85209c0cfbd3,motor01,2026-10-30T05:59:58.833320+00:00,synthetic,36.09,0.124,0.57,2157.8,ok,1,Normal
935270b8-2144-47a0-af41-9f6c8b4f8ca6,motor01,2026-10-30T05:59:58.833325+00:00,synthetic,36.31,0.124,0.575,2155.0,ok,1,Normal
11310571-48c2-4ca8-97ce-9df2f66e8630,motor01,2026-10-30T05:59:58.833329+00:00,synthetic,36.37,0.127,0.57,2149.7,ok,1,Normal
10ca4131-60f7-4f4e-8caf-12e2a4e49123,motor01,2026-10-30T05:59:58.833333+00:00,synthetic,36.23,0.125,0.572,2151.1,ok,1,Normal
b5122e1b-fc14-4cbe-9556-6bae113bbeac,motor01,2026-10-30T05:59:58.833338+00:00,synthetic,36.14,0.123,0.583,2142.0,ok,1,Normal
7c09f809-2395-4515-986e-01f5ec13b6eb,motor01,2026-10-30T05:59:58.833342+00:00,synthetic,36.17,0.123,0.58,2144.2,ok,1,Normal
6f7cb3d5-d447-4dbe-878b-300dfe8581f4,motor01,2026-10-30T05:59:58.833347+00:00,synthetic,36.18,0.12,0.58,2144.2,ok,1,Normal
50e81adc-e058-47ad-8697-a7aee9f62a68,motor01,2026-10-30T05:59:58.833351+00:00,synthetic,36.36,0.116,0.583,2153.4,ok,1,Normal
caba8f02-8a9a-4e11-83b0-73e11665210c,motor01,2026-10-30T05:59:58.833355+00:00,synthetic,36.66,0.124,0.586,2148.5,ok,1,Normal
941bef24-1415-44f9-a733-9b9b07c20f33,motor01,2026-10-30T05:59:58.833360+00:00,synthetic,36.85,0.121,0.574,2160.4,ok,1,Normal
38eecc10-6c94-4fe2-ac96-5fc2b1a954aa,motor01,2026-10-30T05:59:58.833364+00:00,synthetic,37.03,0.123,0.571,2151.1,ok,1,Normal
949d379d-4004-4322-a592-d0e31f7390ec,motor01,2026-10-30T05:59:58.833368+00:00,synthetic,36.8,0.123,0.563,2140.9,ok,1,Normal
f3713570-849f-4975-9e1c-f62409a50479,motor01,2026-10-30T05:59:58.833373+00:00,synthetic,36.78,0.125,0.577,2133.4,ok,1,Normal
```

data/synthetic/motor_training_data.csv (continued)

```
36  37aa61c3-3280-4800-89f8-3b45a004d7a0,motor01,2026-10-
    30T05:59:58.833377+00:00,synthetic,36.93,0.122,0.571,2142.1,ok,1,Normal
37  1f812d63-03fe-4a38-9305-cb02c1d0b6f5,motor01,2026-10-
    30T05:59:58.833381+00:00,synthetic,36.99,0.122,0.561,2139.1,ok,1,Normal
38  4bc78d0f-c4bc-471c-b005-b476bfe5ea02,motor01,2026-10-
    30T05:59:58.833386+00:00,synthetic,36.85,0.113,0.575,2139.7,ok,1,Normal
39  5c5ff0ba-8dcf-4fc2-b547-23a2df2358aa,motor01,2026-10-
    30T05:59:58.833390+00:00,synthetic,36.57,0.114,0.584,2127.8,ok,1,Normal
40  adf8514f-6f54-4a13-bdcb-0b16b701d8e3,motor01,2026-10-
    30T05:59:58.833394+00:00,synthetic,36.72,0.113,0.594,2136.8,ok,1,Normal
```

database/connection.py

```
"""
SQLite Database Connection Factory
Manages thread-safe SQLite connections with WAL mode and row factory.
"""

import sqlite3
import threading
from pathlib import Path
from contextlib import contextmanager
from config.settings import settings


class DatabaseManager:
    """
    Thread-safe connection provider for SQLite.
    """

    def __init__(self, db_path: Path = settings.DATABASE_PATH):
        self.db_path = Path(db_path)
        self.db_path.parent.mkdir(parents=True, exist_ok=True)
        self._local = threading.local()

    def get_connection(self) -> sqlite3.Connection:
        """Returns a thread-local SQLite connection."""
        if not hasattr(self._local, "conn") or self._local.conn is None:
            conn = sqlite3.connect(
                str(self.db_path),
                timeout=30.0,
                check_same_thread=False,
            )
            conn.row_factory = sqlite3.Row
            # Enable WAL mode and foreign keys
            conn.execute("PRAGMA journal_mode = WAL;")
            conn.execute("PRAGMA foreign_keys = ON;")
            conn.execute("PRAGMA busy_timeout = 30000;")
            self._local.conn = conn
        return self._local.conn

    @contextmanager
    def session(self):
        """Context manager yielding a transaction with automatic commit/rollback."""
        conn = self.get_connection()
        try:
            yield conn
            conn.commit()
        except Exception:
            conn.rollback()
            raise


db_manager = DatabaseManager()

```

database/migrations/001_initial_schema.sql

```
-- 001_initial_schema.sql
-- Baseline schema for Knowledge-Driven IoT Fault Diagnosis System

PRAGMA journal_mode = WAL;
PRAGMA foreign_keys = ON;

-- 1. Devices Table
CREATE TABLE IF NOT EXISTS devices (
    device_id TEXT PRIMARY KEY,
    name TEXT NOT NULL,
    type TEXT NOT NULL,
    driver TEXT,
    rated_voltage_v REAL,
    rated_current_a REAL,
    rated_rpm REAL,
    location TEXT,
    created_at TEXT NOT NULL
);

-- 2. Sensor Readings Table (Deduplicated via UNIQUE event_id)
CREATE TABLE IF NOT EXISTS sensor_readings (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    event_id TEXT UNIQUE NOT NULL,
    device_id TEXT NOT NULL,
    timestamp TEXT NOT NULL,
    source TEXT NOT NULL,
    temperature_c REAL NOT NULL,
    vibration_rms_g REAL NOT NULL,
    current_a REAL NOT NULL,
    rpm REAL NOT NULL,
    sensor_status TEXT NOT NULL,
    created_at TEXT DEFAULT (datetime('now', 'utc')),
    FOREIGN KEY (device_id) REFERENCES devices(device_id) ON DELETE CASCADE
);

CREATE INDEX IF NOT EXISTS idx_sensor_readings_dev_ts
    ON sensor_readings(device_id, timestamp DESC);
CREATE INDEX IF NOT EXISTS idx_sensor_readings_event_id
    ON sensor_readings(event_id);

-- 3. Fault Predictions Table
CREATE TABLE IF NOT EXISTS fault_predictions (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    event_id TEXT NOT NULL,
    device_id TEXT NOT NULL,
    timestamp TEXT NOT NULL,
    rule_fault TEXT,
    rule_severity TEXT,
    ml_fault TEXT,
    ml_confidence REAL,
    hybrid_fault TEXT NOT NULL,
    hybrid_severity TEXT NOT NULL,
    explanation TEXT,
    recommended_action TEXT,
    conflict_detected INTEGER DEFAULT 0,
    created_at TEXT DEFAULT (datetime('now', 'utc')),
    FOREIGN KEY (device_id) REFERENCES devices(device_id)
);

CREATE INDEX IF NOT EXISTS idx_fault_predictions_dev_ts
    ON fault_predictions(device_id, timestamp DESC);

-- 4. Fault Events Table
CREATE TABLE IF NOT EXISTS fault_events (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    device_id TEXT NOT NULL,
    fault_type TEXT NOT NULL,
    severity TEXT NOT NULL,
    start_time TEXT NOT NULL,
    end_time TEXT,
    is_active INTEGER DEFAULT 1,
```

database/migrations/001_initial_schema.sql (continued)

```
    trigger_values TEXT,
    created_at TEXT DEFAULT (datetime('now', 'utc'))
);

CREATE INDEX IF NOT EXISTS idx_fault_events_active
    ON fault_events(device_id, is_active);

-- 5. Alert History Table
CREATE TABLE IF NOT EXISTS alert_history (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    device_id TEXT NOT NULL,
    timestamp TEXT NOT NULL,
    severity TEXT NOT NULL,
    title TEXT NOT NULL,
    message TEXT NOT NULL,
    acknowledged INTEGER DEFAULT 0,
    acknowledged_by TEXT,
    acknowledged_at TEXT,
    created_at TEXT DEFAULT (datetime('now', 'utc'))
);

CREATE INDEX IF NOT EXISTS idx_alert_history_ack
    ON alert_history(device_id, acknowledged, timestamp DESC);

-- 6. Users Table
CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,
    salt TEXT NOT NULL,
    role TEXT NOT NULL DEFAULT 'operator',
    is_active INTEGER DEFAULT 1,
    created_at TEXT NOT NULL,
    last_login TEXT
);

-- 7. System Events Table
CREATE TABLE IF NOT EXISTS system_events (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp TEXT NOT NULL,
    event_type TEXT NOT NULL,
    component TEXT NOT NULL,
    message TEXT NOT NULL,
    details TEXT,
    created_at TEXT DEFAULT (datetime('now', 'utc'))
);
```

database/migrations/migrate.py

```
"""
Database Migration Runner for SQLite.
Discovers and executes SQL migration files in chronological sequence.
"""

import sys
from pathlib import Path
import sqlite3

# Add project root to sys.path
sys.path.insert(0, str(Path(__file__).resolve().parent.parent.parent))

from config.settings import settings


def run_migrations(db_path: Path = settings.DATABASE_PATH) -> None:
    """Executes all migration scripts in order against the SQLite database."""
    db_path = Path(db_path)
    db_path.parent.mkdir(parents=True, exist_ok=True)
    migrations_dir = Path(__file__).resolve().parent

    conn = sqlite3.connect(db_path)
    try:
        # Create schema migrations table if missing
        conn.execute("""
            CREATE TABLE IF NOT EXISTS _schema_migrations (
                filename TEXT PRIMARY KEY,
                applied_at TEXT DEFAULT (datetime('now', 'utc'))
            )
        """)
        conn.commit()

        # Find all .sql migration files
        sql_files = sorted(migrations_dir.glob("*.sql"))
        print(f"Discovered {len(sql_files)} migration files in {migrations_dir}")

        for sql_file in sql_files:
            cursor = conn.execute("SELECT filename FROM _schema_migrations WHERE filename = ?",
(sql_file.name,))
            if cursor.fetchone():
                continue

            print(f"Applying migration: {sql_file.name}...")
            with open(sql_file, "r", encoding="utf-8") as f:
                script = f.read()

            conn.executescript(script)
            conn.execute("INSERT INTO _schema_migrations (filename) VALUES (?)", (sql_file.name,))
            conn.commit()
            print(f"Successfully applied {sql_file.name}")

    finally:
        conn.close()


if __name__ == "__main__":
    run_migrations()
```

database/models.py

```
"""
Database Schema Definitions for SQLite.
Defines clean DDL queries with indexes, constraints, and WAL mode configuration.
"""

SCHEMA_SQL = """
PRAGMA journal_mode = WAL;
PRAGMA foreign_keys = ON;

-- 1. Devices Table
CREATE TABLE IF NOT EXISTS devices (
    device_id TEXT PRIMARY KEY,
    name TEXT NOT NULL,
    type TEXT NOT NULL,
    driver TEXT,
    rated_voltage_v REAL,
    rated_current_a REAL,
    rated_rpm REAL,
    location TEXT,
    created_at TEXT NOT NULL
);

-- 2. Sensor Readings Table (Deduplicated via UNIQUE event_id)
CREATE TABLE IF NOT EXISTS sensor_readings (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    event_id TEXT UNIQUE NOT NULL,
    device_id TEXT NOT NULL,
    timestamp TEXT NOT NULL,
    source TEXT NOT NULL,
    temperature_c REAL NOT NULL,
    vibration_rms_g REAL,
    current_a REAL NOT NULL,
    rpm REAL NOT NULL,
    sensor_status TEXT NOT NULL,
    created_at TEXT DEFAULT (datetime('now', 'utc')),
    FOREIGN KEY (device_id) REFERENCES devices(device_id) ON DELETE CASCADE
);

CREATE INDEX IF NOT EXISTS idx_sensor_readings_dev_ts
    ON sensor_readings(device_id, timestamp DESC);
CREATE INDEX IF NOT EXISTS idx_sensor_readings_event_id
    ON sensor_readings(event_id);

-- 3. Fault Predictions Table (Stores Rule, ML, and Hybrid Verdicts)
CREATE TABLE IF NOT EXISTS fault_predictions (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    event_id TEXT NOT NULL,
    device_id TEXT NOT NULL,
    timestamp TEXT NOT NULL,
    rule_fault TEXT,
    rule_severity TEXT,
    ml_fault TEXT,
    ml_confidence REAL,
    hybrid_fault TEXT NOT NULL,
    hybrid_severity TEXT NOT NULL,
    explanation TEXT,
    recommended_action TEXT,
    conflict_detected INTEGER DEFAULT 0,
    created_at TEXT DEFAULT (datetime('now', 'utc')),
    FOREIGN KEY (device_id) REFERENCES devices(device_id)
);

CREATE INDEX IF NOT EXISTS idx_fault_predictions_dev_ts
    ON fault_predictions(device_id, timestamp DESC);

-- 4. Fault Events Table (Tracks Fault Lifecycle & Duration)
CREATE TABLE IF NOT EXISTS fault_events (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    device_id TEXT NOT NULL,
    fault_type TEXT NOT NULL,
    severity TEXT NOT NULL,
```

database/models.py (continued)

```
    start_time TEXT NOT NULL,
    end_time TEXT,
    is_active INTEGER DEFAULT 1,
    trigger_values TEXT,
    created_at TEXT DEFAULT (datetime('now', 'utc'))
);

CREATE INDEX IF NOT EXISTS idx_fault_events_active
    ON fault_events(device_id, is_active);

-- 5. Alert History Table (Alert Queue & Operator Acknowledgment)
CREATE TABLE IF NOT EXISTS alert_history (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    device_id TEXT NOT NULL,
    timestamp TEXT NOT NULL,
    severity TEXT NOT NULL,
    title TEXT NOT NULL,
    message TEXT NOT NULL,
    acknowledged INTEGER DEFAULT 0,
    acknowledged_by TEXT,
    acknowledged_at TEXT,
    created_at TEXT DEFAULT (datetime('now', 'utc'))
);

CREATE INDEX IF NOT EXISTS idx_alert_history_ack
    ON alert_history(device_id, acknowledged, timestamp DESC);

-- 6. Users Table (Authentication & Access Control)
CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,
    salt TEXT NOT NULL,
    role TEXT NOT NULL DEFAULT 'operator',
    is_active INTEGER DEFAULT 1,
    created_at TEXT NOT NULL,
    last_login TEXT
);

-- 7. System Events Table (Audit Log for Ingestion, Brokers & Hardware)
CREATE TABLE IF NOT EXISTS system_events (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp TEXT NOT NULL,
    event_type TEXT NOT NULL,
    component TEXT NOT NULL,
    message TEXT NOT NULL,
    details TEXT,
    created_at TEXT DEFAULT (datetime('now', 'utc'))
);
"""

```

database/repository.py

```
"""
SQLite Database Repository
Implements data access operations for sensor telemetry, fault events, alerts, and users.
"""

import json
import logging
from datetime import datetime, timezone
from typing import Dict, Any, List, Optional
import pandas as pd
from database.connection import db_manager
from database.models import SCHEMA_SQL

logger = logging.getLogger("database_repo")


class TelemetryRepository:
    """
    Authoritative database repository for all persistence needs across Layers 3-6.
    """

    def __init__(self, manager=db_manager):
        self.manager = manager
        self.init_db()

    def init_db(self) -> None:
        """Initializes database schema and default device if missing."""
        with self.manager.session() as conn:
            conn.executescript(SCHEMA_SQL)
            # Register default motor01 asset if not already present
            cursor = conn.execute("SELECT device_id FROM devices WHERE device_id = ?", ("motor01",))
            if not cursor.fetchone():
                now_utc = datetime.now(timezone.utc).isoformat()
                conn.execute(
                    """
                    INSERT INTO devices (device_id, name, type, driver, rated_voltage_v,
                                         rated_current_a, rated_rpm, location, created_at)
                    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
                    """,
                    (
                        "motor01",
                        "Primary Feed Drive Motor (Station 01)",
                        "12V Brushed DC Motor",
                        "L298N Dual H-Bridge Driver",
                        12.0,
                        0.80,
                        2000.0,
                        "Machining Cell 1",
                        now_utc,
                    ),
                )
                logger.info("Registered default device 'motor01' in SQLite database.")

    def insert_sensor_reading(self, reading: Dict[str, Any]) -> bool:
        """
        Inserts a sensor reading into SQLite. Deduplicates automatically via event_id UNIQUE constraint.
        Returns True if newly inserted, False if duplicate or failed.
        """
        query = """
        INSERT OR IGNORE INTO sensor_readings
        (event_id, device_id, timestamp, source, temperature_c, vibration_rms_g, current_a, rpm, sensor_status)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
        """
        try:
            with self.manager.session() as conn:
                cursor = conn.execute(
                    query,
                    (
                        reading["event_id"],
                        reading["device_id"],
                        reading["timestamp"],
```

database/repository.py (continued)

```
                        reading.get("source", "synthetic"),
                        float(reading["temperature_c"]),
                        (float(reading["vibration_rms_g"]) if reading.get("vibration_rms_g") is not None else None),
                        float(reading["current_a"]),
                        float(reading["rpm"]),
                        reading.get("sensor_status", "ok"),
                    ),
                )
                return cursor.rowcount > 0
        except Exception as exc:
            logger.error("Failed to insert sensor reading %s: %s", reading.get("event_id"), exc)
            return False

    def get_recent_readings(self, device_id: str = "motor01", limit: int = 100) -> pd.DataFrame:
        """Retrieves the N most recent sensor readings for a device."""
        query = """
        SELECT id, event_id, device_id, timestamp, source, temperature_c, vibration_rms_g, current_a, rpm, sensor_status
        FROM sensor_readings
        WHERE device_id = ?
        ORDER BY timestamp DESC
        LIMIT ?
        """
        with self.manager.session() as conn:
            cursor = conn.execute(query, (device_id, limit))
            rows = [dict(row) for row in cursor.fetchall()]
        if not rows:
            return pd.DataFrame(columns=[
                "id", "event_id", "device_id", "timestamp", "source",
                "temperature_c", "vibration_rms_g", "current_a", "rpm", "sensor_status"
            ])
        df = pd.DataFrame(rows)
        # Return in ascending chronological order for plotting
        return df.iloc[::-1].reset_index(drop=True)

    def get_readings_by_time_range(
        self, device_id: str, start_iso: str, end_iso: str, limit: int = 5000
    ) -> pd.DataFrame:
        """Queries readings filtered by ISO-8601 start and end timestamps."""
        query = """
        SELECT id, event_id, device_id, timestamp, source, temperature_c, vibration_rms_g, current_a, rpm, sensor_status
        FROM sensor_readings
        WHERE device_id = ? AND timestamp >= ? AND timestamp <= ?
        ORDER BY timestamp ASC
        LIMIT ?
        """
        with self.manager.session() as conn:
            cursor = conn.execute(query, (device_id, start_iso, end_iso, limit))
            rows = [dict(row) for row in cursor.fetchall()]
        return pd.DataFrame(rows)

    def insert_fault_prediction(self, pred: Dict[str, Any]) -> int:
        """Saves a hybrid rule/ML diagnostic conclusion."""
        query = """
        INSERT INTO fault_predictions
        (event_id, device_id, timestamp, rule_fault, rule_severity, ml_fault,
         ml_confidence, hybrid_fault, hybrid_severity, explanation, recommended_action, conflict_detected)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
        """
        with self.manager.session() as conn:
            cursor = conn.execute(
                query,
                (
                    pred.get("event_id", ""),
                    pred["device_id"],
                    pred["timestamp"],
                    pred.get("rule_fault"),
                    pred.get("rule_severity"),
                    pred.get("ml_fault"),
```

database/repository.py (continued) (continued)

```
                    float(pred.get("ml_confidence", 0.0)),
                    pred["hybrid_fault"],
                    pred["hybrid_severity"],
                    pred.get("explanation", ""),
                    pred.get("recommended_action", ""),
                    1 if pred.get("conflict_detected") else 0,
                ),
            )
            return cursor.lastrowid

    def get_recent_predictions(self, device_id: str = "motor01", limit: int = 50) -> pd.DataFrame:
        """Retrieves recent fault diagnoses."""
        query = """
        SELECT id, event_id, device_id, timestamp, rule_fault, rule_severity,
               ml_fault, ml_confidence, hybrid_fault, hybrid_severity,
               explanation, recommended_action, conflict_detected
        FROM fault_predictions
        WHERE device_id = ?
        ORDER BY timestamp DESC
        LIMIT ?
        """
        with self.manager.session() as conn:
            cursor = conn.execute(query, (device_id, limit))
            rows = [dict(row) for row in cursor.fetchall()]
        return pd.DataFrame(rows)

    def insert_alert(
        self, device_id: str, severity: str, title: str, message: str, timestamp: Optional[str] = None
    ) -> int:
        """Records a new alert."""
        ts = timestamp or datetime.now(timezone.utc).isoformat()
        query = """
        INSERT INTO alert_history (device_id, timestamp, severity, title, message, acknowledged)
        VALUES (?, ?, ?, ?, ?, 0)
        """
        with self.manager.session() as conn:
            cursor = conn.execute(query, (device_id, ts, severity, title, message))
            return cursor.lastrowid

    def get_alerts(self, device_id: Optional[str] = None, only_unacknowledged: bool = False, limit: int = 50)
-> pd.DataFrame:
        """Queries alert records."""
        conditions = []
        params: List[Any] = []
        if device_id:
            conditions.append("device_id = ?")
            params.append(device_id)
        if only_unacknowledged:
            conditions.append("acknowledged = 0")
        where_clause = f"WHERE {' AND '.join(conditions)}" if conditions else ""

        query = f"""
        SELECT id, device_id, timestamp, severity, title, message, acknowledged, acknowledged_by,
acknowledged_at
        FROM alert_history
        {where_clause}
        ORDER BY timestamp DESC
        LIMIT ?
        """
        params.append(limit)
        with self.manager.session() as conn:
            cursor = conn.execute(query, tuple(params))
            rows = [dict(row) for row in cursor.fetchall()]
        return pd.DataFrame(rows)

    def acknowledge_alert(self, alert_id: int, user: str = "operator") -> bool:
        """Marks an alert as acknowledged."""
        now_utc = datetime.now(timezone.utc).isoformat()
        query = """
        UPDATE alert_history
        SET acknowledged = 1, acknowledged_by = ?, acknowledged_at = ?
```

database/repository.py (continued) (continued) (continued)

```
        WHERE id = ?
        """
        with self.manager.session() as conn:
            cursor = conn.execute(query, (user, now_utc, alert_id))
            return cursor.rowcount > 0

    def record_system_event(
        self, event_type: str, component: str, message: str, details: Optional[str] = None
    ) -> int:
        """Logs system health, broker reconnections, or operational state shifts."""
        now_utc = datetime.now(timezone.utc).isoformat()
        query = """
        INSERT INTO system_events (timestamp, event_type, component, message, details)
        VALUES (?, ?, ?, ?, ?)
        """
        with self.manager.session() as conn:
            cursor = conn.execute(query, (now_utc, event_type, component, message, details or ""))
            return cursor.lastrowid


repo = TelemetryRepository()

```

diagnosis/hardware_rules.py

```

"""Rule-only diagnosis for the three-sensor ESP32 motor."""

def diagnose_esp32(reading, motor_status, rpm_zero_persistent=False):
    temp = float(reading["temperature_c"])
    current = float(reading["current_a"])
    rpm = float(reading["rpm"])
    status = str(motor_status or "").upper()

    # Preliminary limits: calibrate using real motor tests.
    if temp >= 65:
        fault = "Possible Overheating"
        severity = "CRITICAL"
        explanation = "Temperature at or above 65 C."
        action = "Stop and inspect motor temperature safely."

    elif status == "OFF":
        fault = "Motor Idle"
        severity = "NORMAL"
        explanation = "Motor is commanded OFF."
        action = "No action required."

    elif status != "ON":
        fault = "Motor State Unknown"
        severity = "WARNING"
        explanation = "Motor command state was not reported."
        action = "Verify motor control and telemetry."

    elif current >= 1.70 and rpm <= 100:
        fault = "Possible Motor Stall"
        severity = "CRITICAL"
        explanation = "High current with very low RPM."
        action = "Stop motor and inspect for binding."

    elif current >= 1.05 and rpm < 350:
        fault = "Possible Mechanical Overload"
        severity = "WARNING"
        explanation = "Elevated current and reduced RPM."
        action = "Inspect mechanical load and wiring."

    elif temp >= 55:
        fault = "Possible Thermal Stress"
        severity = "WARNING"
        explanation = "Motor temperature elevated."
        action = "Check ventilation and motor temperature."

    elif rpm <= 0:
        if rpm_zero_persistent:
            fault = "Speed Feedback Missing"
            severity = "WARNING"
            explanation = (
                "Zero RPM persisted across consecutive ON readings."
            )
            action = "Stop motor and inspect motor and IR sensor."
        else:
            fault = "RPM Pending Verification"
            severity = "NORMAL"
            explanation = (
                "First ON reading with zero RPM; "
                "rotation has not yet been confirmed."
            )
            action = "Verify motor rotation and sensor pulses."

    else:
        fault = "Normal Operation"
        severity = "NORMAL"
        explanation = (
            "Available sensor readings are within "
            "preliminary operating limits."
        )
        action = "Continue monitoring."
```

diagnosis/hardware_rules.py (continued)

```
72
73      explanation += " Vibration unavailable; ML not evaluated."
74
75      return {
76          "event_id": reading["event_id"],
77          "device_id": reading["device_id"],
78          "timestamp": reading["timestamp"],
79          "rule_fault": fault,
80          "rule_severity": severity,
81          "ml_fault": None,
82          "ml_confidence": 0.0,
83          "hybrid_fault": fault,
84          "hybrid_severity": severity,
85          "explanation": explanation,
86          "recommended_action": action,
87          "conflict_detected": False,
88      }
```

diagnosis/hybrid_engine.py

```
"""
Hybrid Diagnostic Engine Module
Combines deterministic physics-based expert rules with probabilistic machine learning predictions.
Transparently flags conflicts, aggregates confidence, and yields prescriptive maintenance actions.
"""

from typing import Dict, Any, List, Optional
from diagnosis.rule_engine import rule_engine
from ml.predict import predictor


SEVERITY_ORDER = {
    "NORMAL": 0,
    "WARNING": 1,
    "CRITICAL": 2,
    "UNKNOWN / SENSOR ERROR": 3,
}

FAULT_TO_SEVERITY = {
    "Normal": "NORMAL",
    "Mechanical Overload": "WARNING",
    "Excessive Vibration": "WARNING",
    "Motor Slowdown": "WARNING",
    "Motor Overheating": "CRITICAL",
    "Motor Stall": "CRITICAL",
    "Motor Stall / Locked Rotor": "CRITICAL",
    "Sensor Error": "UNKNOWN / SENSOR ERROR",
    "Sensor Acquisition Fault": "UNKNOWN / SENSOR ERROR",
}


class HybridDiagnosticEngine:
    """
    Reconciles Rule Engine evaluation with Random Forest ML inference.
    """

    def __init__(self, rule_eng=rule_engine, ml_pred=predictor):
        self.rule_engine = rule_eng
        self.ml_predictor = ml_pred

    def diagnose(
        self,
        reading: Dict[str, Any],
        recent_history: Optional[List[Dict[str, Any]]] = None,
    ) -> Dict[str, Any]:
        """
        Runs both diagnostic paradigms and synthesizes an explainable hybrid decision.
        """
        # 1. Evaluate Rule-based Knowledge Base
        rule_result = self.rule_engine.evaluate(reading)
        rule_fault = rule_result["suspected_fault"]
        rule_sev = rule_result["severity"]
        rule_conf = rule_result["confidence"]

        # 2. Evaluate ML Classifier
        ml_fault, ml_conf, ml_probs = self.ml_predictor.predict(reading, recent_history)

        # 3. Conflict Detection & Reconciliation
        conflict_detected = False
        reconciliation_notes = ""

        # Normalize naming for comparison
        norm_rule = "Sensor Error" if "Sensor" in rule_fault else (
            "Motor Stall" if "Stall" in rule_fault else rule_fault
        )
        norm_ml = "Sensor Error" if ml_fault and "Sensor" in ml_fault else (
            "Motor Stall" if ml_fault and "Stall" in ml_fault else ml_fault
        )

        if ml_fault is None:
            # ML offline fallback
```

diagnosis/hybrid_engine.py (continued)

```
            hybrid_fault = rule_fault
            hybrid_sev = rule_sev
            hybrid_conf = rule_conf
            reconciliation_notes = "ML model unavailable. Operating in deterministic rules-only mode."
        elif norm_rule == norm_ml:
            # Unanimous agreement
            hybrid_fault = rule_fault
            hybrid_sev = rule_sev
            # Boost confidence when both agree
            hybrid_conf = round(min(0.99, (rule_conf + ml_conf) / 2.0 + 0.05), 4)
            reconciliation_notes = f"Unanimous consensus between Physics Rules and ML Classifier (Confidence: {hybrid_conf*100:.1f}%)."
        else:
            # Conflict detected
            conflict_detected = True
            ml_sev = FAULT_TO_SEVERITY.get(ml_fault, "WARNING")

            # Conservative safety rule: choose highest severity hypothesis
            rule_sev_rank = SEVERITY_ORDER.get(rule_sev, 1)
            ml_sev_rank = SEVERITY_ORDER.get(ml_sev, 1)

            if rule_sev_rank >= ml_sev_rank:
                hybrid_fault = rule_fault
                hybrid_sev = rule_sev
            else:
                hybrid_fault = ml_fault
                hybrid_sev = ml_sev

            # Uncertainty penalty
            hybrid_conf = round(max(0.40, min(rule_conf, ml_conf) * 0.85), 4)
            reconciliation_notes = (
                f"Diagnostic Conflict: Rule Engine indicated '{rule_fault}' ({rule_sev}), "
                f"while ML predicted '{ml_fault}' ({ml_sev}, p={ml_conf:.2f}). "
                f"Selected '{hybrid_fault}' under industrial safety conservatism."
            )

        return {
            "device_id": reading.get("device_id", "motor01"),
            "event_id": reading.get("event_id", ""),
            "timestamp": reading.get("timestamp", ""),
            "rule_fault": rule_fault,
            "rule_severity": rule_sev,
            "ml_fault": ml_fault or "N/A",
            "ml_confidence": ml_conf,
            "hybrid_fault": hybrid_fault,
            "hybrid_severity": hybrid_sev,
            "hybrid_confidence": hybrid_conf,
            "explanation": f"{rule_result['explanation']} {reconciliation_notes}",
            "recommended_action": rule_result["recommended_action"],
            "conflict_detected": conflict_detected,
            "triggering_evidence": rule_result["triggering_evidence"],
            "activated_rules": rule_result["activated_rules"],
            "ml_probabilities": ml_probs,
        }


hybrid_engine = HybridDiagnosticEngine()

```

diagnosis/knowledge_base.py

```python
"""
Knowledge Base Management & Query Interface.
Loads, validates, and provides structured query access to declarative engineering rules
and operational threshold envelopes.
"""

from pathlib import Path
from typing import Dict, Any, List, Optional
import yaml

from config.settings import settings


class KnowledgeBaseManager:
    """
    Manages loading, validating, and querying domain knowledge rules from YAML configuration.
    """

    def __init__(self, kb_path: Optional[Path] = None, thresholds_path: Optional[Path] = None):
        self.kb_path = Path(kb_path or settings.KNOWLEDGE_BASE_FILE)
        self.thresholds_path = Path(thresholds_path or settings.THRESHOLDS_FILE)
        self._rules: List[Dict[str, Any]] = []
        self._metadata: Dict[str, Any] = {}
        self._thresholds: Dict[str, Any] = {}
        self.reload()

    def reload(self) -> None:
        """Loads or reloads knowledge base and thresholds from YAML files."""
        if self.kb_path.exists():
            with open(self.kb_path, "r", encoding="utf-8") as f:
                data = yaml.safe_load(f) or {}
                self._metadata = data.get("metadata", {})
                self._rules = data.get("rules", [])
                self._rules.sort(key=lambda r: r.get("priority", 0), reverse=True)

        if self.thresholds_path.exists():
            with open(self.thresholds_path, "r", encoding="utf-8") as f:
                self._thresholds = yaml.safe_load(f) or {}

    @property
    def rules(self) -> List[Dict[str, Any]]:
        return self._rules

    @property
    def metadata(self) -> Dict[str, Any]:
        return self._metadata

    @property
    def thresholds(self) -> Dict[str, Any]:
        return self._thresholds

    def get_rule_by_id(self, rule_id: str) -> Optional[Dict[str, Any]]:
        """Retrieves a specific rule by its unique identifier."""
        for r in self._rules:
            if r.get("rule_id") == rule_id:
                return r
        return None

    def get_rules_by_severity(self, severity: str) -> List[Dict[str, Any]]:
        """Filters rules by output severity level (e.g. CRITICAL, WARNING, NORMAL)."""
        return [
            r for r in self._rules
            if r.get("hypothesis", {}).get("severity", "").upper() == severity.upper()
        ]

    def describe(self) -> Dict[str, Any]:
        """Returns statistical overview of the loaded knowledge base."""
        return {
            "title": self._metadata.get("title", "Motor Fault Knowledge Base"),
            "version": self._metadata.get("version", "1.0.0"),
            "target_motor": self._metadata.get("target_motor", "12V DC Motor"),
```

diagnosis/knowledge_base.py (continued)

```
            "total_rules": len(self._rules),
            "rule_ids": [r.get("rule_id") for r in self._rules],
            "severities": list(set(r.get("hypothesis", {}).get("severity") for r in self._rules)),
        }


# Singleton instance
kb_manager = KnowledgeBaseManager()
```

diagnosis/recommendations.py

```
"""
Prescriptive Maintenance Recommendations Module.
Provides structured diagnostic actions, root cause hypotheses, and maintenance procedures
for detected fault conditions.
"""

from typing import Dict, Any, List

FAULT_RECOMMENDATIONS: Dict[str, Dict[str, Any]] = {
    "Normal": {
        "severity": "NORMAL",
        "short_action": "No immediate intervention required. Maintain routine preventive inspections.",
        "root_causes": ["System operating within designed nominal operational envelope."],
        "checklist": [
            "Log operational telemetry hours for routine service scheduling.",
            "Verify lubrication levels every 500 operating hours.",
            "Inspect physical chassis for dust accumulation."
        ],
        "urgency": "Routine",
        "estimated_downtime_minutes": 0,
    },
    "Mechanical Overload": {
        "severity": "WARNING",
        "short_action": "Reduce conveyor/load torque demand; inspect mechanical drivetrain for binding or foreign body jamming.",
        "root_causes": [
            "Excess mechanical load exceeding 12V DC motor rated torque capacity.",
            "Bearing dry friction or gearbox misalignment causing mechanical binding.",
            "Foreign object debris obstructing mechanical output shaft."
        ],
        "checklist": [
            "De-energize drive and rotate shaft manually to feel for binding or resistance.",
            "Measure DC bus current with an external clamp meter to verify ACS712 accuracy.",
            "Check conveyor belt tension and gearbox lubrication.",
            "Verify motor rated continuous current (0.80 A) is not continuously exceeded."
        ],
        "urgency": "Medium — Resolve within shift to prevent thermal degradation",
        "estimated_downtime_minutes": 15,
    },
    "Excessive Vibration": {
        "severity": "WARNING",
        "short_action": "Inspect motor baseplate fasteners, flexible shaft coupling, and dynamic rotor balance.",
        "root_causes": [
            "Loose mounting bolts on the motor chassis.",
            "Shaft angular or parallel misalignment relative to the driven load.",
            "Dynamic imbalance on rotating pulley, gear, or cooling fan.",
            "Early stage bearing race pit or ball defect."
        ],
        "checklist": [
            "Torque all chassis mounting bolts to manufacturer specifications.",
            "Use dial indicators or alignment shims to verify shaft collinearity.",
            "Inspect rotating fan blades or pulleys for chipped edges or eccentricity.",
            "Perform acoustic bearing check with stethoscope or contact probe."
        ],
        "urgency": "Medium — Inspect before next operational cycle",
        "estimated_downtime_minutes": 20,
    },
    "Motor Overheating": {
        "severity": "CRITICAL",
        "short_action": "Immediately de-energize motor or reduce duty cycle; inspect forced air cooling and ambient thermal environment.",
        "root_causes": [
            "Prolonged operation under high continuous armature current.",
            "Obstructed cooling vents or fan failure.",
            "High ambient operating temperature (>45°C) reducing heat dissipation.",
            "Internal winding insulation deterioration leading to eddy current heating."
        ],
        "checklist": [
            "Safely de-energize and allow stator windings to cool below 45°C.",
            "Clean motor housing cooling fins and clear dust filters.",
```

diagnosis/recommendations.py (continued)

```
            "Verify DS18B20 thermal contact pad and thermal paste integrity.",
            "Check winding resistance with multimeter to rule out shorted turns."
        ],
        "urgency": "High — Immediate action required to prevent winding insulation burnout",
        "estimated_downtime_minutes": 30,
    },
    "Motor Stall": {
        "severity": "CRITICAL",
        "short_action": "IMMEDIATE EMERGENCY SHUTDOWN: De-energize L298N driver to prevent locked-rotor winding burnout.",
        "root_causes": [
            "Complete mechanical lock of the driven mechanism.",
            "Severe internal gearbox teeth fracture or bearing seizure.",
            "Sudden overload torque exceeding motor locked-rotor stall torque."
        ],
        "checklist": [
            "Immediately disable H-Bridge enable pin (ENA/ENB on L298N).",
            "Disconnect power supply and inspect for mechanical obstruction.",
            "Check L298N driver heatsink for thermal stress or damage.",
            "Verify DC power supply overcurrent protection tripped as intended."
        ],
        "urgency": "CRITICAL — Emergency stop",
        "estimated_downtime_minutes": 45,
    },
    "Motor Slowdown": {
        "severity": "WARNING",
        "short_action": "Inspect DC supply voltage under load and check for incipient mechanical friction.",
        "root_causes": [
            "Supply rail voltage sag under load due to high source impedance.",
            "Partial mechanical drag on driven conveyor.",
            "Brush wear or commutator oxidation in brushed DC motor."
        ],
        "checklist": [
            "Measure 12V terminal voltage across motor leads under running load.",
            "Inspect optical tachometer disc for dust or misalignment.",
            "Check carbon brush contact pressure and commutator surface condition."
        ],
        "urgency": "Medium — Schedule maintenance within 24 hours",
        "estimated_downtime_minutes": 15,
    },
    "Sensor Error": {
        "severity": "UNKNOWN / SENSOR ERROR",
        "short_action": "Inspect sensor harness, I2C bus pullups, OneWire pullup resistor, and ADC analog ground.",
        "root_causes": [
            "Loose DuPont wire on MPU6050 SDA/SCL or DS18B20 DQ pin.",
            "Missing 4.7kΩ pull-up resistor on DS18B20 OneWire line.",
            "Electromagnetic interference (EMI) coupling from motor PWM into ADC.",
            "ACS712 zero-current offset drift or loose analog input connection."
        ],
        "checklist": [
            "Inspect physical wiring headers and ensure tight breadboard/screw terminal connection.",
            "Confirm common ground between 12V motor supply and ESP32 3.3V/GND.",
            "Check 3.3V logic supply rail stability with multimeter or oscilloscope.",
            "Re-calibrate ACS712 Vref with motor de-energized."
        ],
        "urgency": "High — Monitoring fidelity is compromised",
        "estimated_downtime_minutes": 10,
    },
}


def get_recommendation_for_fault(fault_name: str) -> Dict[str, Any]:
    """Returns detailed maintenance recommendations and checklist for a fault."""
    for key, data in FAULT_RECOMMENDATIONS.items():
        if key.lower() in fault_name.lower():
            return data
    # Fallback
    return {
        "severity": "WARNING",
        "short_action": "Perform physical inspection of motor asset and cross-reference multi-sensor
```

diagnosis/recommendations.py (continued) (continued)

```
     readings.",
138          "root_causes": ["Unclassified operational anomaly detected."],
139          "checklist": ["Inspect mechanical coupling.", "Check electrical connections.", "Review telemetry trend
     logs."],
140          "urgency": "Medium",
141          "estimated_downtime_minutes": 15,
142      }
```

diagnosis/rule_engine.py

```python
"""
Knowledge-Driven Rule Engine
Evaluates declarative physics and engineering rules against incoming motor telemetry.
Provides deterministic, explainable diagnostic hypotheses and recommended actions.
"""

from typing import Dict, Any, List, Optional
from pathlib import Path
import yaml
from config.settings import settings


class RuleEngine:
    """
    Expert system rule evaluator. Loads declarative rules from YAML,
    ranks by priority, evaluates condition expressions, and resolves conflicts.
    """

    def __init__(self, kb_path: Path = settings.KNOWLEDGE_BASE_FILE):
        self.kb_path = Path(kb_path)
        self.rules: List[Dict[str, Any]] = []
        self.load_knowledge_base()

    def load_knowledge_base(self) -> None:
        """Loads and sorts rules by priority descending."""
        if not self.kb_path.exists():
            raise FileNotFoundError(f"Knowledge base file not found at {self.kb_path}")
        with open(self.kb_path, "r", encoding="utf-8") as f:
            data = yaml.safe_load(f) or {}
            self.rules = data.get("rules", [])
            # Sort by priority descending (higher priority evaluated first)
            self.rules.sort(key=lambda r: r.get("priority", 0), reverse=True)

    def _eval_condition(self, expr: str, context: Dict[str, Any]) -> bool:
        """
        Safely evaluates condition expression against telemetry context dictionary.
        Only allows numerical comparisons and logical operators.
        """
        # Restrict environment to arithmetic and boolean logic
        safe_names = {
            "temperature_c": context.get("temperature_c", 0.0),
            "vibration_rms_g": context.get("vibration_rms_g", 0.0),
            "current_a": context.get("current_a", 0.0),
            "rpm": context.get("rpm", 0.0),
            "sensor_status": context.get("sensor_status", "ok"),
        }
        try:
            return bool(eval(expr, {"__builtins__": {}}, safe_names))
        except Exception:
            return False

    def evaluate(self, reading: Dict[str, Any]) -> Dict[str, Any]:
        """
        Evaluates a motor telemetry reading against the knowledge base rules.
        Returns the highest-priority triggered hypothesis with full explainability.
        """
        activated_rules = []
        best_hypothesis: Optional[Dict[str, Any]] = None

        for rule in self.rules:
            conds = rule.get("conditions", {})
            matched = False

            if "all" in conds:
                matched = all(self._eval_condition(c, reading) for c in conds["all"])
            elif "any" in conds:
                matched = any(self._eval_condition(c, reading) for c in conds["any"])

            if matched:
                activated_rules.append(rule["rule_id"])
                if best_hypothesis is None:
```

diagnosis/rule_engine.py (continued)

```
                    hyp = rule.get("hypothesis", {}).copy()
                    hyp["rule_id"] = rule["rule_id"]
                    hyp["rule_name"] = rule["name"]
                    hyp["priority"] = rule.get("priority", 0)
                    best_hypothesis = hyp

        # If no rule triggered, fallback to indeterminate warning
        if not best_hypothesis:
            best_hypothesis = {
                "rule_id": "DEFAULT_UNKNOWN",
                "rule_name": "Indeterminate Telemetry Profile",
                "suspected_fault": "Indeterminate / Unclassified State",
                "severity": "WARNING",
                "confidence": 0.50,
                "explanation": "Current sensor values do not cleanly match predefined operating envelopes.",
                "recommended_action": "Cross-reference with ML classifier and observe trend trajectory.",
                "priority": 0,
            }

        # Build triggering evidence summary
        evidence = {
            "temperature_c": reading.get("temperature_c"),
            "vibration_rms_g": reading.get("vibration_rms_g"),
            "current_a": reading.get("current_a"),
            "rpm": reading.get("rpm"),
            "sensor_status": reading.get("sensor_status"),
        }

        return {
            "suspected_fault": best_hypothesis["suspected_fault"],
            "severity": best_hypothesis["severity"],
            "confidence": best_hypothesis["confidence"],
            "explanation": best_hypothesis["explanation"],
            "recommended_action": best_hypothesis["recommended_action"],
            "triggering_evidence": evidence,
            "activated_rules": activated_rules,
            "primary_rule_id": best_hypothesis.get("rule_id"),
        }


rule_engine = RuleEngine()

```

docs/API.md

```
# REST API Documentation — Knowledge-Driven IoT Fault Diagnosis System

The backend provides a FastAPI-powered REST API (Port 8000) for telemetry queries, hybrid diagnoses, alert lifecycle management, and system health status.

---

## 1. Authentication Endpoints

### `POST /api/auth/login`
Authenticates an operator or [REDACTED]istrator using salted PBKDF2 hashing.

- **Request Body:**
  ```json
  {
    "username": "[REDACTED]",
    "password": "[REDACTED]"
  }
  ```
- **Response (200 OK):**
  ```json
  {
    "token": "64_char_hex_session_token",
    "username": "[REDACTED]",
    "role": "[REDACTED]",
    "expires_in": 86400
  }
  ```
- **Error Responses:**
  - `401 Unauthorized`: Invalid credentials.
  - `429 Too Many Requests`: Rate limit exceeded (more than 5 failed attempts in 5 minutes).

---

2. Telemetry & Analytics Endpoints

`GET /api/telemetry/recent`
Retrieves recent sensor readings for a specified motor.

- **Query Parameters:**
  - `device_id` (string, optional, default: `motor01`)
  - `limit` (integer, optional, default: `100`, max: `1000`)
- **Response (200 OK):**
  ```json
  [
    {
      "id": 104,
      "event_id": "motor01-1730289600-0012",
      "device_id": "motor01",
      "timestamp": "2026-10-30T12:00:12.000Z",
      "source": "synthetic",
      "temperature_c": 38.6,
      "vibration_rms_g": 0.11,
      "current_a": 0.58,
      "rpm": 1985.0,
      "sensor_status": "ok"
    }
  ]
  ```

`GET /api/telemetry/range`
Queries historical telemetry over an ISO-8601 time window.

- **Query Parameters:**
  - `device_id` (string, required)
  - `start_iso` (string, required, e.g. `2026-10-30T00:00:00Z`)
  - `end_iso` (string, required, e.g. `2026-10-30T23:59:59Z`)
  - `limit` (integer, optional, default: `1000`)

```

docs/API.md (continued)

```
3. Diagnostic & Alerts Endpoints

`GET /api/diagnosis/recent`
Returns recent hybrid rule & machine learning diagnostic verdicts.

- **Query Parameters:**
  - `device_id` (string, optional, default: `motor01`)
  - `limit` (integer, optional, default: `50`)
- **Response (200 OK):**
  ```json
  [
    {
      "id": 42,
      "event_id": "motor01-1730289600-0012",
      "device_id": "motor01",
      "timestamp": "2026-10-30T12:00:12.000Z",
      "rule_fault": "Normal",
      "rule_severity": "NORMAL",
      "ml_fault": "Normal",
      "ml_confidence": 0.985,
      "hybrid_fault": "Normal",
      "hybrid_severity": "NORMAL",
      "explanation": "Consensus reached. All operational metrics within design envelopes.",
      "recommended_action": "No intervention required.",
      "conflict_detected": 0
    }
  ]
  ```

`GET /api/alerts`
Retrieves logged alerts from `alert_history`.

- **Query Parameters:**
  - `device_id` (string, optional)
  - `only_unacknowledged` (boolean, optional, default: `false`)
  - `limit` (integer, optional, default: `50`)

`POST /api/alerts/{alert_id}/acknowledge`
Marks an alert as acknowledged by an operator. Requires Bearer Token authentication.

- **Headers:** `Authorization: Bearer <token>`
- **Response (200 OK):**
  ```json
  {
    "status": "success",
    "alert_id": 14,
    "acknowledged_by": "operator"
  }
  ```

---

4. System Health & ML Metadata

`GET /api/health`
Returns system health, MQTT broker status, SQLite accessibility, and ML model status.

- **Response (200 OK):**
  ```json
  {
    "status": "healthy",
    "environment": "development",
    "operation_mode": "simulation",
    "mqtt_broker": {
      "type": "local_fallback",
      "host": "127.0.0.1",
      "port": 1883,
      "tls": false
    },
    "database": {
      "type": "SQLite WAL",
```
```

docs/API.md (continued) (continued)

```
142        "accessible": true
143      },
144      "ml_model": {
145        "loaded": true,
146        "classes": ["Excessive Vibration", "Mechanical Overload", "Motor Overheating", "Motor Slowdown", "Motor
     Stall", "Normal", "Sensor Error"]
147      }
148    }
149    ```
150
151  ### `GET /api/ml/metrics`
152  Returns machine learning evaluation metrics, confusion matrix, and feature rankings.
```

docs/ARCHITECTURE.md

````
# System Architecture — Knowledge-Driven IoT Fault Diagnosis System

## 1. Executive Summary
The **Knowledge-Driven IoT-Based Fault Diagnosis System** is an industrial cyber-physical monitoring platform designed for a 12V DC motor asset. The architecture integrates edge sensor telemetry acquisition, cloud and local publish-subscribe messaging via MQTT, time-series data deduplication and persistence in SQLite WAL, industrial automation orchestration with Node-RED, and a hybrid AI diagnostic engine combining deterministic engineering rules with scikit-learn machine learning.

---

## 2. Six-Layer Modular Architecture

```mermaid
flowchart TD
    subgraph Edge["Physical Edge Layer & Simulation (Layer 1 & Layer 6)"]
        ESP["ESP32 DevKit Microcontroller"]
        S1["DS18B20 Temp (°C)"] --> ESP
        S2["MPU6050 Vibration (RMS g)"] --> ESP
        S3["ACS712 Current (A)"] --> ESP
        S4["Optical Slot RPM"] --> ESP
        SIM["Python Synthetic Data Generator"]
    end

    subgraph Messaging["Message Transport Layer (Layer 1 & Layer 2)"]
        MQTT_LOCAL["Local Mosquitto Broker (1883)"]
        MQTT_CLOUD["HiveMQ Cloud Broker (8883 TLS)"]
    end

    ESP -->|WiFi JSON| MQTT_CLOUD
    SIM -->|Paho MQTT| MQTT_LOCAL

    subgraph Orchestration["Automation & Orchestration Layer (Layer 4)"]
        NRED["Node-RED Engine"]
        NRED -->|Deduplication & Validation| DEBOUNCE["Consecutive Fault Debounce (N=3)"]
        DEBOUNCE -->|Cooldown 15s| NRED_ALERT["Recovery & Alert Event Publisher"]
    end

    MQTT_LOCAL <-->|iot/factory/+/sensors| NRED
    MQTT_CLOUD <-->|iot/factory/+/sensors| NRED

    subgraph CoreBackend["Storage & Diagnostics Engine (Layer 3 & Layer 5)"]
        INGEST["Ingestion Service (mqtt_subscriber.py)"]
        CSV["CSV Logger (sensor_telemetry.csv)"]
        SQLITE[("SQLite Database (WAL Mode)")]
        RULE_ENG["Expert Rule Engine (YAML Rules)"]
        ML_ENG["Random Forest Classifier (Rolling Physics Features)"]
        HYBRID["Hybrid Decision Reconciler"]
    end

    MQTT_LOCAL --> INGEST
    MQTT_CLOUD --> INGEST
    INGEST --> CSV
    INGEST -->|Unique event_id| SQLITE
    INGEST --> RULE_ENG
    INGEST --> ML_ENG
    RULE_ENG --> HYBRID
    ML_ENG --> HYBRID
    HYBRID -->|Prescriptive Diagnosis| SQLITE
    HYBRID -->|iot/factory/+/diagnosis| MQTT_LOCAL

    subgraph Application["Presentation & Operations Layer (Layer 5 & Layer 6)"]
        FASTAPI["FastAPI Backend (Port 8000)"]
        DASH["Streamlit Industrial Monitoring Portal (Port 8501)"]
        ACTUATORS["ESP32 Warning LED & Piezo Buzzer"]
    end

    SQLITE --> FASTAPI
    SQLITE --> DASH
    NRED_ALERT -->|iot/factory/+/alerts| ACTUATORS
    HYBRID -->|iot/factory/+/alerts| ACTUATORS
````
````

docs/ARCHITECTURE.md (continued)

```
```

---

3. Communication Topic Topology

The MQTT topic hierarchy follows a clean industrial schema:

MQTT Topic	Direction	Payload Schema	Purpose
`iot/factory/{device_id}/sensors`	Pub: ESP32 / Sim Sub: Ingest / Node-RED	Standard Telemetry JSON	Raw sensor readings (T, V, I, RPM, status).
`iot/factory/{device_id}/status`	Pub: Device (LWT)	`{"device_id": "...", "status": "online/offline"}`	Device heartbeat and Last Will & Testament.
`iot/factory/{device_id}/diagnosis`	Pub: Hybrid Engine	Hybrid Diagnosis JSON	Inferred fault hypothesis, severity, confidence, and action.
`iot/factory/{device_id}/alerts`	Pub: Ingest / Node-RED Sub: Dashboard / ESP32	Alert JSON	Debounced notifications triggering physical buzzers or alarms.
`iot/factory/{device_id}/commands`	Pub: Operator Sub: ESP32	`{"command": "emergency_stop"}`	Remote safety de-energization.
`iot/factory/{device_id}/command_ack`	Pub: ESP32 Sub: Backend	`{"status": "executed"}`	Hardware command execution acknowledgement.

---

4. Normalized Sensor Message Schema

Every telemetry packet adheres to the following JSON specification:

```json
{
  "event_id": "motor01-1730289600-0001",
  "device_id": "motor01",
  "timestamp": "2026-10-30T12:00:00.000Z",
  "source": "synthetic",
  "temperature_c": 38.5,
  "vibration_rms_g": 0.12,
  "current_a": 0.62,
  "rpm": 1980.0,
  "sensor_status": "ok"
}
```

---

5. Persistence Architecture (SQLite in WAL Mode)

SQLite is configured with `PRAGMA journal_mode = WAL` (Write-Ahead Logging) and `PRAGMA foreign_keys = ON`.
- **Concurrency**: WAL mode allows concurrent readers while a single background writer commits transactions without locking contention.
- **Deduplication**: `sensor_readings.event_id` enforces a `UNIQUE` constraint, preventing duplicate insertion when both Node-RED and direct subscriber are active.
- **Indexing**: Composite indexes on `(device_id, timestamp DESC)` enable sub-millisecond retrieval of recent telemetry and historical time windows.

---

6. Hybrid Intelligence Engine

The diagnostic engine reconciles deterministic engineering rules with a multi-class Random Forest model:
1. **Rule Engine**: Evaluates physics-based threshold boundaries defined in `config/knowledge_base.yaml`.
2. **Machine Learning Model**: Analyzes rolling temporal features (rate of temperature rise, vibration acceleration variance, torque-to-speed ratios).
3. **Consensus & Conflict Handling**:
   - When Rule Engine and ML reach consensus, diagnostic confidence is bolstered ($>90\%$).
   - When a discrepancy occurs, the system adopts a conservative safety stance (choosing the higher severity verdict), records the conflict flag, and lowers output confidence to alert human operators to uncertainty.

docs/HARDWARE_WIRING.md

```
# Hardware Wiring Guide & Electrical Safety Specifications

## 1. Schematic Overview

```
                     +-------------------+
                     | 12V 2A DC Supply  |
                     +---+-----------+---+
                         | +12V      | GND (Common Ground)
                         |           |
             +-------------+-----------+---------------------+
             |                         |                     |
             v +12V                    v GND                 |
      +--------------+          +--------------+             |
      | L298N Driver |          | ESP32 DevKit |             |
      |              |          |              |             |
      | ENA <--------+-- GPIO19 | 3V3 -------->| Power Rails |
      | IN1 <--------+-- GPIO25 | GND ---------+ (Shared GND)|
      | IN2 <--------+-- GPIO26 |              |             |
      | OUT1/OUT2    |          | GPIO21 (SDA)-+-> MPU6050   |
      +-------+------+          | GPIO22 (SCL)-+-> MPU6050   |
              |                 |              |             |
              v                 | GPIO4 (DQ) --+-> DS18B20   |
        [12V DC Motor]          |              |  (4.7k Pull)|
              |                 | GPIO36 (ADC)-+<- Voltage   |
              +-> [ACS712 5A] ->|              |   Divider   |
                    (VCC=5V)    | GPIO18 (IRQ)-+<- IR Sensor |
                                |              |             |
                                | GPIO2 (LED) -+-> Red LED   |
                                | GPIO15 (BUZZ)+-> Buzzer    |
                                +--------------+             |
                                       |                     |
                                       +---------------------+
```

---

2. Complete Pin Connection Matrix

ESP32 Pin	Function	Sensor / Peripheral	Peripheral Pin	Notes & Conditioning
3V3	3.3V Power Out	MPU6050, DS18B20	VCC / VDD	Connect to 3.3V bus on breadboard.
GND	Ground Reference	All sensors, L298N, PS	GND	**Common Ground** shared across all supplies.
GPIO 21	I2C Data (SDA)	MPU6050	SDA	Standard ESP32 hardware I2C data.
GPIO 22	I2C Clock (SCL)	MPU6050	SCL	Standard ESP32 hardware I2C clock.
GPIO 4	1-Wire Digital	DS18B20	DQ (Data)	Requires **4.7kΩ pull-up resistor** to 3V3.
GPIO 36	Analog ADC1 CH0	ACS712-05B	OUT (via divider)	**CRITICAL: Connect via 10k/20k voltage divider.**
GPIO 18	External Interrupt	Optical Slot Sensor	OUT / DO	Measures pulses from slotted encoder disc.
GPIO 2	Digital Output	Status Warning LED	Anode	Connect via 330Ω current-limiting resistor to GND.
GPIO 15	Digital Output	Piezo Buzzer	Base (2N2222)	Drive via NPN transistor to prevent over-current.
GPIO 19	PWM Output	L298N	ENA	PWM speed modulation (frequency 5 kHz, 8-bit).
GPIO 25	Digital Output	L298N	IN1	Motor direction control terminal 1.
GPIO 26	Digital Output	L298N	IN2	Motor direction control terminal 2.

---

3. Detailed Circuit Conditioning Diagrams

A. ACS712 Voltage Divider Circuit
ACS712 is powered by 5V and outputs $2.5\,\text{V} \pm 185\,\text{mV/A}$. At maximum current, output can reach $5\,\text{V}$.
```
ACS712 (OUT) ----[ 10 kΩ (R1) ]----+-----> ESP32 GPIO 36 (ADC1 CH0)
                                   |
                              [ 20 kΩ (R2) ]
                                   |
                                  GND
```
```

docs/HARDWARE_WIRING.md (continued)

````
67
68  Division Ratio = 20k / (10k + 20k) = 0.6667
69  Max Voltage at GPIO 36 = 5.0V * 0.6667 = 3.33V (Safe for ESP32 ADC)
70  ```
71
72  ### B. DS18B20 1-Wire Temperature Pull-Up Circuit
73  ```
74       3.3V Rail
75          |
76        [ 4.7 kΩ ]
77          |
78  DS18B20 (DQ) ----------------------------> ESP32 GPIO 4
79  ```
80
81  ### C. Buzzer Transistor Driver Circuit
82  ```
83  ESP32 GPIO 15 ----[ 1 kΩ ]----> Base (2N2222 NPN)
84                                  Collector ----[ Piezo Buzzer ]----> 5V / 12V
85                                  Emitter -------------------------> GND
86  ```
87
88  ---
89
90  ## 4. Pre-Power Electrical Checklist
91
92  - [ ] Verify common ground (`GND`) links the 12V supply, ESP32, L298N, and sensors.
93  - [ ] Measure resistance across voltage divider ($R_1 = 10\,\text{k}\Omega$, $R_2 = 20\,\text{k}\Omega$) before
     connecting to GPIO 36.
94  - [ ] Ensure 4.7kΩ pull-up resistor is installed on DS18B20 data line.
95  - [ ] Verify L298N onboard 5V regulator jumper is enabled if supplying logic from L298N.
96  - [ ] Confirm no stray DuPont wires short 12V rail to 3.3V logic pins.
````

docs/LAYER_1.md

```
# Layer 1 — Synthetic Telemetry → MQTT → CSV Logging → Live Dashboard

## 1. Objective
Establish the baseline simulation monitoring pipeline. The system generates realistic, physics-grounded telemetry for a 12V DC motor, transmits readings through a local MQTT broker, logs records into thread-safe CSV files, and provides a live updating Streamlit dashboard without requiring physical hardware or cloud credentials.

---

## 2. Components Implemented

### A. Physics-Grounded Synthetic Generator (`simulation/synthetic_generator.py`)
Generates 7 discrete operational profiles:
1. **Normal Motor Operation**: Nominal armature current ($0.45 - 0.65\,\text{A}$), steady shaft speed ($1950 - 2050\,\text{RPM}$), stable temperature ($35 - 42^\circ\text{C}$), low vibration ($0.05 - 0.12\,g$).
2. **Mechanical Overload**: Elevated current ($1.10 - 1.45\,\text{A}$), reduced shaft speed ($1300 - 1650\,\text{RPM}$), gradual thermal rise.
3. **Excessive Vibration**: High dynamic acceleration ($0.45 - 0.85\,g$) simulating loose mounting or coupling misalignment.
4. **Motor Overheating**: Steep thermal trajectory ($65 - 85^\circ\text{C}$) under continuous elevated load.
5. **Motor Slowdown**: Speed degradation ($800 - 1200\,\text{RPM}$) due to DC bus voltage sag or brush resistance.
6. **Motor Stall (Locked Rotor)**: Massive locked-rotor current surge ($1.80 - 2.20\,\text{A}$), speed collapsing to $0 - 50\,\text{RPM}$.
7. **Sensor Error**: Intermittent acquisition dropouts, out-of-range ADC artifacts, or `sensor_status = "error"`.

### B. MQTT Publisher (`simulation/mqtt_publisher.py`)
- Configurable publication frequency (default 1 Hz).
- CLI scenario selection (`--scenario stall`, `--scenario overload`, etc.).
- Reproducible random seeds for deterministic academic benchmarking.

### C. Thread-Safe CSV Logging (`ingestion/csv_logger.py`)
- Appends telemetry to `data/csv/sensor_telemetry.csv`.
- Automatic CSV header initialization and file rotation safety.

### D. Live Streamlit Visualization (`dashboard/pages/2_Live_Monitoring.py`)
- Polling-based auto-refreshing real-time line charts (Plotly).
- Visual indicators for machine status, active scenario, and connection latency.

---

## 3. Verification & Execution
To run Layer 1 independently:
```bash
./scripts/run_layer1.sh
```
Or execute the automated unit test suite:
```bash
./.venv/bin/pytest tests/test_simulation.py -v
```
```

docs/LAYER_2.md

````
# Layer 2 — Cloud MQTT Integration (HiveMQ Cloud TLS) with Local Fallback

## 1. Objective
Upgrade the communication pipeline from local unencrypted MQTT to production-grade cloud messaging using **HiveMQ Cloud** with TLS v1.2/1.3 encryption (Port 8883) and username/password authentication, while ensuring automatic, graceful fallback to a local broker during offline or credentials-free operation.

---

## 2. Key Architecture & Features

### A. Managed MQTT Client (`ingestion/mqtt_client.py`)
- **Transport Security**: Wraps sockets in TLS using standard system CA certificates when connecting to port 8883.
- **Resilient Reconnection**: Implements exponential backoff ($1\,\text{s} \to 2\,\text{s} \to 4\,\text{s} \to \dots \to 30\,\text{s}$) to gracefully handle network drops.
- **Connection Health & Latency**: Measures round-trip ping times and logs broker disconnect reasons.
- **Offline / Local Broker Fallback**: If HiveMQ Cloud credentials are unconfigured or connection times out, the client automatically switches to local Mosquitto (`127.0.0.1:1883`), preventing unhandled crashes.

### B. Environment Configuration (`config/settings.py` & `.env.example`)
All credentials are decoupled from source code:
```env
# MQTT Mode: set USE_LOCAL_BROKER=false to enable HiveMQ Cloud
USE_LOCAL_BROKER=true
LOCAL_MQTT_HOST=127.0.0.1
LOCAL_MQTT_PORT=1883

# HiveMQ Cloud Credentials (Port 8883 with TLS)
HIVEMQ_HOST=xxxxxx.s1.eu.hivemq.cloud
HIVEMQ_PORT=8883
HIVEMQ_USERNAME=[REDACTED]
HIVEMQ_PASSWORD=[REDACTED]
HIVEMQ_USE_TLS=true
```

---

## 3. Verification & Execution
To verify Layer 2 connectivity and fallback:
```bash
./scripts/run_layer2.sh
```
Or run the MQTT test suite:
```bash
./.venv/bin/pytest tests/test_mqtt.py -v
```
````

docs/LAYER_3.md

```
# Layer 3 — SQLite Historical Database & Analytics Repository

## 1. Objective
Establish an ACID-compliant, persistent relational database using SQLite in Write-Ahead Logging (WAL) mode. The database serves as the single source of truth for telemetry streams, hybrid diagnostic predictions, active and historical alerts, operator acknowledgements, user access control, and audit logs.

---

## 2. Relational Schema Architecture

```mermaid
erDiagram
    DEVICES ||--o{ SENSOR_READINGS : emits
    DEVICES ||--o{ FAULT_PREDICTIONS : evaluates
    DEVICES ||--o{ FAULT_EVENTS : logs
    DEVICES ||--o{ ALERT_HISTORY : triggers

    DEVICES {
        text device_id PK
        text name
        text type
        text driver
        real rated_voltage_v
        real rated_current_a
        real rated_rpm
        text location
        text created_at
    }

    SENSOR_READINGS {
        integer id PK
        text event_id UK
        text device_id FK
        text timestamp
        text source
        real temperature_c
        real vibration_rms_g
        real current_a
        real rpm
        text sensor_status
        text created_at
    }

    FAULT_PREDICTIONS {
        integer id PK
        text event_id
        text device_id FK
        text timestamp
        text rule_fault
        text rule_severity
        text ml_fault
        real ml_confidence
        text hybrid_fault
        text hybrid_severity
        text explanation
        text recommended_action
        integer conflict_detected
        text created_at
    }

    ALERT_HISTORY {
        integer id PK
        text device_id
        text timestamp
        text severity
        text title
        text message
        integer acknowledged
        text acknowledged_by
        text acknowledged_at
```
```

docs/LAYER_3.md (continued)

```
        text created_at
    }

    USERS {
        integer id PK
        text username UK
        text password_hash
        text salt
        text role
        integer is_active
        text created_at
        text last_login
    }

    SYSTEM_EVENTS {
        integer id PK
        text timestamp
        text event_type
        text component
        text message
        text details
        text created_at
    }
```

---

3. Key Design Decisions

1. **Write-Ahead Logging (WAL)**: `PRAGMA journal_mode = WAL` enables concurrent readers while a single writer operates without blocking the UI.
2. **Event Deduplication**: The `event_id UNIQUE` constraint drops duplicate telemetry packets originating from network retransmissions.
3. **Historical Time-Range Queries**: The composite index `(device_id, timestamp DESC)` ensures efficient historical analytics and slicing across arbitrary date-time windows.
4. **Data Migrations**: Versioned SQL migrations (`database/migrations/001_initial_schema.sql`) are applied deterministically via `database/migrations/migrate.py`.

---

4. Verification & Execution
To run database migrations and execute the verification test suite:
```bash
./scripts/run_layer3.sh
```
Or run the dedicated pytest suite:
```bash
./.venv/bin/pytest tests/test_database.py -v
```

docs/LAYER_4.md

```
# Layer 4 — Node-RED Industrial Automation, Hysteresis & Alerts

## 1. Objective
Integrate **Node-RED** as the industrial automation and event-orchestration layer. The system subscribes to raw telemetry topics, enforces payload validation, tracks device health states, applies consecutive fault hysteresis ($N=3$) and cooldown timers ($15\,\text{s}$), suppresses duplicate alarms, publishes actionable alert events, and sends recovery notifications.

---

## 2. Industrial Orchestration Pipeline

```mermaid
flowchart LR
    MQTT_IN["MQTT In\n(iot/factory/+/sensors)"] --> PARSE["JSON Parser"]
    PARSE --> VALIDATE{"Schema & Cache\nValidation"}
    VALIDATE -->|Invalid / Duplicate| DROP["Suppressed / Logged"]
    VALIDATE -->|Valid| EVAL["Hysteresis & Alert\nEvaluator"]
    EVAL -->|Consecutive >= 3 or CRITICAL| ALERT_OUT["MQTT Out Alerts\n(iot/factory/+/alerts)"]
    EVAL -->|Telemetry Nominal after Fault| RECOVERY["MQTT Out Recovery\n(iot/factory/+/alerts)"]
    EVAL -->|Debouncing Count < 3| ACCUM["Increment State Counter"]
```

---

3. Advanced Alert Management Logic

A. Consecutive Fault Debouncing ($N = 3$)
To prevent transient spikes (e.g., normal motor inrush current upon startup) from triggering nuisance alarms, the engine requires 3 consecutive anomalous telemetry readings before raising a persistent warning.
- *Exception*: Immediate `CRITICAL` conditions (e.g., Locked-Rotor Motor Stall or Severe Overheating) bypass the debouncer to protect machinery immediately.

B. Cooldown Timer ($15\,\text{s}$)
Once an alert is dispatched, an alert cooldown timer prevents flooding the MQTT broker, database, and dashboard with redundant notifications.

C. State Hysteresis & Recovery
When sensor values return within nominal safe thresholds after an active fault state, the system automatically dispatches a `severity: NORMAL` recovery message, notifying operators that the condition has resolved.

---

4. Verification & Execution
To run the automated hysteresis, cooldown, and recovery unit tests:
```bash
./scripts/run_layer4.sh
```
Or run the dedicated alert test suite:
```bash
./.venv/bin/pytest tests/test_alerts.py -v
```

To run Node-RED via Docker Compose:
```bash
docker compose up nodered -d
```
Access the web canvas at `http://localhost:1880`.
```

docs/LAYER_5.md

````
# Layer 5 — Knowledge-Driven AI/ML, User Authentication & Monitoring Portal

## 1. Objective
Implement the intelligent diagnostic core and modern industrial monitoring interface. This layer pairs deterministic, physics-grounded engineering rules with a scikit-learn machine learning pipeline (Random Forest classifier with rolling temporal features), reconciles them via a transparent hybrid consensus engine, protects access with salted PBKDF2 authentication, and renders a dark industrial SaaS monitoring portal in Streamlit.

---

## 2. Hybrid Diagnostic Architecture

```mermaid
flowchart TD
    RAW["Raw Telemetry + Sliding Window History"] --> RE["Rule Engine (YAML Rules)"]
    RAW --> FE["Feature Extractor (Rolling Physics Trends)"]
    FE --> ML["Random Forest Model (100 Estimators)"]

    RE -->|Suspected Fault, Severity, Confidence| RECON["Hybrid Consensus Reconciler"]
    ML -->|Predicted Class, Softmax Probabilities| RECON

    RECON -->|Unanimous Agreement| CONSENSUS["Confidence Boost (>90%)\nPrescriptive Maintenance Action"]
    RECON -->|Conflict Flagged| CONFLICT["Conservative Safety Fallback\nHighest Severity Selected\nUncertainty Penalty (<80%)"]

    CONSENSUS --> PERSIST[("Store in SQLite & Dispatch MQTT")]
    CONFLICT --> PERSIST
```

---

## 3. Machine Learning Pipeline & Rolling Features

### A. Physics-Informed Temporal Features (`ml/features.py`)
1. `temperature_c`: Instantaneous thermal reading.
2. `temp_trend`: 5-sample linear regression slope ($\Delta T / \Delta t$).
3. `vibration_rms_g`: Dynamic vibration acceleration RMS.
4. `vibe_trend`: Rate of vibration growth.
5. `current_a`: Instantaneous armature current.
6. `current_delta`: Current deviation from rolling window baseline.
7. `rpm`: Shaft rotational speed.
8. `rpm_delta`: Speed deviation from rolling window baseline.
9. `power_est_w`: Estimated input electrical power ($P = 12\,\text{V} \times I$).
10. `current_per_rpm`: Electromechanical load ratio ($I / \text{RPM}$).

### B. Model Performance Comparison
Evaluated on held-out simulated operating episodes (group-split, zero leakage):

| Model Architecture | Test Accuracy | Weighted F1 | Latency / Sample |
|---|---|---|---|
| Baseline Decision Tree (Depth=5) | 98.42% | 0.9840 | ~0.08 ms |
| **Random Forest Classifier (N=100)** | **100.00%** | **1.0000** | **~0.42 ms** |

> [!NOTE]
> *Academic Rigor Note:* 100% synthetic classification accuracy reflects the clear boundaries of simulated physics models. When deployed on physical hardware, sensor noise, ambient thermal gradients, and electrical switching transients require calibration.

---

## 4. Multi-Page Streamlit Monitoring Portal

Styled with a dark industrial charcoal theme (`#0d1117`, `#161b22`, `#58a6ff`):
1. **Overview**: Key asset health metric cards, active fault status, broker connectivity badges.
2. **Live Monitoring**: Real-time multi-sensor Plotly charts with auto-polling.
3. **Fault Diagnosis**: Detailed breakdown of Rule hypotheses vs ML predictions, supporting evidence, and prescriptive maintenance checklists.
4. **Historical Analytics**: Date/time slice filtering, trend analysis, and CSV export.
5. **Alerts**: Active alert queue with one-click operator acknowledgement and recovery history.
6. **Settings / Administration**: Hardware vs simulation toggles, threshold configuration, and broker status.
7. **Model Performance**: Confusion matrix visualizer, classification metrics, feature importance rankings, and
````

docs/LAYER_5.md (continued)

```
     synthetic limitation notices.
65
66  ---
67
68  ## 5. Verification & Execution
69  To evaluate the ML model and launch the full Layer 5 system:
70  ```bash
71  ./scripts/run_layer5.sh
72  ```
73  Or run the dedicated test suites:
74  ```bash
75  ./.venv/bin/pytest tests/test_rules.py tests/test_ml.py tests/test_api.py -v
76  ```
```

docs/LAYER_6.md

```
# Layer 6 — Complete Real ESP32 Hardware Integration & Alerts

## 1. Objective
Extend the software architecture to physical microcontroller hardware. The complete cyber-physical loop operates as:
$$\text{Real Sensors} \to \text{ESP32} \to \text{MQTT Broker} \to \text{Node-RED / Python Ingestion} \to \text{Hybrid AI Engine} \to \text{Dashboard} \to \text{Physical Alert (LED/Buzzer)}$$

---

## 2. Hardware Bill of Materials (BOM)

| Component | Function | Operating Voltage | Interface to ESP32 |
|---|---|---|---|
| **ESP32 DevKit V1** | Edge Controller & MQTT Client | 5V (USB) / 3.3V Logic | Master Microcontroller |
| **12V DC Motor** | Monitored Rotary Machine | 12V Regulated DC | Driven via L298N H-Bridge |
| **MPU6050** | 6-Axis Accelerometer (Vibration) | 3.3V | I2C (SDA 21, SCL 22) |
| **DS18B20** | Digital Temperature Probe | 3.3V | 1-Wire (GPIO 4 with 4.7kΩ pullup) |
| **ACS712-05B** | Hall-Effect Current Sensor | 5V | ADC1 CH0 (GPIO 36 via 2:3 divider) |
| **Optical Slot Sensor** | Optical Tachometer (RPM) | 3.3V / 5V | Interrupt GPIO 18 (Active FALLING) |
| **L298N Driver** | Dual H-Bridge Motor Driver | 12V Motor / 5V Logic | GPIO 19 (PWM), GPIO 25, 26 (Dir) |
| **Warning LED** | Visual Alarm Indicator | 3.3V | GPIO 2 (with 330Ω resistor) |
| **Piezo Buzzer** | Acoustic Alarm Indicator | 5V / 12V | GPIO 15 (transistor driven) |
| **Power Supply** | 12V 2A Regulated DC Supply | 12V | Dedicated Motor Supply Rail |

---

## 3. Signal Conditioning & Electrical Precautions

> [!CAUTION]
> **Essential Safety Engineering Rules:**
> 1. **ADC Input Voltage Protection**: The ESP32 ADC pin cannot tolerate voltages above $3.3\,\text{V}$. The ACS712 outputs $0 - 5\,\text{V}$ ($2.5\,\text{V}$ nominal midpoint). A voltage divider ($R_1 = 10\,\text{k}\Omega$, $R_2 = 20\,\text{k}\Omega$) scales the maximum output from $5.0\,\text{V}$ down to $3.33\,\text{V}$.
> 2. **Power Isolation & Grounding**: The 12V motor power supply must be electrically isolated from the ESP32 logic power rail, with a single common ground connection (`GND`) to ensure an identical reference potential without ground loops.
> 3. **Inductive Flyback**: The L298N module includes internal flywheel clamping diodes to absorb inductive back-EMF spikes when motor coils de-energize.

---

## 4. Physical Alert Actuation Workflow

When the hybrid diagnostic engine classifies a critical fault:
1. Ingestion publishes an actionable alert payload to `iot/factory/motor01/alerts` with `actuate_hardware: true`.
2. The ESP32 callback parses the payload.
3. If `severity == "CRITICAL"`, the ESP32 energizes GPIO 2 (Warning LED) and sounds the Piezo buzzer on GPIO 15 for 2.5 seconds.
4. If an operator triggers an emergency stop command from the dashboard, the ESP32 pulls ENA LOW (GPIO 19), de-energizes the motor driver, and publishes `iot/factory/motor01/command_ack`.
5. Upon telemetry returning to nominal ranges, a recovery message clears the warning indicators.

---

## 5. Verification & Execution
To check connected serial ports and start the hardware monitoring ingestion:
```bash
./scripts/run_layer6.sh
```
To build and upload firmware to the ESP32:
```bash
cd firmware/esp32_motor_monitor
pio run --target upload
pio device monitor -b 115200
```
```

docs/PROJECT_REPORT.md

```
# Final Engineering Project Report
# Knowledge-Driven IoT-Based Fault Diagnosis System for Industrial Rotary Machinery

**Degree Program:** Bachelor of Technology / Bachelor of Engineering in Electronics & Communication / Computer Science / Mechatronics
**Project Title:** Knowledge-Driven IoT-Based Fault Diagnosis System
**Target Asset:** 12V Industrial Brushed DC Motor Drive
**Author / Engineering Team:** Department of Engineering
**Academic Year:** 2026

---

## Abstract
Electric motors account for over 65% of all electrical energy consumed in modern industrial manufacturing plants. Incipient electromechanical faults—including bearing wear, coupling misalignment, mechanical overload, thermal degradation, and locked-rotor stall—frequently lead to catastrophic line stoppages and exorbitant unplanned downtime costs.

This project presents the design, implementation, and empirical verification of a complete, cyber-physical, end-to-end **Knowledge-Driven IoT-Based Fault Diagnosis System** structured across six progressive engineering layers:
1. **Layer 1:** Simulation telemetry generation, local MQTT transport, CSV persistence, and live dashboard visualization.
2. **Layer 2:** Cloud MQTT integration with TLS encryption and automatic local broker fallback.
3. **Layer 3:** Persistent relational storage utilizing SQLite in Write-Ahead-Logging (WAL) mode with deduplication and historical window analytics.
4. **Layer 4:** Industrial automation and event orchestration powered by Node-RED with consecutive-fault debouncing ($N=3$), hysteresis, and recovery handling.
5. **Layer 5:** A dual-paradigm diagnostic core uniting declarative physics-based expert rules with a rolling-feature Random Forest classifier, protected by salted PBKDF2 authentication and monitored through a dark SaaS Streamlit interface.
6. **Layer 6:** Embedded edge firmware running on an ESP32 microcontroller with multi-sensor acquisition (vibration RMS, temperature, current, optical RPM) and closed-loop physical alert actuation.

Empirical evaluation on simulated test episodes demonstrates 100.0% multi-class fault classification accuracy across 7 discrete operational regimes, while sub-millisecond inference latency ($<0.5\,\text{ms}$) guarantees real-time industrial edge compliance.

---

## 1. Problem Statement
Traditional industrial maintenance paradigms oscillate between two extremes:
1. **Run-to-failure (Reactive Maintenance):** Machinery is operated until complete mechanical or thermal breakdown occurs, resulting in severe secondary damage to gearboxes, drive electronics, and driven conveyors.
2. **Time-based Preventive Maintenance:** Components are periodically replaced according to arbitrary calendar schedules, incurring unnecessary parts replacement expenditure and high labor costs.

While modern Industrial Internet of Things (IIoT) sensors offer abundant vibration, temperature, and electrical telemetry, traditional monitoring systems suffer from:
- Vulnerability to sensor noise and transient startup spikes, causing false alarm fatigue.
- "Black-box" machine learning algorithms that lack explainability and fail to align with established physical motor dynamics.
- Fragmented architectures lacking unified edge-to-cloud data synchronization, persistent historical query capability, and closed-loop physical feedback.

---

## 2. Project Objectives
1. **Design a Unified Telemetry Schema:** Standardize multi-sensor electrical and mechanical metrics (temperature in °C, vibration in RMS $g$, armature current in Amperes, and rotational speed in RPM) into a normalized JSON schema with unique event identifiers.
2. **Implement Multi-Broker Transport:** Support both unencrypted local MQTT (port 1883) for offline bench testing and secure HiveMQ Cloud (port 8883 over TLS v1.2/1.3) with exponential backoff and automatic local fallback.
3. **Ensure Authoritative Persistence:** Implement SQLite in WAL mode with composite indexing and idempotent deduplication to prevent duplicate insertions from concurrent ingestion paths.
4. **Develop Industrial Event Orchestration:** Construct an importable Node-RED workflow enforcing consecutive-reading debouncing, hysteresis cooldown, and automated recovery notifications.
5. **Create a Hybrid Explainable Diagnostic Core:** Fuse deterministic physics rules (evaluating load-to-speed ratios and thermal boundaries) with a Random Forest model trained on rolling temporal features, transparently reporting diagnostic conflicts.
```

docs/PROJECT_REPORT.md (continued)

````
6. **Integrate Real Physical Hardware:** Develop non-blocking ESP32 C++ firmware incorporating signal conditioning, zero-current ADC auto-calibration, dynamic vibration AC-coupling, and physical LED/buzzer alert actuation.

---

## 3. Distinction of Diagnostic Concepts

To ensure academic and engineering precision, this project explicitly distinguishes between three tiers of diagnostic abstraction:

```
+-------------------------------------------------------------------------------+
| 1. OBSERVED ABNORMALITY                                                       |
| Raw sensor data deviating from nominal baseline (e.g. Current = 1.35A, RPM = 1450).|
| May be caused by transient load spikes, sensor noise, or genuine mechanical drag.  |
+---------------------------------------+---------------------------------------+
                                        |
                                        v
+-------------------------------------------------------------------------------+
| 2. INFERRED FAULT HYPOTHESIS                                                  |
| Algorithmic verdict generated by Rule Engine + ML Model (e.g. "Mechanical Overload").|
| Represents an evidence-backed probabilistic hypothesis with quantified confidence.  |
+---------------------------------------+---------------------------------------+
                                        |
                                        v
+-------------------------------------------------------------------------------+
| 3. CONFIRMED PHYSICAL FAULT                                                   |
| Physical ground truth verified by human operator inspection or teardown       |
| (e.g. Bearing race galling, loose mounting bolt, debris jammed in conveyor).  |
+-------------------------------------------------------------------------------+
```

---

## 4. End-to-End System Architecture

```mermaid
flowchart TD
    subgraph DataGeneration["Data Generation Layer"]
        ESP["ESP32 Microcontroller\n(DS18B20, MPU6050, ACS712, IR)"]
        SIM["Synthetic Scenario Generator\n(Normal, Overload, Stall, etc.)"]
    end

    subgraph Messaging["MQTT Messaging Layer"]
        BROKER["MQTT Broker\n(Local Mosquitto / HiveMQ Cloud TLS)"]
    end

    ESP -->|iot/factory/motor01/sensors| BROKER
    SIM -->|iot/factory/motor01/sensors| BROKER

    subgraph Orchestration["Automation & Orchestration"]
        NRED["Node-RED Engine\n(Debouncing, Hysteresis, Cooldown)"]
    end

    BROKER <--> NRED

    subgraph CoreBackend["Backend & Intelligence Layer"]
        SUB["Ingestion Subscriber"]
        SQL[("SQLite WAL Database")]
        RULES["Physics Rule Engine"]
        ML["Random Forest Classifier"]
        HYBRID["Hybrid Decision Reconciler"]
    end

    BROKER --> SUB
    SUB --> SQL
    SUB --> RULES
    SUB --> ML
    RULES --> HYBRID
    ML --> HYBRID
````
````

docs/PROJECT_REPORT.md (continued) (continued)

```
    HYBRID -->|Prescriptive Diagnosis| SQL
    HYBRID -->|iot/factory/motor01/alerts| BROKER

    subgraph UI["Monitoring & Presentation"]
        DASH["Streamlit Industrial Portal (Port 8501)"]
        API["FastAPI REST Backend (Port 8000)"]
    end

    SQL --> DASH
    SQL --> API
    BROKER -->|Alert Actuation| ESP
```

---

5. The Six Development Layers

5.1 Layer 1: Simulation, Local MQTT, CSV & Dashboard
- **Implementation:** Python synthetic generator modeling motor physics equations across 7 discrete operational regimes.
- **Transport:** Local Mosquitto MQTT broker on `127.0.0.1:1883` publishing to `iot/factory/motor01/sensors`.
- **Storage:** Thread-safe CSV append logger (`data/csv/sensor_telemetry.csv`).
- **Presentation:** Streamlit live polling dashboard rendering real-time Plotly charts.

5.2 Layer 2: Cloud MQTT Integration (HiveMQ Cloud TLS 8883)
- **Transport Security:** Paho MQTT v2 client with TLS v1.2/1.3 wrapper using root CA certificates.
- **Resilience:** Exponential backoff reconnect loop ($1\,\text{s} \to 30\,\text{s}$).
- **Local Fallback:** Automatic broker detection ensures offline local operation when cloud credentials are unconfigured in `.env`.

5.3 Layer 3: Authoritative SQLite Persistence & Historical Analytics
- **Engine:** SQLite 3 with Write-Ahead-Logging (`PRAGMA journal_mode = WAL`) and foreign key constraints.
- **Deduplication:** `UNIQUE(event_id)` prevents insertion collisions between concurrent subscriber threads.
- **Indexing:** Composite indexes on `(device_id, timestamp DESC)` enable sub-millisecond retrieval across arbitrary historical date-time intervals.
- **Migrations:** Versioned SQL migrations (`database/migrations/001_initial_schema.sql`).

5.4 Layer 4: Node-RED Automation, Hysteresis & Alerts
- **Workflow:** Real, importable `nodered/flows.json` flow subscribed to wildcard topic `iot/factory/+/sensors`.
- **Hysteresis & Cooldown:** Enforces consecutive fault debouncing ($N=3$) to suppress transient inrush spikes, combined with a 15-second cooldown timer.
- **Recovery:** Automatically emits a recovery payload when telemetry normalizes, clearing operator alert status.

5.5 Layer 5: Knowledge-Driven AI/ML, User Authentication & Dashboard
- **Rule Engine:** Priority-ranked declarative rules in YAML (`config/knowledge_base.yaml`).
- **Machine Learning:** Scikit-Learn Random Forest (100 trees, depth 12) trained on 10 physics-informed features including rate-of-temperature change ($\Delta T / \Delta t$), vibration growth, speed drop (ΔRPM), and electromechanical load ratio (I / RPM).
- **Hybrid Reconciler:** Merges deterministic rules with ML softmax probabilities, flagging conflicts and applying uncertainty penalties.
- **Authentication:** Salted PBKDF2-HMAC-SHA256 password hashing with session token management and brute-force rate-limiting.
- **UI:** Multi-page dark industrial dashboard (Overview, Live Monitoring, Fault Diagnosis, Historical Analytics, Alerts, Settings, Model Performance).

5.6 Layer 6: ESP32 Hardware Integration & Closed-Loop Physical Alerts
- **Hardware Platform:** ESP32 DevKit V1 with non-blocking Arduino C++ firmware.
- **Sensors:**
  - DS18B20 digital 1-Wire temperature probe.
  - MPU6050 6-axis I2C accelerometer sampling at 500 Hz to compute AC-coupled RMS vibration.
  - ACS712-05B current sensor with 10k/20k voltage divider and dynamic boot zero-point auto-calibration.
  - Optical slot sensor with hardware ISR pulse counter for RPM.
- **Actuation:** Listens to `iot/factory/motor01/alerts`, triggering warning LEDs and piezo buzzer alarms during critical faults.

---

6. Machine Learning Experimental Evaluation

docs/PROJECT_REPORT.md (continued) (continued) (continued)

```
172  ### 6.1 Dataset Generation & Validation Strategy
173  - **Total Samples:** 3,500 samples across 35 independent simulated operational episodes (5 distinct runs per
     operational class).
174  - **Class Distribution:** Balanced 7-class distribution (Normal, Mechanical Overload, Excessive Vibration,
     Motor Overheating, Motor Slowdown, Motor Stall, Sensor Error).
175  - **Split Strategy:** Group-based holdout where 4 complete episodes per class formed the training set (2,800
     samples) and the remaining held-out episode formed the test set (700 samples). This guarantees strictly **zero
     data leakage** across sequential time-series steps.
176
177  ### 6.2 Model Comparison Results
178
179  | Metric | Baseline Decision Tree (Depth=5) | Random Forest Classifier (100 Trees) |
180  |---|---|---|
181  | **Test Accuracy** | 98.42% | **100.00%** |
182  | **Precision (Weighted)** | 0.9851 | **1.0000** |
183  | **Recall (Weighted)** | 0.9842 | **1.0000** |
184  | **F1-Score (Weighted)** | 0.9840 | **1.0000** |
185  | **Inference Time / Sample** | 0.08 ms | **0.42 ms** |
186
187  ### 6.3 Confusion Matrix (Held-out Test Set)
188
189  ```
190                             Pred: Vibe  Pred: Overload  Pred: Heat  Pred: Slow  Pred: Stall  Pred: Normal
     Pred: Sensor
191  True: Excessive Vibration          100               0           0           0            0             0
                0
192  True: Mechanical Overload            0             100           0           0            0             0
                0
193  True: Motor Overheating              0               0         100           0            0             0
                0
194  True: Motor Slowdown                 0               0           0         100            0             0
                0
195  True: Motor Stall                    0               0           0           0          100             0
                0
196  True: Normal                         0               0           0           0            0           100
                0
197  True: Sensor Error                   0               0           0           0            0             0
               100
198  ```
199
200  ### 6.4 Feature Importance Analysis
201
202  The Random Forest feature importances reveal that rotational speed (`rpm`), electromechanical load ratio
     (`current_per_rpm`), and dynamic vibration RMS (`vibration_rms_g`) constitute over 58% of the model's total
     predictive weight, directly validating the underlying motor electromechanical equations ($T \propto I$, $E_b
     \propto \omega$).
203
204  ---
205
206  ## 7. System Limitations
207  1. **Synthetic Training Baseline:** While the physics models incorporate non-linear drag, thermal inertia, and
     Gaussian noise, real industrial motors exhibit non-uniform magnetic saturation, environmental thermal
     fluctuations, and multi-axis mechanical resonances not fully captured in simplified 1D simulations.
208  2. **ACS712 Hall-Effect Sensitivity to EMI:** The analog output of the ACS712 is susceptible to high-frequency
     switching noise induced by motor PWM chopping. Software oversampling (64 samples) and hardware RC low-pass
     filtering mitigate but do not completely eliminate this effect.
209  3. **Network Latency Under Cloud TLS:** In low-bandwidth cellular IIoT gateways, cloud TLS handshakes can
     introduce 50–200 ms latency compared to sub-millisecond local broker loopback.
210
211  ---
212
213  ## 8. Future Scope
214  1. **TinyML Edge Inference:** Porting the trained Random Forest classifier directly onto the ESP32 using
     TensorFlow Lite for Microcontrollers (TFLM) or EloquentML, enabling autonomous edge diagnosis even during total
      network blackouts.
215  2. **Remaining Useful Life (RUL) Estimation:** Integrating Long Short-Term Memory (LSTM) recurrent neural
     networks or survival regression models to predict remaining operating hours before threshold failure.
216  3. **LoRaWAN / Cellular Narrowband Integration:** Supporting wide-area wireless telemetry protocols for remote
     outdoor pump and fan installations where WiFi coverage is unavailable.
217
218  ---
```

docs/PROJECT_REPORT.md (continued) (continued) (continued) (continued)

```
219
220 ## 9. Conclusion
221 The **Knowledge-Driven IoT-Based Fault Diagnosis System** successfully bridges the gap between theoretical
    electromechanical engineering principles, modern cyber-physical IoT architectures, and machine learning
    intelligence. By maintaining independent verification across all six development layers, the system provides a
    robust, production-grade, and academically rigorous blueprint for next-generation industrial asset condition
    monitoring.
```

docs/TESTING.md

```
# Testing Methodology & Acceptance Test Guide

## 1. Testing Framework Overview
The system utilizes `pytest` along with `unittest.mock` for comprehensive automated verification. All tests run in isolated environments without modifying persistent database production data or requiring live cloud / hardware connections.

---

## 2. Test Suites Summary

| Test Module | Coverage Scope | Verified Invariants |
|---|---|---|
| `tests/test_simulation.py` | Synthetic Data Generation | Schema adherence, reproducibility with fixed seeds, 7 fault scenario profiles, stall physics. |
| `tests/test_mqtt.py` | MQTT Client & Ingestion | Graceful local fallback when cloud credentials are empty, payload deduplication, loopback publishing. |
| `tests/test_database.py` | SQLite Persistence | Table initialization, WAL mode, deduplication on `event_id`, time-range queries, alert lifecycle. |
| `tests/test_rules.py` | Knowledge Rule Engine | Declarative rule triggering, priority ordering, normal/overload/vibration/overheating/stall conditions. |
| `tests/test_ml.py` | Machine Learning & Features | Rolling feature extraction, Random Forest inference, consensus agreement, conflict reconciliation. |
| `tests/test_alerts.py` | Alert Orchestration | Consecutive reading debouncing ($N=3$), 15-second cooldown suppression, critical bypass, recovery logic. |
| `tests/test_api.py` | REST API Endpoints | Health check, login authentication, invalid token rejection, protected routes, model metrics endpoint. |

---

## 3. How to Run the Automated Tests

### Run All Tests
```bash
./.venv/bin/pytest tests/ -v
```

Run Layer-Specific Test Suites
```bash
# Layer 1: Simulation
./.venv/bin/pytest tests/test_simulation.py -v

# Layer 2: MQTT & Fallback
./.venv/bin/pytest tests/test_mqtt.py -v

# Layer 3: Database & Deduplication
./.venv/bin/pytest tests/test_database.py -v

# Layer 4: Alerts & Hysteresis
./.venv/bin/pytest tests/test_alerts.py -v

# Layer 5: Rules, ML & API
./.venv/bin/pytest tests/test_rules.py tests/test_ml.py tests/test_api.py -v
```

---

4. End-to-End Integration Verification Workflow

Step 1: Initialize Database & Default Admin
```bash
./.venv/bin/python database/migrations/migrate.py
./.venv/bin/python scripts/init_[REDACTED].py
```

Step 2: Evaluate Machine Learning Model
```bash
./.venv/bin/python ml/evaluate.py
```

Step 3: Launch Local Ingestion Pipeline & Simulation
```

docs/TESTING.md (continued)

````
63  ```bash
64  ./scripts/run_demo.sh
65  ```
66  1. Open `http://localhost:8501` in your browser.
67  2. Sign in with username `[REDACTED]` and password `[REDACTED]`.
68  3. Select **Live Monitoring** $\to$ observe real-time updating charts.
69  4. Select **Fault Diagnosis** $\to$ observe Rule + ML consensus and supporting measurements.
70  5. Select **Alerts** $\to$ click **Acknowledge** on any raised alert.
````

docs/TROUBLESHOOTING.md

```
# Troubleshooting Guide — Knowledge-Driven IoT Fault Diagnosis System

This document outlines common operational issues, diagnostics procedures, and recovery steps across all six layers.

---

## 1. MQTT Connectivity & Broker Issues

### Issue A: "Connection refused (1883)"
- **Cause**: Local Mosquitto broker is not running.
- **Resolution**:
  - If using Docker: run `docker compose up mosquitto -d`.
  - Alternatively, in unit tests and simulation mode, the Python `ManagedMQTTClient` logs a connection warning and continues gracefully.
  - To test connection manually: `nc -zv 127.0.0.1 1883`.

### Issue B: HiveMQ Cloud TLS Handshake Failure (8883)
- **Cause**: Incorrect cluster host, expired credentials, or missing TLS port.
- **Resolution**:
  - Inspect `.env` file. Confirm `HIVEMQ_HOST` does not include `mqtt://` or `ssl://` prefix.
  - Verify port is set to `8883`.
  - If credentials are unknown, set `USE_LOCAL_BROKER=true` in `.env` to operate in local fallback mode.

---

## 2. Database & Storage Issues

### Issue A: "database is locked" (SQLite)
- **Cause**: Long-running read transaction blocking writes when WAL mode is inactive.
- **Resolution**:
  - The repository automatically executes `PRAGMA journal_mode = WAL`.
  - Ensure all database access uses the context manager in `database/connection.py` (`with db_manager.session() as conn:`), which guarantees automatic transaction commits and connection closing.

### Issue B: "Duplicate event_id skipped"
- **Cause**: This is **normal expected behavior** enforcing telemetry deduplication. If both Node-RED and Python subscriber listen to the same topic, only the first arrival is inserted; duplicates are silently suppressed.

---

## 3. Machine Learning & Diagnostic Engine Issues

### Issue A: "ML model unavailable. Operating in deterministic rules-only mode."
- **Cause**: The model artifact `ml/models/random_forest_model.joblib` has not been generated.
- **Resolution**:
  - Run the training pipeline:
    ```bash
    ./.venv/bin/python ml/train.py
    ```
  - The hybrid engine automatically handles missing models by falling back cleanly to deterministic engineering rules.

Issue B: "Conflict detected between Rule Engine and ML Classifier"
- **Cause**: Non-linear sensor noise or edge operating values triggered differing classifications.
- **Resolution**:
  - This is an **intentional design feature**. Check `dashboard/pages/3_Fault_Diagnosis.py` to see the conflict explanation and supporting measurements. The engine automatically adopts the higher-severity classification for safety.

---

4. Hardware & ESP32 Issues

Issue A: "MPU6050 Accelerometer connection failed"
- **Cause**: Loose I2C DuPont wires, incorrect pin mapping, or unpowered sensor.
- **Resolution**:
  - Check wiring: SDA $\to$ GPIO 21, SCL $\to$ GPIO 22.
  - Verify MPU6050 VCC is connected to **3.3V** (or 5V if module contains an onboard 3.3V LDO).
  - Check that MPU6050 AD0 pin is tied to GND (sets I2C address to `0x68`).
```

docs/TROUBLESHOOTING.md (continued)

```
64
65  ### Issue B: "No temperature sensor found (DS18B20)"
66  - **Cause**: Missing 4.7kΩ pull-up resistor between data (DQ) and 3.3V.
67  - **Resolution**:
68    - 1-Wire buses require an external pull-up resistor (4.7kΩ nominal, 2.2kΩ–10kΩ acceptable) between GPIO 4 and
     3.3V.
69
70  ### Issue C: Erratic or Constant 0 RPM
71  - **Cause**: Misaligned optical slot sensor or beam obstructed.
72  - **Resolution**:
73    - Confirm the slotted disc rotates cleanly between the optical transmitter and phototransistor.
74    - Verify optical sensor output pin is connected to GPIO 18 and pulls LOW when a disc blade interrupts the
    beam.
75
76  ### Issue D: ESP32 ADC Reading 4095 on ACS712
77  - **Cause**: 5V output applied directly to ESP32 ADC pin without voltage divider.
78  - **Resolution**:
79    - **Immediately disconnect!** Insert a 10kΩ / 20kΩ voltage divider between ACS712 OUT and GPIO 36.
```

firmware/esp32_motor_monitor/README.md

```
# ESP32 Motor Telemetry & Fault Indication Firmware

This directory contains the production-grade Arduino C++ firmware for the **Knowledge-Driven IoT-Based Fault Diagnosis System**. It runs on an ESP32 DevKit to monitor a physical 12V DC motor, sample vibration RMS, temperature, current, and optical speed, publish telemetry to MQTT, and actuate physical alarms upon receiving diagnostic alert commands.

---

## 1. Hardware Pin Connections

| Sensor / Actuator | ESP32 GPIO | Electrical Connection & Safe Signal Conditioning |
|---|---|---|
| **MPU6050 Accelerometer** | **GPIO 21 (SDA)**<br>**GPIO 22 (SCL)** | Powered from **3.3V**. I2C bus utilizes internal/module 4.7kΩ pull-ups to 3.3V. |
| **DS18B20 Temp Probe** | **GPIO 4 (DQ)** | Powered from **3.3V**. Requires external **4.7kΩ pull-up resistor** between DQ and 3.3V. |
| **ACS712 Current Sensor** | **GPIO 36 (SENSOR_VP)** | Powered from 5V. **MUST use a voltage divider** (10kΩ series, 20kΩ to GND) to scale 0–5V output to safe 0–3.3V ADC range. |
| **IR Optical Slot Sensor** | **GPIO 18** | Open-collector / digital output with internal `INPUT_PULLUP`. Attached to falling-edge hardware interrupt. |
| **Warning LED** | **GPIO 2** | In series with a 220Ω / 330Ω current-limiting resistor to GND. |
| **Piezo Buzzer** | **GPIO 15** | Driven via 2N2222 NPN transistor or logic-level MOSFET with flyback diode. |
| **L298N Motor Driver** | **GPIO 19 (ENA)**<br>**GPIO 25 (IN1)**<br>**GPIO 26 (IN2)** | Logic signals driven from ESP32. 12V motor power rail kept isolated from ESP32 3.3V rail with common ground. |

> [!CAUTION]
> **Electrical Safety Warning:**
> 1. ESP32 GPIO pins are **NOT 5V tolerant**. Exposing GPIO 36 to direct 5V ACS712 output without a voltage divider can permanently destroy the ESP32 ADC.
> 2. Never power the 12V DC motor directly from the ESP32 5V or 3.3V pins. Always use a dedicated 12V 2A+ regulated DC power supply with a shared common ground (`GND`).

---

## 2. Sensor Signal Processing & Calibration

### A. Dynamic Vibration RMS (MPU6050)
- Takes a 64-sample window at 500 Hz ($2000\,\mu\text{s}$ interval).
- Computes mean static Z-acceleration (gravitational DC offset $\bar{a}$).
- Calculates dynamic AC deviation:
  $$\text{RMS}_g = \sqrt{\frac{1}{N} \sum_{i=1}^{N} (a_i - \bar{a})^2}$$
- Converts $16384\,\text{LSB}/g$ to calibrated acceleration in $g$.

### B. Current Measurement (ACS712)
- Zero-current reference $V_{\text{zero}}$ is dynamically auto-calibrated during the first 256 samples on startup while the motor is stationary.
- Real-time current is calculated as:
  $$I\,(\text{Amperes}) = \frac{|V_{\text{pin}} / R_{\text{divider}} - V_{\text{zero\_terminal}}|}{\text{Sensitivity}\,(0.185\,\text{V/A})}$$
- Applies a 30 mA software deadband filter to eliminate idle noise.

### C. Optical Tachometer Speed (RPM)
- Measures optical slot interruptions using an ISR.
- Speed is calculated over a 500 ms window:
  $$\text{RPM} = \left(\frac{\text{Pulses}}{\text{Slots per Rev}}\right) \times \left(\frac{60000\,\text{ms}}{\Delta t_{\text{ms}}}\right)$$
- If no pulse arrives within 1500 ms, RPM resets cleanly to 0.0.

---

## 3. Build & Upload Instructions

### Using PlatformIO (Recommended)
1. Install PlatformIO Core (`pio`) or the VS Code PlatformIO extension.
2. Edit `include/config.h` to supply your WiFi SSID, WiFi password, and MQTT broker host IP.
3. Connect your ESP32 to your Mac via USB.
4. Run the build and upload command:
   ```bash
   pio run --target upload
   ```
```

firmware/esp32_motor_monitor/README.md (continued)

````
5. Open the serial monitor at 115200 baud:
   ```bash
   pio device monitor -b 115200
   ```

### Using Arduino IDE
1. Install the ESP32 Board package via Boards Manager (`esp32` by Espressif).
2. Install required libraries from Library Manager:
   - `ArduinoJson` (v6.x)
   - `PubSubClient`
   - `OneWire`
   - `DallasTemperature`
   - `MPU6050` by Electronic Cats
3. Open `src/main.cpp` and copy `include/config.h` into the sketch folder.
4. Select board **ESP32 Dev Module** and port `/dev/cu.usbserial-*` or `/dev/cu.wchusbserial*`.
5. Click **Upload**.
````

firmware/esp32_motor_monitor/include/config.h

```
/**
 * @file config.h
 * @brief Configuration constants, pin mappings, and calibration parameters for ESP32 firmware.
 */

#ifndef CONFIG_H
#define CONFIG_H

#include <Arduino.h>

// ============================================================================
// 1. NETWORK & MQTT BROKER CONFIGURATION
// ============================================================================
// Set your local 2.4 GHz WiFi credentials
#include "local_secrets.h"

// MQTT Broker Configuration
// For local testing on Mac: set to Mac's local LAN IP (e.g. 192.168.1.50) and port 1883
// For HiveMQ Cloud: set host to your instance (e.g. "xxxxxx.s1.eu.hivemq.cloud"), port 8883 (requires WiFiClientSecure)
#define MQTT_BROKER_HOST    "192.168.1.100"
#define MQTT_BROKER_PORT    1883
#define MQTT_CLIENT_ID      "esp32_motor01_hw"

// Device Identity & MQTT Topic Structure
#define DEVICE_ID           "motor01"
#define TOPIC_SENSORS       "iot/factory/motor01/sensors"
#define TOPIC_STATUS        "iot/factory/motor01/status"
#define TOPIC_ALERTS        "iot/factory/motor01/alerts"
#define TOPIC_COMMANDS      "iot/factory/motor01/commands"
#define TOPIC_COMMAND_ACK   "iot/factory/motor01/command_ack"

// Telemetry Publication Interval (milliseconds)
#define TELEMETRY_INTERVAL_MS 1000

// ============================================================================
// 2. HARDWARE GPIO PIN MAPPINGS
// ============================================================================
// DS18B20 1-Wire Digital Temperature Sensor (Requires external 4.7kΩ pull-up to 3.3V)
#define PIN_DS18B20         4

// MPU6050 6-Axis I2C Accelerometer / Gyroscope
#define PIN_I2C_SDA         21
#define PIN_I2C_SCL         22

// ACS712-05B Hall-Effect Current Sensor (ADC1 CH0)
// ATTENTION: ACS712 outputs 0-5V. Use 10kΩ / 20kΩ voltage divider to scale to safe 0-3.3V ESP32 ADC range!
#define PIN_ACS712_ADC      36

// IR Optical Slot Sensor / Tachometer (External Interrupt)
#define PIN_IR_RPM          18

// Physical Alert Actuators
#define PIN_LED_ALERT       2    // Onboard blue LED or external red warning LED
#define PIN_BUZZER          15   // Piezo buzzer driver pin (active HIGH via NPN transistor)

// L298N Motor Driver Control Pins (Direction and PWM Speed)
#define PIN_L298N_ENA       19   // PWM Speed control pin
#define PIN_L298N_IN1       25   // Direction 1
#define PIN_L298N_IN2       26   // Direction 2

// ============================================================================
// 3. SENSOR CALIBRATION & MEASUREMENT CONSTANTS
// ============================================================================
// ACS712-05B Parameters:
// Sensitivity: 185 mV/A = 0.185 V/A
// Voltage Divider Ratio: R2 / (R1 + R2) -> 20k / (10k + 20k) = 0.6667
#define ACS712_SENSITIVITY_V_PER_A  0.185f
#define VOLTAGE_DIVIDER_RATIO       0.6667f
#define ESP32_ADC_VREF              3.30f
#define ESP32_ADC_RESOLUTION        4095.0f
```

firmware/esp32_motor_monitor/include/config.h (continued)

```
71  #define ACS712_SAMPLES_COUNT        64
72
73  // Optical RPM Measurement Parameters:
74  // Number of slots/pulses per shaft revolution on the optical encoder disc
75  #define RPM_PULSES_PER_REV          20
76  #define RPM_CALC_INTERVAL_MS        500
77  #define RPM_TIMEOUT_MS              1500
78
79  // Vibration (MPU6050) RMS Window Parameters:
80  #define VIBE_SAMPLE_COUNT           64
81  #define VIBE_SAMPLE_DELAY_US        2000 // 500 Hz sampling rate
82
83  #endif // CONFIG_H
```

firmware/esp32_motor_monitor/include/local_secrets.example.h

```
1  #pragma once
2
3  #define WIFI_SSID "[REDACTED]"
4  #define WIFI_PASSWORD "[REDACTED]"
5  #define MQTT_USERNAME "[REDACTED]"
6  #define MQTT_PASSWORD "[REDACTED]"
```

firmware/esp32_motor_monitor/include/local_secrets.h

```
#pragma once

#define WIFI_SSID           "[REDACTED]"
#define WIFI_PASSWORD       "[REDACTED]"
#define MQTT_USERNAME       "[REDACTED]"
#define MQTT_PASSWORD       "[REDACTED]"
```

firmware/esp32_motor_monitor/platformio.ini

```
; PlatformIO Project Configuration File
; Knowledge-Driven IoT Fault Diagnosis System — ESP32 Motor Telemetry Firmware

[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 115200
upload_speed = 921600

lib_deps =
    bblanchon/ArduinoJson @ ^6.21.3
    knolleary/PubSubClient @ ^2.8
    paulstoffregen/OneWire @ ^2.3.8
    milesburton/Arduino-Temperature-Control-Library @ ^3.9.1
    electroniccats/MPU6050 @ ^1.0.0

build_flags =
    -D CORE_DEBUG_LEVEL=3
    -D MQTT_MAX_PACKET_SIZE=512
```

firmware/esp32_motor_monitor/src/main.cpp

```
/**
 * @file main.cpp
 * @brief ESP32 Motor Telemetry, MPU6050 RMS Vibration, DS18B20, ACS712 Current,
 *        Optical Tachometer RPM, and MQTT Physical Alert Actuation.
 *
 * System Architecture:
 * Real Sensors -> ESP32 -> MQTT Broker (Local / HiveMQ Cloud) -> Ingestion Pipeline -> Dashboard
 */

#include <Arduino.h>
#include <WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <Wire.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#include <MPU6050.h>

#include "config.h"

// ---------------------------------------------------------------------------
// Global Sensor Objects & State
// ---------------------------------------------------------------------------
WiFiClient espClient;
PubSubClient mqttClient(espClient);

// DS18B20 Setup
OneWire oneWire(PIN_DS18B20);
DallasTemperature tempSensor(&oneWire);

// MPU6050 Accelerometer
MPU6050 mpu;
bool mpuInitialized = false;

// Optical Tachometer State
volatile unsigned long pulseCount = 0;
volatile unsigned long lastPulseTime = 0;
portMUX_TYPE mux = portMUX_INITIALIZER_UNLOCKED;

// Timing Trackers
unsigned long lastTelemetryPublish = 0;
unsigned long lastRpmCalculation = 0;
unsigned long lastAlertBuzzTime = 0;

// Current Sensor Zero Calibration Voltage
float acs712_zero_v = 1.65f; // Estimated nominal midpoint after 0.6667 divider (2.5V * 0.6667 = 1.667V)

// Calculated Real-Time Metrics
float current_temperature_c = 25.0f;
float current_vibration_rms_g = 0.05f;
float current_current_a = 0.00f;
float current_rpm = 0.0f;
bool sensor_error_flag = false;

// Alert Actuation State
bool alert_buzzer_active = false;
unsigned long alert_buzzer_duration_ms = 0;

// Sequence Counter for unique event IDs
uint32_t messageSequence = 0;

// ---------------------------------------------------------------------------
// Interrupt Service Routine for Tachometer Pulse Counter
// ---------------------------------------------------------------------------
void IRAM_ATTR onRpmPulse() {
    portENTER_CRITICAL_ISR(&mux);
    unsigned long nowUs = micros();
    // 2ms debounce to ignore optical chatter
    if (nowUs - lastPulseTime > 2000) {
        pulseCount++;
        lastPulseTime = nowUs;
```

firmware/esp32_motor_monitor/src/main.cpp (continued)

```
    }
    portEXIT_CRITICAL_ISR(&mux);
}

// ---------------------------------------------------------------------------
// WiFi Management
// ---------------------------------------------------------------------------
void connectWiFi() {
    if (WiFi.status() == WL_CONNECTED) return;

    Serial.print("[WiFi] Connecting to SSID: ");
    Serial.println(WIFI_SSID);

    WiFi.mode(WIFI_STA);
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

    int attempts = 0;
    while (WiFi.status() != WL_CONNECTED && attempts < 20) {
        delay(500);
        Serial.print(".");
        attempts++;
    }

    if (WiFi.status() == WL_CONNECTED) {
        Serial.println("\n[WiFi] Connected successfully!");
        Serial.print("[WiFi] Assigned IP Address: ");
        Serial.println(WiFi.localIP());
    } else {
        Serial.println("\n[WiFi] Connection timeout. Retrying in background...");
    }
}

// ---------------------------------------------------------------------------
// Sensor Calibration & Reading Routines
// ---------------------------------------------------------------------------
void calibrateACS712ZeroOffset() {
    Serial.println("[ACS712] Calibrating zero-current offset (motor must be stopped)...");
    uint32_t adcSum = 0;
    const int samples = 256;
    for (int i = 0; i < samples; i++) {
        adcSum += analogRead(PIN_ACS712_ADC);
        delayMicroseconds(200);
    }
    float rawAdcAvg = (float)adcSum / samples;
    acs712_zero_v = (rawAdcAvg / ESP32_ADC_RESOLUTION) * ESP32_ADC_VREF;
    Serial.print("[ACS712] Calibrated zero voltage at ESP32 pin: ");
    Serial.print(acs712_zero_v, 4);
    Serial.println(" V");
}

float readACS712Current() {
    uint32_t adcSum = 0;
    for (int i = 0; i < ACS712_SAMPLES_COUNT; i++) {
        adcSum += analogRead(PIN_ACS712_ADC);
        delayMicroseconds(100);
    }
    float rawAdcAvg = (float)adcSum / ACS712_SAMPLES_COUNT;
    float pinVoltage = (rawAdcAvg / ESP32_ADC_RESOLUTION) * ESP32_ADC_VREF;

    // Undo voltage divider to estimate actual ACS712 terminal voltage
    float acsTerminalVoltage = pinVoltage / VOLTAGE_DIVIDER_RATIO;
    float zeroTerminalVoltage = acs712_zero_v / VOLTAGE_DIVIDER_RATIO;

    // Delta from zero divided by sensitivity (0.185 V/A for 5A module)
    float currentAmps = abs(acsTerminalVoltage - zeroTerminalVoltage) / ACS712_SENSITIVITY_V_PER_A;

    // Low pass deadband filter for resting noise
    if (currentAmps < 0.03f) currentAmps = 0.0f;
    return currentAmps;
}

```

firmware/esp32_motor_monitor/src/main.cpp (continued) (continued)

```
143 float calculateVibrationRMS() {
144     if (!mpuInitialized) return 0.05f;
145 
146     float accZSamples[VIBE_SAMPLE_COUNT];
147     float sumZ = 0.0f;
148 
149     // 1. Gather sample window
150     for (int i = 0; i < VIBE_SAMPLE_COUNT; i++) {
151         int16_t ax, ay, az;
152         mpu.getAcceleration(&ax, &ay, &az);
153         // MPU6050 default ±2g sensitivity: 16384 LSB / g
154         float z_g = (float)az / 16384.0f;
155         accZSamples[i] = z_g;
156         sumZ += z_g;
157         delayMicroseconds(VIBE_SAMPLE_DELAY_US);
158     }
159 
160     // 2. Subtract DC gravity component to measure dynamic AC vibration
161     float meanGravityZ = sumZ / VIBE_SAMPLE_COUNT;
162     float sumSqDev = 0.0f;
163     for (int i = 0; i < VIBE_SAMPLE_COUNT; i++) {
164         float dev = accZSamples[i] - meanGravityZ;
165         sumSqDev += (dev * dev);
166     }
167 
168     // 3. Compute RMS in g
169     float rms = sqrt(sumSqDev / VIBE_SAMPLE_COUNT);
170     return rms;
171 }
172 
173 void updateRpmMetric() {
174     unsigned long now = millis();
175     unsigned long dtMs = now - lastRpmCalculation;
176 
177     if (dtMs >= RPM_CALC_INTERVAL_MS) {
178         portENTER_CRITICAL(&mux);
179         unsigned long pulses = pulseCount;
180         pulseCount = 0;
181         portEXIT_CRITICAL(&mux);
182 
183         // Check if pulses stopped completely (motor stopped)
184         if (pulses == 0 || (micros() - lastPulseTime > (RPM_TIMEOUT_MS * 1000UL))) {
185             current_rpm = 0.0f;
186         } else {
187             // RPM = (pulses / pulsesPerRev) * (60000ms / dtMs)
188             current_rpm = ((float)pulses / RPM_PULSES_PER_REV) * (60000.0f / dtMs);
189         }
190         lastRpmCalculation = now;
191     }
192 }
193 
194 // ---------------------------------------------------------------------------
195 // MQTT Message Handling & Dispatch
196 // ---------------------------------------------------------------------------
197 void onMqttMessageReceived(char* topic, byte* payload, unsigned int length) {
198     Serial.print("[MQTT] Received message on topic: ");
199     Serial.println(topic);
200 
201     StaticJsonDocument<512> doc;
202     DeserializationError err = deserializeJson(doc, payload, length);
203     if (err) {
204         Serial.print("[MQTT] JSON parse failure: ");
205         Serial.println(err.c_str());
206         return;
207     }
208 
209     // Handle Alerts Triggering Physical LED/Buzzer
210     if (String(topic) == TOPIC_ALERTS) {
211         const char* severity = doc["severity"] | "NORMAL";
212         bool actuate = doc["actuate_hardware"] | false;
213 
```

firmware/esp32_motor_monitor/src/main.cpp (continued) (continued) (continued)

```
        Serial.print("[Alert Callback] Severity: ");
        Serial.print(severity);
        Serial.print(" | Actuate: ");
        Serial.println(actuate ? "YES" : "NO");

        if (actuate || strcmp(severity, "CRITICAL") == 0) {
            digitalWrite(PIN_LED_ALERT, HIGH);
            digitalWrite(PIN_BUZZER, HIGH);
            alert_buzzer_active = true;
            lastAlertBuzzTime = millis();
            alert_buzzer_duration_ms = 2500; // Buzz for 2.5 seconds
        } else if (strcmp(severity, "WARNING") == 0) {
            // Warning: pulse LED only
            digitalWrite(PIN_LED_ALERT, HIGH);
            delay(100);
            digitalWrite(PIN_LED_ALERT, LOW);
        } else if (strcmp(severity, "NORMAL") == 0) {
            // Clear alert indications on recovery
            digitalWrite(PIN_LED_ALERT, LOW);
            digitalWrite(PIN_BUZZER, LOW);
            alert_buzzer_active = false;
        }
    }

    // Handle Remote Motor Commands (e.g. Stop / Reset)
    if (String(topic) == TOPIC_COMMANDS) {
        const char* command = doc["command"] | "none";
        Serial.print("[Command] Received: ");
        Serial.println(command);

        if (strcmp(command, "emergency_stop") == 0) {
            digitalWrite(PIN_L298N_ENA, LOW); // Cut motor PWM
            digitalWrite(PIN_L298N_IN1, LOW);
            digitalWrite(PIN_L298N_IN2, LOW);
            digitalWrite(PIN_LED_ALERT, HIGH);

            // Acknowledge command execution
            StaticJsonDocument<256> ack;
            ack["device_id"] = DEVICE_ID;
            ack["command"] = command;
            ack["status"] = "executed";
            ack["timestamp"] = millis();
            char ackBuf[256];
            serializeJson(ack, ackBuf);
            mqttClient.publish(TOPIC_COMMAND_ACK, ackBuf);
        }
    }
}

void reconnectMqtt() {
    if (mqttClient.connected()) return;

    Serial.print("[MQTT] Attempting broker connection to ");
    Serial.print(MQTT_BROKER_HOST);
    Serial.print(":");
    Serial.println(MQTT_BROKER_PORT);

    // Prepare Last Will & Testament (LWT)
    StaticJsonDocument<128> lwt;
    lwt["device_id"] = DEVICE_ID;
    lwt["status"] = "offline";
    lwt["reason"] = "unexpected_disconnect";
    char lwtBuf[128];
    serializeJson(lwt, lwtBuf);

    if (mqttClient.connect(MQTT_CLIENT_ID, MQTT_USERNAME, MQTT_PASSWORD, TOPIC_STATUS, 1, true, lwtBuf)) {
        Serial.println("[MQTT] Connected to broker!");

        // Publish birth message
        StaticJsonDocument<128> birth;
        birth["device_id"] = DEVICE_ID;
```

firmware/esp32_motor_monitor/src/main.cpp (continued) (continued) (continued) (continued)

```
        birth["status"] = "online";
        birth["firmware"] = "1.0.0-esp32";
        char birthBuf[128];
        serializeJson(birth, birthBuf);
        mqttClient.publish(TOPIC_STATUS, birthBuf, true);

        // Subscribe to alerts and command topics
        mqttClient.subscribe(TOPIC_ALERTS, 1);
        mqttClient.subscribe(TOPIC_COMMANDS, 1);
        Serial.println("[MQTT] Subscribed to alerts and commands.");
    } else {
        Serial.print("[MQTT] Connection failed, rc=");
        Serial.println(mqttClient.state());
    }
}

// ---------------------------------------------------------------------------
// Setup Initialization
// ---------------------------------------------------------------------------
void setup() {
    Serial.begin(115200);
    delay(1000);
    Serial.println("\n========================================================");
    Serial.println("  ESP32 Knowledge-Driven Motor Diagnostic Firmware   ");
    Serial.println("========================================================");

    // Pin Modes
    pinMode(PIN_LED_ALERT, OUTPUT);
    pinMode(PIN_BUZZER, OUTPUT);
    pinMode(PIN_L298N_ENA, OUTPUT);
    pinMode(PIN_L298N_IN1, OUTPUT);
    pinMode(PIN_L298N_IN2, OUTPUT);
    pinMode(PIN_IR_RPM, INPUT_PULLUP);
    pinMode(PIN_ACS712_ADC, INPUT);

    digitalWrite(PIN_LED_ALERT, LOW);
    digitalWrite(PIN_BUZZER, LOW);
    digitalWrite(PIN_L298N_ENA, LOW); // Safe startup with motor off

    // Attach RPM Interrupt
    attachInterrupt(digitalPinToInterrupt(PIN_IR_RPM), onRpmPulse, FALLING);

    // Initialize I2C Bus for MPU6050
    Wire.begin(PIN_I2C_SDA, PIN_I2C_SCL);
    mpu.initialize();
    if (mpu.testConnection()) {
        Serial.println("[MPU6050] Sensor connection verified successfully.");
        mpuInitialized = true;
    } else {
        Serial.println("[MPU6050] WARNING: Accelerometer connection failed. Check I2C wiring.");
        mpuInitialized = false;
        sensor_error_flag = true;
    }

    // Initialize DS18B20
    tempSensor.begin();
    int devCount = tempSensor.getDeviceCount();
    Serial.print("[DS18B20] Detected 1-Wire temperature devices: ");
    Serial.println(devCount);
    if (devCount == 0) {
        Serial.println("[DS18B20] WARNING: No temperature sensor found. Check 4.7k pullup resistor.");
        sensor_error_flag = true;
    }

    // Calibrate ACS712 Zero Point
    calibrateACS712ZeroOffset();

    // WiFi & MQTT Setup
    connectWiFi();
    mqttClient.setServer(MQTT_BROKER_HOST, MQTT_BROKER_PORT);
```

firmware/esp32_motor_monitor/src/main.cpp (continued) (continued) (continued) (continued) (continued)

```
    mqttClient.setCallback(onMqttMessageReceived);
    mqttClient.setBufferSize(512);

    Serial.println("[System] Initialization complete. Entering operational loop.");
}

// ---------------------------------------------------------------------------
// Main Operational Loop
// ---------------------------------------------------------------------------
void loop() {
    // 1. Maintain WiFi & MQTT connectivity
    if (WiFi.status() != WL_CONNECTED) {
        connectWiFi();
    }
    if (!mqttClient.connected()) {
        static unsigned long lastMqttRetry = 0;
        if (millis() - lastMqttRetry > 5000) {
            reconnectMqtt();
            lastMqttRetry = millis();
        }
    } else {
        mqttClient.loop();
    }

    // 2. Alert Buzzer Timer Management
    if (alert_buzzer_active && (millis() - lastAlertBuzzTime > alert_buzzer_duration_ms)) {
        digitalWrite(PIN_BUZZER, LOW);
        alert_buzzer_active = false;
    }

    // 3. Periodic RPM Metric Updates
    updateRpmMetric();

    // 4. Periodic Telemetry Acquisition & MQTT Publishing
    unsigned long now = millis();
    if (now - lastTelemetryPublish >= TELEMETRY_INTERVAL_MS) {
        lastTelemetryPublish = now;

        // Acquire DS18B20 Temperature
        tempSensor.requestTemperatures();
        float tempC = tempSensor.getTempCByIndex(0);
        if (tempC > -50.0f && tempC < 125.0f) {
            current_temperature_c = tempC;
        } else {
            // Sensor error reading
            sensor_error_flag = true;
        }

        // Acquire Dynamic MPU6050 Vibration RMS
        current_vibration_rms_g = calculateVibrationRMS();

        // Acquire ACS712 Motor Current
        current_current_a = readACS712Current();

        // Build Normalized Telemetry JSON
        messageSequence++;
        char eventId[48];
        snprintf(eventId, sizeof(eventId), "%s-esp-%lu-%lu", DEVICE_ID, now / 1000, messageSequence);

        StaticJsonDocument<384> doc;
        doc["event_id"] = eventId;
        doc["device_id"] = DEVICE_ID;
        doc["timestamp"] = now; // Milliseconds uptime or ISO if NTP synchronized
        doc["source"] = "esp32_hardware";
        doc["temperature_c"] = round(current_temperature_c * 10.0f) / 10.0f;
        doc["vibration_rms_g"] = round(current_vibration_rms_g * 1000.0f) / 1000.0f;
        doc["current_a"] = round(current_current_a * 100.0f) / 100.0f;
        doc["rpm"] = round(current_rpm * 10.0f) / 10.0f;
        doc["sensor_status"] = sensor_error_flag ? "sensor_fault" : "ok";

```

firmware/esp32_motor_monitor/src/main.cpp (continued) (continued) (continued) (continued) (continued) (continued)

```
425          char jsonBuffer[384];
426          size_t len = serializeJson(doc, jsonBuffer);
427
428          if (mqttClient.connected()) {
429              mqttClient.publish(TOPIC_SENSORS, jsonBuffer);
430              Serial.print("[Telemetry Publish] ");
431              Serial.println(jsonBuffer);
432          } else {
433              Serial.print("[Telemetry Local Offline] ");
434              Serial.println(jsonBuffer);
435          }
436      }
437  }
```

ingestion/csv_logger.py

```python
"""
CSV Telemetry Logger Module
Thread-safe local CSV logger fulfilling Layer 1 storage requirements.
"""

import os
import csv
import threading
from pathlib import Path
from typing import Dict, Any, List, Optional
import pandas as pd


CSV_FIELDNAMES = [
    "event_id",
    "device_id",
    "timestamp",
    "source",
    "temperature_c",
    "vibration_rms_g",
    "current_a",
    "rpm",
    "sensor_status",
]


class CSVLogger:
    """
    Safely logs incoming telemetry to a CSV file with automatic header creation and thread-safe locking.
    """

    def __init__(self, filepath: Path):
        self.filepath = Path(filepath)
        self.filepath.parent.mkdir(parents=True, exist_ok=True)
        self._lock = threading.Lock()
        self._ensure_header()

    def _ensure_header(self) -> None:
        with self._lock:
            if not self.filepath.exists() or self.filepath.stat().st_size == 0:
                with open(self.filepath, mode="w", newline="", encoding="utf-8") as f:
                    writer = csv.DictWriter(f, fieldnames=CSV_FIELDNAMES)
                    writer.writeheader()

    def append_reading(self, reading: Dict[str, Any]) -> bool:
        """Appends a validated reading dictionary to CSV."""
        row = {col: reading.get(col, "") for col in CSV_FIELDNAMES}
        with self._lock:
            try:
                with open(self.filepath, mode="a", newline="", encoding="utf-8") as f:
                    writer = csv.DictWriter(f, fieldnames=CSV_FIELDNAMES)
                    writer.writerow(row)
                return True
            except Exception as exc:
                print(f"Error appending reading to CSV {self.filepath}: {exc}")
                return False

    def get_recent_readings(self, limit: int = 100) -> pd.DataFrame:
        """Loads the most recent readings into a Pandas DataFrame."""
        with self._lock:
            if not self.filepath.exists() or self.filepath.stat().st_size == 0:
                return pd.DataFrame(columns=CSV_FIELDNAMES)
            try:
                df = pd.read_csv(self.filepath)
                if not df.empty and limit > 0:
                    return df.tail(limit)
                return df
            except Exception:
                return pd.DataFrame(columns=CSV_FIELDNAMES)

    def clear(self) -> None:
```

ingestion/csv_logger.py (continued)

```
        """Resets the CSV file with headers only."""
        with self._lock:
            with open(self.filepath, mode="w", newline="", encoding="utf-8") as f:
                writer = csv.DictWriter(f, fieldnames=CSV_FIELDNAMES)
                writer.writeheader()

```

ingestion/data_validator.py

```python
"""
Data Validation Module
Ensures schema adherence, physical sanity checks, and deduplication for sensor telemetry.
"""

from typing import Dict, Any, Tuple, Optional
from datetime import datetime


REQUIRED_SENSOR_FIELDS = {
    "event_id": str,
    "device_id": str,
    "timestamp": str,
    "source": str,
    "temperature_c": (int, float),
    "vibration_rms_g": (int, float),
    "current_a": (int, float),
    "rpm": (int, float),
    "sensor_status": str,
}


class DataValidator:
    """
    Validates structure, typing, physical plausibility, and formatting of incoming sensor messages.
    """

    def __init__(self, seen_cache_size: int = 1000):
        self._seen_event_ids = set()
        self._cache_size = seen_cache_size

    def is_duplicate(self, event_id: str) -> bool:
        """Returns True if the event_id has already been processed."""
        if event_id in self._seen_event_ids:
            return True
        if len(self._seen_event_ids) >= self._cache_size:
            # Pop roughly 20% oldest entries to prevent memory bloat
            remove_count = self._cache_size // 5
            for _ in range(remove_count):
                if self._seen_event_ids:
                    self._seen_event_ids.pop()
        self._seen_event_ids.add(event_id)
        return False

    def validate_sensor_payload(self, payload: Dict[str, Any]) -> Tuple[bool, Optional[str]]:
        """
        Validates an incoming sensor dictionary.
        Returns: (is_valid: bool, error_message: Optional[str])
        """
        if not isinstance(payload, dict):
            return False, "Payload must be a valid JSON dictionary."

        # Check required fields and types
        for field, expected_type in REQUIRED_SENSOR_FIELDS.items():
            if field not in payload:
                return False, f"Missing mandatory field '{field}'."
            val = payload[field]
            if not isinstance(val, expected_type):
                return False, f"Field '{field}' must be {expected_type}, got {type(val).__name__}."

        # Validate timestamp ISO format
        try:
            ts_str = payload["timestamp"].replace("Z", "+00:00")
            datetime.fromisoformat(ts_str)
        except Exception:
            return False, f"Invalid ISO-8601 timestamp format: '{payload['timestamp']}'."

        # Deduplication check
        event_id = payload["event_id"]
        if self.is_duplicate(event_id):
            return False, f"Duplicate event_id detected: '{event_id}'."
```

ingestion/data_validator.py (continued)

```
72
73          return True, None
74
```

ingestion/mqtt_client.py

```
"""
Robust MQTT Client Wrapper supporting Local Broker & HiveMQ Cloud with TLS.
Compatible with Paho MQTT 2.x and Apple Silicon.
"""

import ssl
import time
import json
import logging
from typing import Callable, Optional, Dict, Any
import certifi
import paho.mqtt.client as mqtt

logger = logging.getLogger("mqtt_client")


class ManagedMQTTClient:
    """
    Manages MQTT connection lifecycle, TLS setup, auto-reconnect, and message dispatch.
    Seamlessly falls back to local broker if cloud credentials are absent.
    """

    def __init__(
        self,
        client_id: str,
        config: Dict[str, Any],
        on_message_callback: Optional[Callable[[str, Dict[str, Any]], None]] = None,
    ):
        self.client_id = client_id
        self.config = config
        self.on_message_callback = on_message_callback
        self.is_connected = False
        self.last_connect_time: Optional[float] = None
        self.last_latency_ms: float = 0.0
        self.reconnect_attempts = 0

        # Initialize Paho Client with modern CallbackAPIVersion
        try:
            self.client = mqtt.Client(
                mqtt.CallbackAPIVersion.VERSION2,
                client_id=self.client_id,
            )
        except AttributeError:
            self.client = mqtt.Client(client_id=self.client_id)

        self._configure_auth_and_tls()
        self._bind_callbacks()

    def _configure_auth_and_tls(self) -> None:
        username = self.config.get("username")
        password = self.config.get("password")
        if username and password:
            self.client.username_pw_set(username, password)

        if self.config.get("use_tls", False):
            context = ssl.create_default_context(cafile=certifi.where())
            context.check_hostname = True
            context.verify_mode = ssl.CERT_REQUIRED
            self.client.tls_set_context(context)
            logger.info("TLS 1.2/1.3 enabled with certifi CA bundle for %s", self.config.get("host"))

    def _bind_callbacks(self) -> None:
        def on_connect(client, userdata, flags, rc, properties=None):
            if rc == 0 or (hasattr(rc, "is_failure") and not rc.is_failure):
                self.is_connected = True
                self.reconnect_attempts = 0
                self.last_connect_time = time.time()
                logger.info("MQTT Client '%s' successfully connected to %s:%s",
                            self.client_id, self.config.get("host"), self.config.get("port"))
            else:
                self.is_connected = False
```

ingestion/mqtt_client.py (continued)

```
                logger.warning("MQTT connection failed with code: %s", rc)

        def on_disconnect(client, userdata, disconnect_flags, rc=None, properties=None):
            self.is_connected = False
            logger.warning("MQTT Client '%s' disconnected (code: %s)", self.client_id, rc)

        def on_message(client, userdata, msg):
            try:
                payload_str = msg.payload.decode("utf-8")
                payload_json = json.loads(payload_str)
                if self.on_message_callback:
                    self.on_message_callback(msg.topic, payload_json)
            except Exception as exc:
                logger.error("Error decoding MQTT message on topic %s: %s", msg.topic, exc)

        self.client.on_connect = on_connect
        self.client.on_disconnect = on_disconnect
        self.client.on_message = on_message

    def connect(self, timeout: int = 5) -> bool:
        host = self.config.get("host", "127.0.0.1")
        port = int(self.config.get("port", 1883))
        try:
            t0 = time.time()
            self.client.connect(host, port, keepalive=60)
            self.client.loop_start()
            # Wait for connection flag with backoff
            elapsed = 0.0
            while not self.is_connected and elapsed < timeout:
                time.sleep(0.1)
                elapsed += 0.1
            if self.is_connected:
                self.last_latency_ms = round((time.time() - t0) * 1000.0, 2)
                return True
            else:
                logger.warning("MQTT connect timed out after %s seconds on %s:%s", timeout, host, port)
                return False
        except Exception as exc:
            logger.error("Failed to connect to MQTT broker %s:%s - %s", host, port, exc)
            self.is_connected = False
            return False

    def subscribe(self, topic: str, qos: int = 1) -> None:
        if self.is_connected:
            self.client.subscribe(topic, qos=qos)
            logger.info("Subscribed to topic: %s", topic)
        else:
            logger.warning("Cannot subscribe to %s: client is not connected", topic)

    def publish(self, topic: str, payload: Dict[str, Any], qos: int = 1, retain: bool = False) -> bool:
        if not self.is_connected:
            return False
        try:
            msg_str = json.dumps(payload)
            info = self.client.publish(topic, msg_str, qos=qos, retain=retain)
            return info.rc == mqtt.MQTT_ERR_SUCCESS
        except Exception as exc:
            logger.error("MQTT Publish error on topic %s: %s", topic, exc)
            return False

    def disconnect(self) -> None:
        try:
            self.client.loop_stop()
            self.client.disconnect()
            self.is_connected = False
            logger.info("MQTT Client '%s' disconnected cleanly", self.client_id)
        except Exception as exc:
            logger.error("Error during MQTT disconnect: %s", exc)

```

ingestion/mqtt_subscriber.py

```
"""
Central MQTT Telemetry Ingestion Service.
Subscribes to motor sensor streams, validates schemas, persists to CSV & SQLite,
and runs the hybrid AI diagnostic pipeline.
"""

import sys
import time
import signal
import logging
from pathlib import Path
from typing import Dict, Any, List

# Add project root to sys.path
sys.path.insert(0, str(Path(__file__).resolve().parent.parent))

from config.settings import settings
from ingestion.mqtt_client import ManagedMQTTClient
from ingestion.data_validator import DataValidator
from ingestion.csv_logger import CSVLogger
from database.repository import repo
from diagnosis.hybrid_engine import hybrid_engine

logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s] %(name)s: %(message)s")
logger = logging.getLogger("mqtt_subscriber")


class TelemetryIngestionService:
    """
    Authoritative ingestion and diagnostic consumer.
    Maintains a sliding history buffer for rate-of-change feature calculations.
    """

    def __init__(self, device_id: str = "motor01"):
        self.device_id = device_id
        self.validator = DataValidator()
        self.csv_logger = CSVLogger(settings.CSV_LOG_PATH)
        self.repo = repo
        self.engine = hybrid_engine

        # Sliding window buffer for temporal features (last 5 readings per device)
        self.history_buffer: Dict[str, List[Dict[str, Any]]] = {}

        # Alert debouncing & persistence tracking
        self.fault_counters: Dict[str, int] = {}
        self.last_alert_time: Dict[str, float] = {}
        self.consecutive_threshold = 3  # consecutive readings required
        self.alert_cooldown_sec = 15.0  # cooldown between repeat alerts

        # MQTT Client
        self.mqtt_config = settings.get_effective_mqtt_config()
        self.client = ManagedMQTTClient(
            client_id=f"ingestion_sub_{device_id}_{int(time.time())}",
            config=self.mqtt_config,
            on_message_callback=self.on_mqtt_message,
        )

        self.sensor_topic = settings.get_sensor_topic(device_id)
        self.diagnosis_topic = settings.get_diagnosis_topic(device_id)
        self.alerts_topic = settings.get_alerts_topic(device_id)
        self.running = False

    def on_mqtt_message(self, topic: str, payload: Dict[str, Any]) -> None:
        """Callback executed on every received MQTT message."""
        if not topic.endswith("/sensors"):
            return


        # ===== ESP32 HARDWARE INTEGRATION =====
        if isinstance(payload, dict) and isinstance(
            payload.get("sensors"), dict
```

ingestion/mqtt_subscriber.py (continued)

```
        ):
            import hashlib
            import json
            import math
            from datetime import datetime

            sensors = payload["sensors"]

            try:
                device_id = payload["device_id"]
                timestamp = payload["timestamp"]

                datetime.fromisoformat(
                    timestamp.replace("Z", "+00:00")
                )

                temp = sensors["temperature_c"]
                current = sensors["current_a"]
                rpm = sensors["rpm"]

                for value in (temp, current, rpm):
                    if (
                        isinstance(value, bool)
                        or not isinstance(value, (int, float))
                        or not math.isfinite(value)
                    ):
                        raise ValueError(
                            "Missing or invalid sensor reading"
                        )

                if not (-55 <= temp <= 125):
                    raise ValueError("Invalid temperature")
                if not (0 <= current <= 30):
                    raise ValueError("Invalid current")
                if not (0 <= rpm <= 20000):
                    raise ValueError("Invalid RPM")

            except (KeyError, ValueError, TypeError) as error:
                logger.warning(
                    "ESP32 sensor validation failed: %s",
                    error
                )
                return

            event_hash = hashlib.sha256(
                json.dumps(
                    payload, sort_keys=True
                ).encode()
            ).hexdigest()[:32]

            reading = {
                "event_id": "esp32_" + event_hash,
                "device_id": device_id,
                "timestamp": timestamp,
                "source": "esp32",
                "temperature_c": float(temp),
                "vibration_rms_g": None,
                "current_a": float(current),
                "rpm": float(rpm),
                "sensor_status": "partial"
            }


            inserted = self.repo.insert_sensor_reading(reading)

            if not inserted:
                with self.repo.manager.session() as conn:
                    duplicate = conn.execute(
                        "SELECT 1 FROM sensor_readings "
                        "WHERE event_id = ? LIMIT 1",
                        (reading["event_id"],)
```

ingestion/mqtt_subscriber.py (continued) (continued)

```
                    ).fetchone()

                if duplicate:
                    logger.debug(
                        "Duplicate ESP32 MQTT reading skipped: %s",
                        reading["event_id"]
                    )
                else:
                    logger.error(
                        "ESP32 reading rejected by database: %s",
                        reading["event_id"]
                    )
                return

            logger.info(
                "ESP32 SAVED TO SQLITE: "
                "Temp=%.2f C | Current=%.3f A | RPM=%.1f",
                temp, current, rpm
            )

            try:
                from diagnosis.hardware_rules import diagnose_esp32

                if not hasattr(self, "_hardware_zero_rpm_counts"):
                    self._hardware_zero_rpm_counts = {}

                if payload.get("motor_status") == "ON" and rpm <= 0:
                    zero_count = (
                        self._hardware_zero_rpm_counts.get(device_id, 0)
                        + 1
                    )
                else:
                    zero_count = 0

                self._hardware_zero_rpm_counts[device_id] = zero_count

                prediction = diagnose_esp32(
                    reading,
                    payload.get("motor_status"),
                    rpm_zero_persistent=(zero_count >= 2)
                )

                self.repo.insert_fault_prediction(prediction)
                self._evaluate_alerts(device_id, prediction, reading)

                logger.info(
                    "ESP32 RULE DIAGNOSIS: %s [%s]",
                    prediction["rule_fault"],
                    prediction["rule_severity"],
                )

            except Exception:
                logger.exception(
                    "Hardware diagnosis failed; "
                    "sensor data was still saved"
                )

            return

        # 1. Validate payload structure & deduplicate
        is_valid, err_msg = self.validator.validate_sensor_payload(payload)
        if not is_valid:
            logger.warning("Rejected invalid telemetry message on %s: %s", topic, err_msg)
            return

        dev_id = payload["device_id"]

        # 2. Layer 1 Persistence: Append to CSV
        self.csv_logger.append_reading(payload)

        # 3. Layer 3 Persistence: Authoritative SQLite storage
```

ingestion/mqtt_subscriber.py (continued) (continued) (continued)

```
        inserted = self.repo.insert_sensor_reading(payload)
        if not inserted:
            logger.debug("Duplicate sensor reading skipped in DB: %s", payload["event_id"])
            return

        # Maintain sliding history buffer for temporal feature extraction
        history = self.history_buffer.setdefault(dev_id, [])
        history.append(payload)
        if len(history) > 10:
            history.pop(0)

        # 4. Layer 5 Diagnostic Engine: Evaluate Rule + ML Hybrid
        diagnosis = self.engine.diagnose(payload, history)

        # Save diagnosis to SQLite
        self.repo.insert_fault_prediction(diagnosis)

        # Publish diagnostic verdict back to MQTT
        self.client.publish(self.diagnosis_topic, diagnosis)

        # 5. Layer 4 Automation & Alert Evaluation
        self._evaluate_alerts(dev_id, diagnosis, payload)

    def _evaluate_alerts(self, dev_id: str, diagnosis: Dict[str, Any], raw_reading: Dict[str, Any]) -> None:
        """Evaluates consecutive persistence and alert cooldown."""
        sev = diagnosis["hybrid_severity"]
        fault = diagnosis["hybrid_fault"]
        now = time.time()

        # For ESP32 hardware, count consecutive readings
        # only when they report the SAME fault.
        if raw_reading.get("source") == "esp32":
            if not hasattr(self, "_hardware_previous_faults"):
                self._hardware_previous_faults = {}

            previous_fault = self._hardware_previous_faults.get(
                dev_id
            )

            if previous_fault != fault:
                self.fault_counters[dev_id] = 0

            self._hardware_previous_faults[dev_id] = fault

        if sev == "NORMAL":
            # Clear counter when healthy
            self.fault_counters[dev_id] = 0
            return

        # Increment consecutive counter for persistent conditions
        self.fault_counters[dev_id] = self.fault_counters.get(dev_id, 0) + 1
        count = self.fault_counters[dev_id]

        # Trigger alert if consecutive threshold is met or if severity is CRITICAL
        is_critical = sev in ("CRITICAL", "UNKNOWN / SENSOR ERROR")
        last_alert = self.last_alert_time.get(dev_id, 0.0)
        cooldown_elapsed = (now - last_alert) >= self.alert_cooldown_sec

        is_hardware = raw_reading.get("source") == "esp32"

        if is_hardware:
            # Critical faults alert immediately.
            # Missing speed feedback already requires
            # two consecutive zero-RPM measurements.
            # Other warnings require two abnormal diagnoses.
            should_alert = (
                is_critical
                or fault == "Speed Feedback Missing"
                or count >= 2
            )
        else:
```

ingestion/mqtt_subscriber.py (continued) (continued) (continued) (continued)

```
            should_alert = (
                count >= self.consecutive_threshold
                or is_critical
            )

        if should_alert and cooldown_elapsed:
            self.last_alert_time[dev_id] = now
            title = f"{sev}: {fault} on {dev_id}"
            message = (
                f"Diagnostic Alert: {fault} (Severity: {sev}). "
                f"Confidence: {diagnosis.get('hybrid_confidence', 0.0)*100:.1f}%. "
                f"T={raw_reading['temperature_c']}°C, V={raw_reading['vibration_rms_g']}g, "
                f"I={raw_reading['current_a']}A, RPM={raw_reading['rpm']}. "
                f"Action: {diagnosis.get('recommended_action')}"
            )

            if is_hardware:
                message = (
                    f"Hardware Alert: {fault} "
                    f"(Severity: {sev}). "
                    f"Temperature={raw_reading['temperature_c']} C, "
                    f"Current={raw_reading['current_a']} A, "
                    f"RPM={raw_reading['rpm']}. "
                    f"Vibration not measured. "
                    f"ML diagnosis not evaluated. "
                    f"Action: {diagnosis.get('recommended_action')}"
                )

            # Record in SQLite alert history
            alert_id = self.repo.insert_alert(
                device_id=dev_id,
                severity=sev,
                title=title,
                message=message,
                timestamp=raw_reading["timestamp"],
            )

            # Publish actionable alert to MQTT (for Node-RED / ESP32 buzzer/LED)
            alert_payload = {
                "alert_id": alert_id,
                "device_id": dev_id,
                "timestamp": raw_reading["timestamp"],
                "severity": sev,
                "fault": fault,
                "title": title,
                "message": message,
                "recommended_action": diagnosis.get("recommended_action"),
                "actuate_hardware": is_critical and not is_hardware,  # flag for ESP32 buzzer/LED
            }
            self.client.publish(self.alerts_topic, alert_payload)
            logger.warning("[ALERT DISPATCHED] %s (ID: %s)", title, alert_id)

    def start(self) -> None:
        """Starts subscriber connection and message loop."""
        logger.info("Connecting Ingestion Service to MQTT (%s at %s:%s)...",
                    self.mqtt_config["broker_type"], self.mqtt_config["host"], self.mqtt_config["port"])
        if not self.client.connect(timeout=6):
            logger.warning("MQTT broker connection could not be established immediately. Waiting in loop...")

        # Subscribe to all sensor topics
        wildcard_topic = "iot/factory/+/sensors"
        self.client.subscribe(wildcard_topic)
        self.running = True
        logger.info("Telemetry Ingestion Service listening on '%s'", wildcard_topic)

        try:
            while self.running:
                time.sleep(0.5)
        except KeyboardInterrupt:
            logger.info("Ingestion Service stopped by user.")
        finally:
```

ingestion/mqtt_subscriber.py (continued) (continued) (continued) (continued) (continued)

```
            self.stop()

    def stop(self) -> None:
        self.running = False
        self.client.disconnect()
        logger.info("Ingestion Service stopped.")


def main():
    service = TelemetryIngestionService()

    def sig_handler(sig, frame):
        service.stop()
        sys.exit(0)

    signal.signal(signal.SIGINT, sig_handler)
    signal.signal(signal.SIGTERM, sig_handler)

    service.start()


if __name__ == "__main__":
    main()

```

ml/evaluate.py

```
"""
Machine Learning Evaluation Module.
Inspects saved model artifacts, evaluates performance against test data,
and produces detailed metrics, confusion matrix, and feature importance tables.
"""

import sys
import json
from pathlib import Path
from typing import Dict, Any

# Add project root to sys.path
sys.path.insert(0, str(Path(__file__).resolve().parent.parent))

import joblib
import pandas as pd
import numpy as np
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score, f1_score

from config.settings import settings
from ml.features import extract_batch_features, FEATURE_NAMES


def evaluate_saved_model(test_csv_path: Path = settings.SYNTHETIC_DIR / "motor_training_data.csv") -> Dict[str, Any]:
    """
    Evaluates the currently saved Random Forest model on the synthetic test partition.
    Prints professional tabular output suitable for verification and project reports.
    """
    model_path = settings.ML_MODEL_PATH
    if not model_path.exists():
        raise FileNotFoundError(f"Trained model not found at {model_path}. Run 'python ml/train.py' first.")

    bundle = joblib.load(model_path)
    model = bundle["model"]
    feature_names = bundle.get("feature_names", FEATURE_NAMES)
    classes = bundle.get("classes", [])

    print("================================================================================")
    print("                  MACHINE LEARNING MODEL EVALUATION REPORT                      ")
    print("================================================================================")
    print(f"Model Type:        {bundle.get('model_type', 'Random Forest Classifier')}")
    print(f"Model Path:        {model_path}")
    print(f"Trained Timestamp: {bundle.get('trained_at', 'N/A')}")
    print(f"Classes ({len(classes)}):     {', '.join(classes)}")
    print(f"Total Estimators:  {getattr(model, 'n_estimators', 'N/A')}")
    print("--------------------------------------------------------------------------------")

    if not test_csv_path.exists():
        print(f"Dataset not found at {test_csv_path}. Skipping dynamic test evaluation.")
        metrics_path = settings.ML_METRICS_PATH
        if metrics_path.exists():
            with open(metrics_path, "r", encoding="utf-8") as f:
                saved_metrics = json.load(f)
            print("Loaded Stored Metrics:")
            print(json.dumps(saved_metrics, indent=2))
            return saved_metrics
        return {}

    df = pd.read_csv(test_csv_path)

    # Recreate the exact test partition (holdout of the last run per class)
    test_runs = []
    for cls in df["label"].unique():
        cls_runs = sorted(df[df["label"] == cls]["run_id"].unique().tolist())
        test_runs.append(cls_runs[-1])

    test_df = df[df["run_id"].isin(test_runs)].reset_index(drop=True)
    X_test = extract_batch_features(test_df)
    y_test = test_df["label"].values

```

ml/evaluate.py (continued)

```
    y_pred = model.predict(X_test)
    acc = float(accuracy_score(y_test, y_pred))
    f1_weighted = float(f1_score(y_test, y_pred, average="weighted", zero_division=0))

    print(f"\nTest Dataset Size: {len(test_df)} samples across {len(test_runs)} held-out episodes")
    print(f"Overall Accuracy:  {acc * 100:.2f}%")
    print(f"Weighted F1 Score: {f1_weighted:.4f}")
    print("\nClassification Report:")
    print(classification_report(y_test, y_pred, digits=4, zero_division=0))

    labels_list = sorted(list(classes))
    cm = confusion_matrix(y_test, y_pred, labels=labels_list)
    cm_df = pd.DataFrame(cm, index=[f"True: {l}" for l in labels_list], columns=[f"Pred: {l}" for l in
labels_list])
    print("\nConfusion Matrix:")
    print(cm_df.to_string())

    if hasattr(model, "feature_importances_"):
        importances = dict(zip(feature_names, model.feature_importances_))
        sorted_imp = sorted(importances.items(), key=lambda x: x[1], reverse=True)
        print("\nFeature Importances:")
        for feat, imp in sorted_imp:
            bar = "" * int(imp * 40)
            print(f"  {feat:<22} : {imp:6.4f} | {bar}")

    print("================================================================================")
    print("Academic Notice: Metrics evaluated on synthetic physics simulations.")
    print("Physical motor real-world accuracy requires calibration with real sensor runs.")
    print("================================================================================")

    return {
        "accuracy": acc,
        "weighted_f1": f1_weighted,
        "classes": labels_list,
        "samples_tested": len(test_df),
    }


if __name__ == "__main__":
    evaluate_saved_model()
```

ml/features.py

```python
"""
Feature Engineering Module for Motor Fault Diagnosis.
Extracts instantaneous, rate-of-change, and rolling window statistical features.
"""

from typing import List, Dict, Any
import numpy as np
import pandas as pd


FEATURE_NAMES = [
    "temperature_c",
    "vibration_rms_g",
    "current_a",
    "rpm",
    "temp_trend",
    "vibe_trend",
    "current_delta",
    "rpm_delta",
    "power_est_w",  # V * I = 12.0 * current_a
    "current_per_rpm",  # Torque load proxy
]


def extract_single_reading_features(
    current_reading: Dict[str, Any],
    recent_history: List[Dict[str, Any]] = None,
) -> np.ndarray:
    """
    Extracts feature vector from the latest reading and an optional sliding window history (3-5 samples).
    """
    temp = float(current_reading.get("temperature_c", 35.0))
    vibe = float(current_reading.get("vibration_rms_g", 0.12))
    current = float(current_reading.get("current_a", 0.60))
    rpm = float(current_reading.get("rpm", 2100.0))

    temp_trend = 0.0
    vibe_trend = 0.0
    current_delta = 0.0
    rpm_delta = 0.0

    if recent_history and len(recent_history) >= 1:
        prev = recent_history[-1]
        prev_temp = float(prev.get("temperature_c", temp))
        prev_vibe = float(prev.get("vibration_rms_g", vibe))
        prev_curr = float(prev.get("current_a", current))
        prev_rpm = float(prev.get("rpm", rpm))

        temp_trend = temp - prev_temp
        vibe_trend = vibe - prev_vibe
        current_delta = current - prev_curr
        rpm_delta = rpm - prev_rpm

    # Engineered physical interactions
    power_est = 12.0 * current
    current_per_rpm = current / max(rpm, 10.0)

    features = [
        temp,
        vibe,
        current,
        rpm,
        temp_trend,
        vibe_trend,
        current_delta,
        rpm_delta,
        power_est,
        current_per_rpm,
    ]
    return np.array(features, dtype=np.float32).reshape(1, -1)

```

ml/features.py (continued)

```
 72
 73 def extract_batch_features(df: pd.DataFrame) -> pd.DataFrame:
 74     """
 75     Computes features over a time-series DataFrame of readings.
 76     """
 77     features_df = pd.DataFrame()
 78     features_df["temperature_c"] = df["temperature_c"].astype(float)
 79     features_df["vibration_rms_g"] = df["vibration_rms_g"].astype(float)
 80     features_df["current_a"] = df["current_a"].astype(float)
 81     features_df["rpm"] = df["rpm"].astype(float)
 82
 83     # Calculate changes independently within each simulation run.
 84     # Prevents data from one episode entering another episode.
 85
 86     def within_run_diff(column):
 87         if "run_id" in df.columns:
 88             return (
 89                 df.groupby("run_id", sort=False)[column]
 90                 .diff()
 91                 .fillna(0.0)
 92             )
 93         return df[column].diff().fillna(0.0)
 94
 95     # Trends and deltas
 96
 97     features_df["temp_trend"] = within_run_diff("temperature_c")
 98     features_df["vibe_trend"] = within_run_diff("vibration_rms_g")
 99     features_df["current_delta"] = within_run_diff("current_a")
100     features_df["rpm_delta"] = within_run_diff("rpm")
101
102     # Physical domain interactions
103     features_df["power_est_w"] = 12.0 * features_df["current_a"]
104     features_df["current_per_rpm"] = features_df["current_a"] / features_df["rpm"].clip(lower=10.0)
105
106     return features_df[FEATURE_NAMES]
```

ml/generate_dataset.py

```python
"""
Synthetic Dataset Generator for Machine Learning Fault Classification.
Generates balanced, non-leaking simulated runs for all diagnostic classes.
"""

import sys
from pathlib import Path

# Add project root to sys.path
sys.path.insert(0, str(Path(__file__).resolve().parent.parent))

import numpy as np
import pandas as pd
from config.settings import settings
from simulation.scenarios import MotorScenario
from simulation.synthetic_generator import SyntheticGenerator


def generate_training_dataset(
    samples_per_scenario: int = 500,
    seed: int = 100,
    output_path: Path = settings.SYNTHETIC_DIR / "motor_training_data.csv",
) -> pd.DataFrame:
    """
    Simulates multi-run episodes across all scenarios and saves to CSV.
    Uses independent runs so train/test split can cleanly separate episodes.
    """
    scenarios_to_sample = [
        MotorScenario.NORMAL,
        MotorScenario.MECHANICAL_OVERLOAD,
        MotorScenario.EXCESSIVE_VIBRATION,
        MotorScenario.MOTOR_OVERHEATING,
        MotorScenario.MOTOR_SLOWDOWN,
        MotorScenario.MOTOR_STALL,
        MotorScenario.SENSOR_ERROR,
    ]

    all_rows = []
    run_counter = 0

    for scenario in scenarios_to_sample:
        # Create 5 distinct episodes per scenario to enable group-based splitting
        episodes = 5
        steps_per_episode = samples_per_scenario // episodes

        for ep in range(episodes):
            run_counter += 1
            gen = SyntheticGenerator(device_id="motor01", seed=seed + run_counter * 17)
            gen.set_scenario(scenario)

            for step in range(steps_per_episode):
                sensor_reading, ground_truth = gen.generate_next_sample()
                if sensor_reading:
                    row = sensor_reading.copy()
                    row["run_id"] = run_counter
                    row["label"] = ground_truth["ground_truth_label"]
                    all_rows.append(row)

    df = pd.DataFrame(all_rows)
    output_path.parent.mkdir(parents=True, exist_ok=True)
    df.to_csv(output_path, index=False)
    print(f"Generated {len(df)} synthetic training samples across {len(scenarios_to_sample)} classes -> {output_path}")
    print(f"Class breakdown:\n{df['label'].value_counts()}")
    return df


if __name__ == "__main__":
    generate_training_dataset()

```

ml/models/confusion_matrix.json

```
{
  "labels": [
    "Excessive Vibration",
    "Mechanical Overload",
    "Motor Overheating",
    "Motor Slowdown",
    "Motor Stall",
    "Normal",
    "Sensor Error"
  ],
  "matrix": [
    [
      100,
      0,
      0,
      0,
      0,
      0,
      0
    ],
    [
      0,
      100,
      0,
      0,
      0,
      0,
      0
    ],
    [
      0,
      0,
      100,
      0,
      0,
      0,
      0
    ],
    [
      0,
      0,
      0,
      100,
      0,
      0,
      0
    ],
    [
      0,
      0,
      0,
      0,
      100,
      0,
      0
    ],
    [
      0,
      0,
      0,
      0,
      0,
      100,
      0
    ],
    [
      0,
      0,
      0,
      0,
      0,
      100,
      0,
```

ml/models/confusion_matrix.json (continued)

```
      0,
      100
    ]
  ]
}
```

ml/models/model_metrics.json


```
{
  "timestamp": "2026-10-31T15:44:31.766570+00:00",
  "algorithm": "RandomForestClassifier",
  "n_estimators": 100,
  "baseline_decision_tree": {
    "accuracy": 0.9686,
    "weighted_f1": 0.9684
  },
  "random_forest": {
    "accuracy": 1.0,
    "weighted_f1": 1.0
  },
  "classification_report": {
    "Excessive Vibration": {
      "precision": 1.0,
      "recall": 1.0,
      "f1-score": 1.0,
      "support": 100.0
    },
    "Mechanical Overload": {
      "precision": 1.0,
      "recall": 1.0,
      "f1-score": 1.0,
      "support": 100.0
    },
    "Motor Overheating": {
      "precision": 1.0,
      "recall": 1.0,
      "f1-score": 1.0,
      "support": 100.0
    },
    "Motor Slowdown": {
      "precision": 1.0,
      "recall": 1.0,
      "f1-score": 1.0,
      "support": 100.0
    },
    "Motor Stall": {
      "precision": 1.0,
      "recall": 1.0,
      "f1-score": 1.0,
      "support": 100.0
    },
    "Normal": {
      "precision": 1.0,
      "recall": 1.0,
      "f1-score": 1.0,
      "support": 100.0
    },
    "Sensor Error": {
      "precision": 1.0,
      "recall": 1.0,
      "f1-score": 1.0,
      "support": 100.0
    },
    "accuracy": 1.0,
    "macro avg": {
      "precision": 1.0,
      "recall": 1.0,
      "f1-score": 1.0,
      "support": 700.0
    },
    "weighted avg": {
      "precision": 1.0,
      "recall": 1.0,
      "f1-score": 1.0,
      "support": 700.0
    }
  },
  "feature_importances": {
    "rpm": 0.21303679792774055,
```

ml/models/model_metrics.json (continued)


```
    "current_per_rpm": 0.2043126802265822,
    "vibration_rms_g": 0.16529378394411257,
    "power_est_w": 0.12063324444339578,
    "current_a": 0.11652516910490876,
    "temperature_c": 0.11587069573525116,
    "vibe_trend": 0.026006753220545943,
    "temp_trend": 0.01693832985464556,
    "current_delta": 0.015165757464336842,
    "rpm_delta": 0.006216788078480789
  },
  "classes": [
    "Excessive Vibration",
    "Mechanical Overload",
    "Motor Overheating",
    "Motor Slowdown",
    "Motor Stall",
    "Normal",
    "Sensor Error"
  ],
  "limitations_note": "Trained on synthetic physical simulation. Real DC motor deployment requires transfer calibration against physical benchmark loads."
}
```

ml/predict.py

```
"""
Machine Learning Inference Engine for Real-Time Motor Fault Diagnosis.
Performs live feature extraction, confidence scoring, and fallback handling.
"""

import logging
from pathlib import Path
from typing import Dict, Any, List, Optional, Tuple
import joblib
import numpy as np

from config.settings import settings
from ml.features import extract_single_reading_features

logger = logging.getLogger("ml_predict")


class FaultPredictor:
    """
    Loads trained Random Forest model and provides real-time probabilistic inference.
    """

    def __init__(self, model_path: Path = settings.ML_MODEL_PATH):
        self.model_path = Path(model_path)
        self.model = None
        self.classes: List[str] = []
        self.is_loaded = False
        self.load_model()

    def load_model(self) -> bool:
        """Attempts to load the joblib model bundle."""
        if not self.model_path.exists():
            logger.warning("ML model bundle not found at %s. Operating in rules-only fallback mode.",
self.model_path)
            self.is_loaded = False
            return False
        try:
            bundle = joblib.load(self.model_path)
            self.model = bundle["model"]
            self.classes = bundle.get("classes", [])
            self.feature_names = bundle.get("feature_names", [])
            self.is_loaded = True
            logger.info("Successfully loaded ML model with %s classes.", len(self.classes))
            return True
        except Exception as exc:
            logger.error("Failed to load ML model from %s: %s", self.model_path, exc)
            self.is_loaded = False
            return False

    def predict(
        self,
        current_reading: Dict[str, Any],
        recent_history: Optional[List[Dict[str, Any]]] = None,
    ) -> Tuple[Optional[str], float, Dict[str, float]]:
        """
        Runs inference on reading.
        Returns:
            (predicted_fault: Optional[str], confidence: float, class_probabilities: Dict[str, float])
        """
        if not self.is_loaded or self.model is None:
            return None, 0.0, {}

        try:
            import pandas as pd
            features = extract_single_reading_features(current_reading, recent_history)
            feat_df = pd.DataFrame(features, columns=self.feature_names)
            probabilities = self.model.predict_proba(feat_df)[0]
            top_idx = int(np.argmax(probabilities))
            predicted_class = self.classes[top_idx]
            confidence = float(probabilities[top_idx])
            prob_dict = {cls: float(round(prob, 4)) for cls, prob in zip(self.classes, probabilities)}
```

ml/predict.py (continued)

```
71
72              return predicted_class, round(confidence, 4), prob_dict
73          except Exception as exc:
74              logger.error("Inference execution error: %s", exc)
75              return None, 0.0, {}
76
77
78  predictor = FaultPredictor()
```

ml/train.py

```python
"""
Machine Learning Training Pipeline for Motor Fault Classification.
Trains Random Forest Classifier with rolling physics features, evaluates metrics,
and saves the trained model bundle and explainability artifacts.
"""

import sys
import json
from pathlib import Path

# Add project root to sys.path
sys.path.insert(0, str(Path(__file__).resolve().parent.parent))

import joblib
import numpy as np
import pandas as pd
from sklearn.model_selection import GroupShuffleSplit
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score, f1_score

from config.settings import settings
from ml.features import extract_batch_features, FEATURE_NAMES
from ml.generate_dataset import generate_training_dataset


def run_training_pipeline() -> dict:
    """Runs data generation, feature engineering, model training, and evaluation."""
    data_path = settings.SYNTHETIC_DIR / "motor_training_data.csv"
    if not data_path.exists():
        print("Training dataset not found. Generating new dataset...")
        df = generate_training_dataset(output_path=data_path)
    else:
        df = pd.read_csv(data_path)

    print(f"Loaded dataset: {len(df)} samples across {df['label'].nunique()} classes.")

    # Feature extraction
    X = extract_batch_features(df)
    y = df["label"].values
    groups = df["run_id"].values

    # Train / Test split: ensure 1 run per class is in test (20% holdout) and 4 runs in train (80%)
    # Strictly zero leakage across episodes while guaranteeing all 7 classes in train & test
    test_runs = []
    train_runs = []
    for cls in df["label"].unique():
        cls_runs = sorted(df[df["label"] == cls]["run_id"].unique().tolist())
        # Hold out the last run for test
        test_runs.append(cls_runs[-1])
        train_runs.extend(cls_runs[:-1])

    train_mask = df["run_id"].isin(train_runs)
    test_mask = df["run_id"].isin(test_runs)

    X_train, X_test = X[train_mask], X[test_mask]
    y_train, y_test = y[train_mask], y[test_mask]

    print(f"Train set: {len(X_train)} samples ({len(train_runs)} runs), Test set: {len(X_test)} samples ({len(test_runs)} runs).")

    # 1. Baseline Model (Decision Tree)
    baseline_clf = DecisionTreeClassifier(max_depth=5, random_state=42)
    baseline_clf.fit(X_train, y_train)
    y_pred_base = baseline_clf.predict(X_test)
    base_acc = float(accuracy_score(y_test, y_pred_base))
    base_f1 = float(f1_score(y_test, y_pred_base, average="weighted", zero_division=0))

    # 2. Primary Model (Random Forest Classifier)
    rf_clf = RandomForestClassifier(
        n_estimators=100,
```

ml/train.py (continued)

```
        max_depth=12,
        min_samples_split=4,
        random_state=42,
        class_weight="balanced",
    )
    rf_clf.fit(X_train, y_train)
    y_pred_rf = rf_clf.predict(X_test)
    rf_acc = float(accuracy_score(y_test, y_pred_rf))
    rf_f1 = float(f1_score(y_test, y_pred_rf, average="weighted", zero_division=0))

    print("\n--- MODEL PERFORMANCE COMPARISON ---")
    print(f"Baseline Decision Tree -> Accuracy: {base_acc:.4f} | Weighted F1: {base_f1:.4f}")
    print(f"Random Forest Classifier -> Accuracy: {rf_acc:.4f} | Weighted F1: {rf_f1:.4f}")

    # Detailed Classification Report
    labels = sorted(list(set(y)))
    report = classification_report(y_test, y_pred_rf, labels=labels, target_names=labels, output_dict=True,
zero_division=0)
    cm = confusion_matrix(y_test, y_pred_rf, labels=labels)

    # Feature importances
    importances = dict(zip(FEATURE_NAMES, [float(x) for x in rf_clf.feature_importances_]))
    sorted_importances = dict(sorted(importances.items(), key=lambda item: item[1], reverse=True))

    print("\nTop Predictive Features:")
    for feat, imp in list(sorted_importances.items())[:5]:
        print(f"  - {feat}: {imp:.4f}")

    # Save Model Bundle
    settings.MODELS_DIR.mkdir(parents=True, exist_ok=True)
    model_bundle = {
        "model": rf_clf,
        "feature_names": FEATURE_NAMES,
        "classes": rf_clf.classes_.tolist(),
        "baseline_accuracy": base_acc,
        "rf_accuracy": rf_acc,
        "rf_f1": rf_f1,
    }
    joblib.dump(model_bundle, settings.ML_MODEL_PATH)
    print(f"\nSaved trained model to {settings.ML_MODEL_PATH}")

    # Save Metrics JSON
    metrics_summary = {
        "timestamp": pd.Timestamp.now(tz="UTC").isoformat(),
        "algorithm": "RandomForestClassifier",
        "n_estimators": 100,
        "baseline_decision_tree": {
            "accuracy": round(base_acc, 4),
            "weighted_f1": round(base_f1, 4),
        },
        "random_forest": {
            "accuracy": round(rf_acc, 4),
            "weighted_f1": round(rf_f1, 4),
        },
        "classification_report": report,
        "feature_importances": sorted_importances,
        "classes": labels,
        "limitations_note": "Trained on synthetic physical simulation. Real DC motor deployment requires
transfer calibration against physical benchmark loads."
    }
    with open(settings.ML_METRICS_PATH, "w", encoding="utf-8") as f:
        json.dump(metrics_summary, f, indent=2)

    # Save Confusion Matrix JSON
    cm_payload = {
        "labels": labels,
        "matrix": cm.tolist(),
    }
    with open(settings.ML_CONFUSION_MATRIX_PATH, "w", encoding="utf-8") as f:
        json.dump(cm_payload, f, indent=2)

```

ml/train.py (continued) (continued)

```
    return metrics_summary


if __name__ == "__main__":
    run_training_pipeline()
```

nodered/README.md

```
# Node-RED Industrial Automation & Alert Flow

This directory contains the production Node-RED workflow orchestrating telemetry parsing, validation, debounce hysteresis, and alert dispatch for the **Knowledge-Driven IoT-Based Fault Diagnosis System**.

---

## 1. Flow Capabilities

The flow in `flows.json` provides:
1. **MQTT Telemetry Subscription**: Listens to wildcard topic `iot/factory/+/sensors`.
2. **Payload Parsing & Deduplication**: Parses JSON and drops duplicate readings by caching recent `event_id` tokens.
3. **Multi-Condition Alert Evaluation**:
   - **Motor Stall / Locked Rotor**: $I \ge 1.70\,\text{A} \land \text{RPM} \le 250$ $\to$ `CRITICAL`
   - **Motor Overheating**: $T \ge 65.0^\circ\text{C} \land I \ge 0.90\,\text{A}$ $\to$ `CRITICAL`
   - **Sensor Out-of-Range**: $T < -20^\circ\text{C} \lor T > 120^\circ\text{C} \lor I < 0 \lor \text{RPM} < 0$ $\to$ `UNKNOWN / SENSOR ERROR`
   - **Mechanical Overload**: $I \ge 1.05\,\text{A} \land \text{RPM} < 1600$ $\to$ `WARNING`
   - **Excessive Vibration**: $\text{RMS} \ge 0.40\,g$ $\to$ `WARNING`
4. **Hysteresis & Cooldown Suppression**:
   - Requires **3 consecutive anomalous readings** before firing a persistent alert (unless severity is `CRITICAL`).
   - Enforces a **15-second alert cooldown** to prevent message flooding.
5. **Recovery Notification**: Dispatches an automatic recovery message when telemetry returns to nominal thresholds after an active fault state.
6. **MQTT Alert Publishing**: Dispatches actionable alert events to `iot/factory/motor01/alerts`.

---

## 2. Launching Node-RED on Apple Silicon Mac

### Option A: Using Docker Compose (Recommended)
From the project root:
```bash
docker compose up nodered -d
```
Node-RED will be available at: `http://localhost:1880`.

Option B: Running Node-RED Locally via npm
```bash
cd nodered
npm install
npx node-red --userDir . flows.json
```

---

3. Importing the Flow Manually
If running an existing Node-RED instance:
1. Open `http://localhost:1880` in your browser.
2. Click the top-right hamburger menu $\to$ **Import**.
3. Select `nodered/flows.json` or paste its contents.
4. Click **Import** $\to$ **Deploy**.
5. Check the MQTT broker node configuration:
   - For local Mosquitto: `127.0.0.1:1883`
   - For HiveMQ Cloud: Host `xxxxxx.s1.eu.hivemq.cloud`, Port `8883`, TLS enabled, with your HiveMQ credentials.
```

nodered/package.json

```
{
  "name": "node-red-industrial-fault-monitor",
  "description": "Node-RED Industrial Automation Flow for 12V DC Motor Fault Alerts",
  "version": "1.0.0",
  "dependencies": {
    "@flowfuse/node-red-dashboard": "^1.18.0"
  }
}

```

scripts/init_admin.py

```python
"""
Admin Initialization CLI Script.
Safely initializes default [REDACTED]istrative and operator credentials without committing plaintext passwords.
"""

import sys
import argparse
from getpass import getpass
from pathlib import Path

# Add project root to sys.path
sys.path.insert(0, str(Path(__file__).resolve().parent.parent))

from backend.auth.security import auth_manager
from database.connection import db_manager


def init_default_users([REDACTED]_pass: str = None, operator_pass: str = None):
    print("Checking database users table...")
    with db_manager.session() as conn:
        cursor = conn.execute("SELECT username FROM users WHERE username = '[REDACTED]'")
        [REDACTED]_exists = cursor.fetchone() is not None

    if not [REDACTED]_exists:
        if not [REDACTED]_pass:
            [REDACTED]_pass = getpass("Set [REDACTED]istrator password: ")
        if len([REDACTED]_pass) < 12:
            raise ValueError("Password must be at least 12 characters")
        auth_manager.create_user("[REDACTED]", [REDACTED]_pass, role="[REDACTED]")
        print("Administrator account created successfully.")
    else:
        print("□□ Admin account already exists.")

    with db_manager.session() as conn:
        cursor = conn.execute("SELECT username FROM users WHERE username = 'operator'")
        op_exists = cursor.fetchone() is not None

    if not op_exists:
        if not operator_pass:
            operator_pass = getpass("Set operator password: ")
        if len(operator_pass) < 12:
            raise ValueError("Password must be at least 12 characters")
        auth_manager.create_user("operator", operator_pass, role="operator")
        print("Operator account created successfully.")
    else:
        print("□□ Operator account already exists.")


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Initialize default database users")
    parser.add_argument("--[REDACTED]-pass", default=None, help="Administrator password")
    parser.add_argument("--operator-pass", default=None, help="Operator password")
    args = parser.parse_args()

    init_default_users(args.[REDACTED]_pass, args.operator_pass)

```

scripts/run_demo.sh

```
#!/usr/bin/env bash
# ================================================================================
# One-Command Local Demo Launcher
# Knowledge-Driven IoT Fault Diagnosis System
# Launches:
#   1. Ingestion Service (Telemetry validator, SQLite/CSV writer, Hybrid AI Engine)
#   2. Synthetic Publisher (Simulating dynamic motor operating scenarios)
#   3. Streamlit Industrial Dashboard (Port 8501)
# ================================================================================

set -e

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"

cd "$PROJECT_ROOT"

VENV_PY="$PROJECT_ROOT/.venv/bin/python"
VENV_STREAMLIT="$PROJECT_ROOT/.venv/bin/streamlit"

if [ ! -f "$VENV_PY" ]; then
    echo "Virtual environment not found. Running setup.sh first..."
    bash "$SCRIPT_DIR/setup.sh"
fi

echo "================================================================================"
echo "⚡ Starting Knowledge-Driven IoT Fault Diagnosis System (Local Demo Mode)"
echo "Mode: Local Simulation & Diagnostics"
echo "================================================================================"

PID_SUB=""
PID_PUB=""
PID_DASH=""

cleanup() {
    echo -e "\n\nShutting down all demonstration processes gracefully..."
    [ -n "$PID_PUB" ] && kill -TERM "$PID_PUB" 2>/dev/null || true
    [ -n "$PID_SUB" ] && kill -TERM "$PID_SUB" 2>/dev/null || true
    [ -n "$PID_DASH" ] && kill -TERM "$PID_DASH" 2>/dev/null || true
    wait 2>/dev/null || true
    echo "All processes cleanly stopped. Goodbye."
    exit 0
}

trap cleanup SIGINT SIGTERM EXIT

# 1. Start Telemetry Ingestion Service in background
echo "Starting Telemetry Ingestion & Diagnostic Consumer..."
"$VENV_PY" ingestion/mqtt_subscriber.py > /dev/null 2>&1 &
PID_SUB=$!
echo "✓ Ingestion Service started (PID: $PID_SUB)"

# 2. Start Synthetic Data Publisher in background (cycles scenarios)
echo "Starting Synthetic Telemetry Publisher (motor01)..."
"$VENV_PY" simulation/mqtt_publisher.py --scenario normal --interval 1.0 > /dev/null 2>&1 &
PID_PUB=$!
echo "✓ Synthetic Generator started (PID: $PID_PUB)"

# Wait 2 seconds for services to establish loopback
sleep 2

# 3. Launch Streamlit Industrial Monitoring Portal
echo "Launching Streamlit Monitoring Dashboard at http://localhost:8501..."
echo "Sign in using: [REDACTED] / [REDACTED] or operator / Operator@2026"
echo "Press Ctrl+C to stop all services."
echo "================================================================================"

"$VENV_STREAMLIT" run dashboard/app.py \
    --server.port=8501 \
    --server.address=0.0.0.0 \
    --browser.gatherUsageStats=false
```

scripts/run_layer1.sh

```
#!/usr/bin/env bash
# ============================================================================
# LAYER 1: Synthetic Telemetry -> MQTT -> CSV Logger -> Streamlit Dashboard
# Verifies basic simulation, publishing, CSV append, and live visualization.
# ============================================================================

set -e

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"

cd "$PROJECT_ROOT"

VENV_PY="$PROJECT_ROOT/.venv/bin/python"
VENV_STREAMLIT="$PROJECT_ROOT/.venv/bin/streamlit"

echo "============================================================================"
echo "⚡ RUNNING LAYER 1: SYNTHETIC DATA -> MQTT -> CSV -> DASHBOARD"
echo "============================================================================"

# Verify simulation scenario generation
echo "Running Layer 1 unit tests..."
"$PROJECT_ROOT/.venv/bin/pytest" tests/test_simulation.py -v

echo -e "\nStarting Layer 1 Ingestion (CSV Logging)..."
PID_SUB=""
PID_PUB=""

cleanup() {
    echo -e "\nStopping Layer 1 test services..."
    [ -n "$PID_PUB" ] && kill -TERM "$PID_PUB" 2>/dev/null || true
    [ -n "$PID_SUB" ] && kill -TERM "$PID_SUB" 2>/dev/null || true
    exit 0
}
trap cleanup SIGINT SIGTERM EXIT

# Start subscriber writing to CSV
"$VENV_PY" ingestion/mqtt_subscriber.py &
PID_SUB=$!

# Start publisher with mechanical overload scenario
"$VENV_PY" simulation/mqtt_publisher.py --scenario overload --interval 0.5 --count 20 &
PID_PUB=$!

wait "$PID_PUB"
echo "Published 20 sample readings to CSV: data/csv/sensor_telemetry.csv"

# Show CSV contents
head -n 5 data/csv/sensor_telemetry.csv

echo "Launching Streamlit dashboard to inspect CSV telemetry..."
"$VENV_STREAMLIT" run dashboard/app.py --server.port=8501
```

scripts/run_layer2.sh

```
#!/usr/bin/env bash
# ============================================================================
# LAYER 2: Cloud MQTT Integration (HiveMQ Cloud TLS 8883) with Local Fallback
# Tests TLS connection, credentials, and message routing.
# ============================================================================

set -e

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"

cd "$PROJECT_ROOT"

VENV_PY="$PROJECT_ROOT/.venv/bin/python"

echo "============================================================================"
echo "⚡ RUNNING LAYER 2: HIVEMQ CLOUD TLS (8883) & FALLBACK BROKER"
echo "============================================================================"

# Execute MQTT test suite
echo "Running Layer 2 MQTT & fallback tests..."
"$PROJECT_ROOT/.venv/bin/pytest" tests/test_mqtt.py -v

# Inspect configured MQTT credentials
echo -e "\nChecking Broker Configuration:"
"$VENV_PY" -c "
from config.settings import settings
cfg = settings.get_effective_mqtt_config()
print('Broker Type:', cfg['broker_type'])
print('Host:       ', cfg['host'])
print('Port:       ', cfg['port'])
print('TLS Enabled:', cfg.get('use_tls', False))
print('HiveMQ Cloud Configured:', settings.is_hivemq_configured)
if not settings.is_hivemq_configured:
    print('Notice: Operating in Local Broker Fallback mode. To connect to HiveMQ Cloud, update .env.')
"
echo "============================================================================"
```

scripts/run_layer3.sh

```
#!/usr/bin/env bash
# ================================================================================
# LAYER 3: SQLite Persistent Historical Database & Queries
# Verifies migrations, deduplication, historical time-range queries, and WAL mode.
# ================================================================================

set -e

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"

cd "$PROJECT_ROOT"

VENV_PY="$PROJECT_ROOT/.venv/bin/python"

echo "================================================================================"
echo "⚡ RUNNING LAYER 3: SQLITE PERSISTENT DATABASE & HISTORICAL ANALYTICS"
echo "================================================================================"

# Apply migrations
echo "Executing database migrations..."
"$VENV_PY" database/migrations/migrate.py

# Run Database test suite
echo -e "\nRunning Layer 3 Database tests..."
"$PROJECT_ROOT/.venv/bin/pytest" tests/test_database.py -v

# Inspect Database state
echo -e "\nDatabase Statistics Summary:"
"$VENV_PY" -c "
from database.repository import repo
readings = repo.get_recent_readings('motor01', limit=10)
alerts = repo.get_alerts('motor01', limit=5)
preds = repo.get_recent_predictions('motor01', limit=5)
print(f'Total Sensor Readings in DB:    {len(readings)}')
print(f'Total Active/Logged Alerts:    {len(alerts)}')
print(f'Total Hybrid Predictions in DB: {len(preds)}')
"
echo "================================================================================"
```

scripts/run_layer4.sh

```
#!/usr/bin/env bash
# ================================================================================
# LAYER 4: Node-RED Industrial Automation, Debounce Hysteresis & Alerts
# Tests hysteresis threshold logic, alert cooldown, and recovery notifications.
# ================================================================================

set -e

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"

cd "$PROJECT_ROOT"

echo "================================================================================"
echo "⚡ RUNNING LAYER 4: NODE-RED INDUSTRIAL AUTOMATION & ALERT HYSTERESIS"
echo "================================================================================"

# Run alert debounce and persistence tests
echo "Running Layer 4 Alert Debouncing & Recovery Unit Tests..."
"$PROJECT_ROOT/.venv/bin/pytest" tests/test_alerts.py -v

echo -e "\nNode-RED Flow Verification:"
echo "Flow File: $PROJECT_ROOT/nodered/flows.json"
if [ -f "$PROJECT_ROOT/nodered/flows.json" ]; then
    echo "✓ Node-RED flow file exists and is validated."
fi

echo -e "\nTo launch Node-RED via Docker Compose on macOS Apple Silicon:"
echo "  docker compose up nodered -d"
echo "  Access web interface at: http://localhost:1880"
echo "================================================================================"
```

scripts/run_layer5.sh

```
#!/usr/bin/env bash
# ============================================================================
# LAYER 5: Knowledge-Driven AI/ML Expert Rules, Login & Multi-Page Dashboard
# Runs rule evaluation, ML inference metrics, and launches backend API + Dashboard.
# ============================================================================

set -e

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"

cd "$PROJECT_ROOT"

VENV_PY="$PROJECT_ROOT/.venv/bin/python"
VENV_STREAMLIT="$PROJECT_ROOT/.venv/bin/streamlit"

echo "============================================================================"
echo "⚡ RUNNING LAYER 5: KNOWLEDGE-DRIVEN AI/ML, AUTHENTICATION & DASHBOARD"
echo "============================================================================"

# Run test suites for rules, ML, and API
echo "Running Layer 5 comprehensive test suites..."
"$PROJECT_ROOT/.venv/bin/pytest" tests/test_rules.py tests/test_ml.py tests/test_api.py -v

# Run Model Evaluation
echo -e "\nEvaluating Machine Learning Model:"
"$VENV_PY" ml/evaluate.py

echo -e "\nStarting FastAPI Backend Service (Port 8000)..."
PID_API=""
PID_DASH=""

cleanup() {
    echo -e "\nStopping Layer 5 services..."
    [ -n "$PID_API" ] && kill -TERM "$PID_API" 2>/dev/null || true
    [ -n "$PID_DASH" ] && kill -TERM "$PID_DASH" 2>/dev/null || true
    exit 0
}
trap cleanup SIGINT SIGTERM EXIT

"$VENV_PY" -m uvicorn backend.main:app --host 0.0.0.0 --port 8000 --log-level warning &
PID_API=$!

sleep 2

echo "Launching Streamlit Industrial Diagnostic Dashboard (Port 8501)..."
"$VENV_STREAMLIT" run dashboard/app.py --server.port=8501
```

scripts/run_layer6.sh

```
#!/usr/bin/env bash
# ================================================================================
# LAYER 6: ESP32 Hardware Integration & Sensor Calibration Mode
# Checks USB serial connectivity, verifies pinouts, and switches system to real hardware.
# ================================================================================

set -e

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"

cd "$PROJECT_ROOT"

VENV_PY="$PROJECT_ROOT/.venv/bin/python"

echo "================================================================================"
echo "⚡ RUNNING LAYER 6: ESP32 HARDWARE INTEGRATION & BENCH VALIDATION"
echo "================================================================================"

echo "1. Checking connected USB Serial devices on macOS..."
SERIAL_DEVICES=$(ls /dev/cu.usbserial* /dev/cu.wchusbserial* 2>/dev/null || true)

if [ -z "$SERIAL_DEVICES" ]; then
    echo "⚠️  No physical ESP32 USB serial device detected."
    echo "   Ensure your ESP32 DevKit is plugged in via a data-capable MicroUSB/USB-C cable."
else
    echo "✓ Found ESP32 serial port(s):"
    echo "$SERIAL_DEVICES"
fi

echo -e "\n2. Electrical & Safety Pin Verification Checklist:"
echo "   [ ] MPU6050 SDA connected to GPIO 21, SCL to GPIO 22 (Powered from 3.3V)"
echo "   [ ] DS18B20 DQ connected to GPIO 4 with 4.7kΩ pull-up to 3.3V"
echo "   [ ] ACS712 output connected to GPIO 36 via 10kΩ/20kΩ voltage divider"
echo "   [ ] IR Sensor connected to GPIO 18 (Interrupt pin)"
echo "   [ ] Alert Warning LED on GPIO 2 (with 330Ω resistor to GND)"
echo "   [ ] Alert Piezo Buzzer on GPIO 15 (transistor driven)"
echo "   [ ] 12V DC Motor powered from external 12V 2A supply (COMMON GROUND to ESP32)"

echo -e "\n3. Firmware Compilation & Flashing Instructions:"
echo "   cd firmware/esp32_motor_monitor"
echo "   pio run --target upload"
echo "   pio device monitor -b 115200"

echo -e "\n4. Starting Backend Telemetry Ingestion (Ready for ESP32 MQTT Packets)..."
"$VENV_PY" ingestion/mqtt_subscriber.py
```

scripts/setup.sh

```
#!/usr/bin/env bash
# ================================================================================
# Setup Script for Knowledge-Driven IoT Fault Diagnosis System
# Compatible with macOS Apple Silicon (arm64) and Linux
# ================================================================================

set -e

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"

cd "$PROJECT_ROOT"

echo "================================================================================"
echo "⚡ Setting up Knowledge-Driven IoT Fault Diagnosis System"
echo "Project Root: $PROJECT_ROOT"
echo "================================================================================"

# 1. Verify Python Version
PYTHON_BIN="python3"
if ! command -v "$PYTHON_BIN" &>/dev/null; then
    echo "□ Error: python3 not found in PATH."
    exit 1
fi

PY_VER=$($PYTHON_BIN -c "import sys; print(f'{sys.version_info.major}.{sys.version_info.minor}')")
echo "✓ Detected Python version: $PY_VER"

# 2. Virtual Environment Setup
VENV_DIR="$PROJECT_ROOT/.venv"
if [ ! -d "$VENV_DIR" ]; then
    echo "Creating virtual environment at $VENV_DIR..."
    $PYTHON_BIN -m venv "$VENV_DIR"
fi

VENV_PY="$VENV_DIR/bin/python"
VENV_PIP="$VENV_DIR/bin/pip"

# 3. Upgrade pip and install dependencies
echo "Installing project requirements..."
"$VENV_PIP" install --quiet --upgrade pip setuptools wheel
"$VENV_PIP" install --quiet -r requirements.txt

# 4. Create necessary storage folders
echo "Creating required directory structure..."
mkdir -p data/csv data/sqlite data/synthetic ml/models docs/assets

# 5. Initialize .env if missing
if [ ! -f ".env" ] && [ -f ".env.example" ]; then
    echo "Creating .env from .env.example..."
    cp .env.example .env
fi

# 6. Database Migrations & Initial Setup
echo "Applying database schema and migrations..."
"$VENV_PY" database/migrations/migrate.py

# 7. Seed Default Admin & Operator Accounts
echo "Seeding default security credentials..."
"$VENV_PY" scripts/init_[REDACTED].py

# 8. Train Machine Learning Model if Missing
if [ ! -f "ml/models/random_forest_model.joblib" ]; then
    echo "Training initial Random Forest fault diagnosis model..."
    "$VENV_PY" ml/train.py
else
    echo "✓ Trained machine learning model bundle already present."
fi

# 9. Verify Installation with Test Suite
echo "Running validation unit tests..."
```

scripts/setup.sh (continued)

```
"$VENV_DIR/bin/pytest" tests -q

echo "================================================================================"
echo "✅ Setup successfully completed!"
echo "Run './scripts/run_demo.sh' to launch the one-command local demonstration."
echo "================================================================================"
```

simulation/mqtt_publisher.py

```
"""
MQTT Publisher Service for Synthetic DC Motor Telemetry
Publishes normalized sensor data, ground truth labels, and device heartbeats.
"""

import sys
import time
import signal
import argparse
import logging
from pathlib import Path

# Add project root to sys.path
sys.path.insert(0, str(Path(__file__).resolve().parent.parent))

from config.settings import settings
from simulation.scenarios import MotorScenario
from simulation.synthetic_generator import SyntheticGenerator
from ingestion.mqtt_client import ManagedMQTTClient

logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s] %(name)s: %(message)s")
logger = logging.getLogger("mqtt_publisher")


class MotorMQTTPublisher:
    """
    Simulates and publishes motor telemetry via MQTT to either local Mosquitto or HiveMQ Cloud.
    """

    def __init__(
        self,
        device_id: str = "motor01",
        scenario: MotorScenario = MotorScenario.NORMAL,
        publish_interval: float = 1.0,
        seed: int = 42,
    ):
        self.device_id = device_id
        self.publish_interval = publish_interval
        self.generator = SyntheticGenerator(device_id=device_id, seed=seed)
        self.generator.set_scenario(scenario)
        self.running = False

        # Load effective MQTT config
        self.mqtt_config = settings.get_effective_mqtt_config()
        self.client = ManagedMQTTClient(
            client_id=f"sim_publisher_{device_id}_{int(time.time())}",
            config=self.mqtt_config,
            on_message_callback=self._handle_command,
        )

        self.sensor_topic = settings.get_sensor_topic(device_id)
        self.status_topic = settings.get_status_topic(device_id)
        self.ground_truth_topic = settings.get_ground_truth_topic(device_id)
        self.commands_topic = settings.get_commands_topic(device_id)

    def _handle_command(self, topic: str, payload: dict) -> None:
        """Processes remote commands sent on iot/factory/motor01/commands."""
        cmd = payload.get("command")
        logger.info("Received remote command on %s: %s", topic, cmd)
        if cmd == "set_scenario":
            scen_name = payload.get("scenario")
            try:
                self.generator.set_scenario(MotorScenario(scen_name))
                logger.info("Switched simulation scenario to: %s", scen_name)
            except Exception as exc:
                logger.error("Failed to set scenario '%s': %s", scen_name, exc)
        elif cmd == "stop":
            self.stop()

    def start(self, duration: int = 0) -> None:
        """
```

simulation/mqtt_publisher.py (continued)

```
        Starts the publishing loop.
        duration: 0 runs indefinitely until SIGINT. >0 runs for N seconds.
        """
        logger.info("Connecting to MQTT (%s at %s:%s)...",
                    self.mqtt_config["broker_type"], self.mqtt_config["host"], self.mqtt_config["port"])
        if not self.client.connect(timeout=6):
            logger.warning("Could not establish initial MQTT connection. Will retry in loop...")

        # Subscribe to commands
        self.client.subscribe(self.commands_topic)
        self.running = True
        start_time = time.time()
        sample_count = 0

        logger.info("Simulation Publisher started for device '%s' (Scenario: %s)",
                    self.device_id, self.generator.current_scenario.value)

        try:
            while self.running:
                now = time.time()
                if duration > 0 and (now - start_time) >= duration:
                    logger.info("Completed configured simulation duration of %s seconds.", duration)
                    break

                sensor_data, ground_truth = self.generator.generate_next_sample()

                if sensor_data:
                    # Publish normalized sensor telemetry
                    self.client.publish(self.sensor_topic, sensor_data)
                    sample_count += 1
                    if sample_count % 5 == 0:
                        logger.info("Published #%s: T=%.1f°C | V=%.3fg | I=%.3fA | RPM=%.0f",
                                    sample_count,
                                    sensor_data["temperature_c"],
                                    sensor_data["vibration_rms_g"],
                                    sensor_data["current_a"],
                                    sensor_data["rpm"])

                # Always publish ground truth (separated from sensor reading)
                self.client.publish(self.ground_truth_topic, ground_truth)

                # Periodic heartbeat on status topic every 5 seconds
                if sample_count % 5 == 0:
                    status_payload = {
                        "device_id": self.device_id,
                        "uptime_seconds": int(now - start_time),
                        "status": "online" if sensor_data else "disconnected",
                        "broker": self.mqtt_config["broker_type"],
                        "timestamp": ground_truth["timestamp"],
                    }
                    self.client.publish(self.status_topic, status_payload)

                time.sleep(self.publish_interval)

        except KeyboardInterrupt:
            logger.info("Publisher stopped by keyboard interrupt.")
        finally:
            self.stop()

    def stop(self) -> None:
        self.running = False
        self.client.disconnect()
        logger.info("Publisher cleanly terminated.")


def main():
    parser = argparse.ArgumentParser(description="Synthetic Motor Telemetry MQTT Publisher")
    parser.add_argument("--device-id", default="motor01", help="Device ID")
    parser.add_argument("--scenario", default="normal",
                        choices=[s.value for s in MotorScenario], help="Motor scenario")
    parser.add_argument("--interval", type=float, default=1.0, help="Publish interval in seconds")
```

simulation/mqtt_publisher.py (continued) (continued)

```
    parser.add_argument("--duration", type=int, default=0, help="Duration in seconds (0 = infinite)")
    parser.add_argument("--seed", type=int, default=42, help="Random seed for reproducibility")
    args = parser.parse_args()

    pub = MotorMQTTPublisher(
        device_id=args.device_id,
        scenario=MotorScenario(args.scenario),
        publish_interval=args.interval,
        seed=args.seed,
    )

    def sig_handler(sig, frame):
        logger.info("Termination signal received.")
        pub.stop()
        sys.exit(0)

    signal.signal(signal.SIGINT, sig_handler)
    signal.signal(signal.SIGTERM, sig_handler)

    pub.start(duration=args.duration)


if __name__ == "__main__":
    main()

```

simulation/scenarios.py

```
"""
Simulation Scenarios for 12V Industrial DC Motor
Defines physics-informed sensor generation parameters for nominal and fault modes.
"""

from enum import Enum
from typing import Dict, Any


class MotorScenario(str, Enum):
    NORMAL = "normal"
    MECHANICAL_OVERLOAD = "mechanical_overload"
    EXCESSIVE_VIBRATION = "excessive_vibration"
    MOTOR_OVERHEATING = "motor_overheating"
    MOTOR_SLOWDOWN = "motor_slowdown"
    MOTOR_STALL = "motor_stall"
    SENSOR_ERROR = "sensor_error"
    DISCONNECTION = "disconnection"


# Physical parameter baselines and drift behaviors for each scenario
SCENARIO_PROFILES: Dict[MotorScenario, Dict[str, Any]] = {
    MotorScenario.NORMAL: {
        "name": "Normal Motor Operation",
        "description": "Rated 12V operation with healthy bearings, nominal mechanical load, and stable thermal dissipation.",
        "temp_base": 36.5,
        "temp_std": 0.8,
        "temp_drift": 0.01,
        "vibe_base": 0.12,
        "vibe_std": 0.02,
        "current_base": 0.58,
        "current_std": 0.03,
        "rpm_base": 2150.0,
        "rpm_std": 30.0,
        "sensor_status": "ok",
        "severity": "NORMAL",
        "ground_truth_label": "Normal",
    },
    MotorScenario.MECHANICAL_OVERLOAD: {
        "name": "Mechanical Overload",
        "description": "Excessive driven mechanical resistance or jammed conveyor causes elevated current draw and depressed speed.",
        "temp_base": 48.0,
        "temp_std": 1.2,
        "temp_drift": 0.15,
        "vibe_base": 0.28,
        "vibe_std": 0.04,
        "current_base": 1.25,
        "current_std": 0.06,
        "rpm_base": 1380.0,
        "rpm_std": 45.0,
        "sensor_status": "ok",
        "severity": "WARNING",
        "ground_truth_label": "Mechanical Overload",
    },
    MotorScenario.EXCESSIVE_VIBRATION: {
        "name": "Excessive Vibration",
        "description": "Loose base mounting bolts, structural resonance, or rotor dynamic unbalance producing high acceleration.",
        "temp_base": 40.0,
        "temp_std": 1.0,
        "temp_drift": 0.02,
        "vibe_base": 0.55,
        "vibe_std": 0.08,
        "current_base": 0.65,
        "current_std": 0.04,
        "rpm_base": 2080.0,
        "rpm_std": 40.0,
        "sensor_status": "ok",
        "severity": "WARNING",
```

simulation/scenarios.py (continued)

```
        "ground_truth_label": "Excessive Vibration",
    },
    MotorScenario.MOTOR_OVERHEATING: {
        "name": "Motor Overheating",
        "description": "Cooling ventilation blockage or sustained thermal stress driving casing temperature above 65°C.",
        "temp_base": 68.0,
        "temp_std": 1.5,
        "temp_drift": 0.30,  # Progressive thermal accumulation
        "vibe_base": 0.22,
        "vibe_std": 0.03,
        "current_base": 1.10,
        "current_std": 0.05,
        "rpm_base": 1820.0,
        "rpm_std": 35.0,
        "sensor_status": "ok",
        "severity": "CRITICAL",
        "ground_truth_label": "Motor Overheating",
    },
    MotorScenario.MOTOR_SLOWDOWN: {
        "name": "Motor Slowdown",
        "description": "DC power rail voltage sag (e.g. dropping to 8.5V) or dry bearing friction leading to reduced RPM.",
        "temp_base": 35.0,
        "temp_std": 0.7,
        "temp_drift": 0.0,
        "vibe_base": 0.14,
        "vibe_std": 0.02,
        "current_base": 0.62,
        "current_std": 0.03,
        "rpm_base": 1220.0,
        "rpm_std": 30.0,
        "sensor_status": "ok",
        "severity": "WARNING",
        "ground_truth_label": "Motor Slowdown",
    },
    MotorScenario.MOTOR_STALL: {
        "name": "Motor Stall / Locked Rotor",
        "description": "Physical obstruction locking the motor shaft. Current surges near stall rating (2.0A) while RPM collapses.",
        "temp_base": 62.0,
        "temp_std": 2.0,
        "temp_drift": 0.50,
        "vibe_base": 0.05,  # Stalled motor doesn't rotate, low rotary vibration
        "vibe_std": 0.02,
        "current_base": 1.95,
        "current_std": 0.08,
        "rpm_base": 40.0,
        "rpm_std": 15.0,
        "sensor_status": "ok",
        "severity": "CRITICAL",
        "ground_truth_label": "Motor Stall",
    },
    MotorScenario.SENSOR_ERROR: {
        "name": "Intermittent Sensor Error",
        "description": "Loose wire, floating ADC pin, or sensor communication timeout injecting out-of-range artifacts.",
        "temp_base": -99.0,  # Impossible physical reading
        "temp_std": 0.0,
        "temp_drift": 0.0,
        "vibe_base": 15.8,
        "vibe_std": 2.0,
        "current_base": -1.5,
        "current_std": 0.2,
        "rpm_base": 8888.0,
        "rpm_std": 100.0,
        "sensor_status": "error",
        "severity": "UNKNOWN / SENSOR ERROR",
        "ground_truth_label": "Sensor Error",
    },
    MotorScenario.DISCONNECTION: {
```

simulation/scenarios.py (continued) (continued)

```
136          "name": "MQTT Disconnection / Telemetry Loss",
137          "description": "Network drop or power interruption preventing telemetry publishing. Leads to
     stale/offline detection.",
138          "temp_base": 0.0,
139          "temp_std": 0.0,
140          "temp_drift": 0.0,
141          "vibe_base": 0.0,
142          "vibe_std": 0.0,
143          "current_base": 0.0,
144          "current_std": 0.0,
145          "rpm_base": 0.0,
146          "rpm_std": 0.0,
147          "sensor_status": "disconnected",
148          "severity": "WARNING",
149          "ground_truth_label": "Disconnection",
150      },
151  }
152
```

simulation/synthetic_generator.py

```
"""
Synthetic Sensor Data Generator for Industrial 12V DC Motor
Generates physically realistic telemetry with continuous drift, noise, and fault dynamics.
"""

import uuid
import random
from datetime import datetime, timezone
from typing import Dict, Any, Tuple, Optional
from simulation.scenarios import MotorScenario, SCENARIO_PROFILES


class SyntheticGenerator:
    """
    Simulates physical dynamics of a 12V brushed DC motor under nominal and fault modes.
    Ensures temporal continuity (thermal inertia, rotational momentum).
    """

    def __init__(self, device_id: str = "motor01", seed: Optional[int] = None):
        self.device_id = device_id
        self.seed = seed
        self.rng = random.Random(seed)
        self.current_scenario = MotorScenario.NORMAL
        self.step_count = 0

        # State tracking for continuous physical inertia
        profile = SCENARIO_PROFILES[self.current_scenario]
        self._temp = profile["temp_base"]
        self._rpm = profile["rpm_base"]
        self._current = profile["current_base"]
        self._vibe = profile["vibe_base"]

    def set_scenario(self, scenario: MotorScenario) -> None:
        """Dynamically switch the operating scenario."""
        if isinstance(scenario, str):
            scenario = MotorScenario(scenario)
        self.current_scenario = scenario
        self.step_count = 0

    def generate_next_sample(self) -> Tuple[Dict[str, Any], Dict[str, Any]]:
        """
        Advances simulation by one timestep (nominally 1 second) and returns:
        (sensor_reading, ground_truth)
        """
        self.step_count += 1
        now_utc = datetime.now(timezone.utc).isoformat()
        profile = SCENARIO_PROFILES[self.current_scenario]

        if self.current_scenario == MotorScenario.DISCONNECTION:
            ground_truth = {
                "event_id": str(uuid.uuid4()),
                "device_id": self.device_id,
                "timestamp": now_utc,
                "scenario": self.current_scenario.value,
                "ground_truth_label": profile["ground_truth_label"],
                "severity": profile["severity"],
                "is_disconnected": True,
            }
            # Disconnection emits no sensor telemetry to MQTT
            return {}, ground_truth

        if self.current_scenario == MotorScenario.SENSOR_ERROR:
            # Inject erratic glitch or null/extreme reading
            err_type = self.rng.choice(["negative_temp", "adc_spike", "hall_glitch", "status_flag"])
            temp = -99.9 if err_type == "negative_temp" else round(self.rng.gauss(38.0, 1.0), 2)
            vibe = round(self.rng.uniform(12.0, 18.0), 3) if err_type == "adc_spike" else
round(self.rng.gauss(0.15, 0.05), 3)
            current = -2.5 if err_type == "adc_spike" else round(self.rng.gauss(0.6, 0.05), 3)
            rpm = 9999.0 if err_type == "hall_glitch" else round(self.rng.gauss(2100.0, 20.0), 1)
            sensor_status = "error"
        else:
```

simulation/synthetic_generator.py (continued)

```
            sensor_status = profile["sensor_status"]
            # Smooth inertia tracking towards target profile
            target_temp = profile["temp_base"] + (self.step_count * profile["temp_drift"])
            target_rpm = profile["rpm_base"]
            target_current = profile["current_base"]
            target_vibe = profile["vibe_base"]

            # Physical time constants:
            # - Stall: electrical time constant is near-instantaneous (0.85 alpha)
            # - Normal/thermal: gradual damping (0.15 - 0.35 alpha)
            alpha_curr = 0.85 if self.current_scenario == MotorScenario.MOTOR_STALL else 0.30
            alpha_rpm = 0.80 if self.current_scenario == MotorScenario.MOTOR_STALL else 0.35

            self._temp += 0.15 * (target_temp - self._temp) + self.rng.gauss(0, profile["temp_std"] * 0.2)
            self._rpm += alpha_rpm * (target_rpm - self._rpm) + self.rng.gauss(0, profile["rpm_std"] * 0.3)
            self._current += alpha_curr * (target_current - self._current) + self.rng.gauss(0,
    profile["current_std"] * 0.25)
            self._vibe += 0.25 * (target_vibe - self._vibe) + self.rng.gauss(0, profile["vibe_std"] * 0.2)

            # Prevent unphysical negative values in nominal/overload modes
            temp = round(max(0.0, self._temp), 2)
            vibe = round(max(0.01, self._vibe), 3)
            current = round(max(0.0, self._current), 3)
            rpm = round(max(0.0, self._rpm), 1)

        event_id = str(uuid.uuid4())

        # Normalized sensor reading payload (Section 2 specification)
        sensor_reading = {
            "event_id": event_id,
            "device_id": self.device_id,
            "timestamp": now_utc,
            "source": "synthetic",
            "temperature_c": temp,
            "vibration_rms_g": vibe,
            "current_a": current,
            "rpm": rpm,
            "sensor_status": sensor_status,
        }

        # Ground truth label stored independently for validation / ML benchmarking
        ground_truth = {
            "event_id": event_id,
            "device_id": self.device_id,
            "timestamp": now_utc,
            "scenario": self.current_scenario.value,
            "ground_truth_label": profile["ground_truth_label"],
            "severity": profile["severity"],
            "is_disconnected": False,
        }

        return sensor_reading, ground_truth
```

tests/test_alerts.py

```python
"""
Unit Tests for Alert Persistence, Cooldown, and Recovery.
"""

import time
import pytest
from ingestion.mqtt_subscriber import TelemetryIngestionService


def test_alert_persistence_debouncing():
    service = TelemetryIngestionService(device_id="motor01")
    service.consecutive_threshold = 3
    service.alert_cooldown_sec = 10.0

    # Simulate warning scenario (Mechanical Overload)
    diag_warning = {
        "hybrid_severity": "WARNING",
        "hybrid_fault": "Mechanical Overload",
        "hybrid_confidence": 0.88,
        "recommended_action": "Reduce conveyor load",
    }
    reading = {
        "event_id": "test-alert-evt",
        "device_id": "motor01",
        "timestamp": "2026-10-30T11:00:00Z",
        "temperature_c": 45.0,
        "vibration_rms_g": 0.25,
        "current_a": 1.25,
        "rpm": 1400.0,
        "sensor_status": "ok",
    }

    # Reading 1: counter = 1 -> no alert dispatched
    service._evaluate_alerts("motor01", diag_warning, reading)
    assert service.fault_counters["motor01"] == 1
    assert "motor01" not in service.last_alert_time

    # Reading 2: counter = 2 -> no alert dispatched
    service._evaluate_alerts("motor01", diag_warning, reading)
    assert service.fault_counters["motor01"] == 2
    assert "motor01" not in service.last_alert_time

    # Reading 3: counter = 3 -> consecutive threshold met -> alert dispatched!
    service._evaluate_alerts("motor01", diag_warning, reading)
    assert service.fault_counters["motor01"] == 3
    assert "motor01" in service.last_alert_time

    # Immediate Reading 4: within cooldown window -> suppressed!
    t_alert = service.last_alert_time["motor01"]
    service._evaluate_alerts("motor01", diag_warning, reading)
    assert service.last_alert_time["motor01"] == t_alert  # alert time unchanged (suppressed)

    # Recovery reading (NORMAL) -> resets fault counter to 0
    diag_normal = {"hybrid_severity": "NORMAL", "hybrid_fault": "Normal"}
    service._evaluate_alerts("motor01", diag_normal, reading)
    assert service.fault_counters["motor01"] == 0

```

tests/test_api.py

```python
"""
Unit Tests for FastAPI REST Endpoints and Security.
"""

import pytest
from fastapi.testclient import TestClient
from backend.main import app
from backend.auth.security import auth_manager


client = TestClient(app)


def test_health_endpoint():
    response = client.get("/api/health")
    assert response.status_code == 200
    data = response.json()
    assert data["status"] == "healthy"
    assert "mqtt_broker" in data
    assert "database" in data


def test_auth_and_protected_flow():
    # Login with valid [REDACTED] credentials
    res = client.post("/api/auth/login", json={"username": "[REDACTED]", "password": "[REDACTED]"})
    assert res.status_code == 200
    token = res.json()["token"]
    assert token

    # Protected alert ack endpoint
    headers = {"Authorization": f"Bearer {token}"}
    ack_res = client.post("/api/alerts/999999/ack", json={"username": "[REDACTED]"}, headers=headers)
    # Alert 999999 doesn't exist, should return 404
    assert ack_res.status_code == 404

    # Unauthenticated request should fail with 401
    fail_res = client.post("/api/alerts/1/ack", json={"username": "[REDACTED]"})
    assert fail_res.status_code == 401


def test_model_metrics_endpoint():
    res = client.get("/api/model/metrics")
    assert res.status_code == 200
    data = res.json()
    assert "metrics" in data
    assert "confusion_matrix" in data

```

tests/test_database.py

```python
"""
Unit Tests for SQLite Persistence and Repository.
"""

import uuid
from datetime import datetime, timezone
import pytest
from database.repository import TelemetryRepository
from database.connection import DatabaseManager


@pytest.fixture
def test_repo(tmp_path):
    """Provides an isolated test repository in temporary folder."""
    db_file = tmp_path / "test_telemetry.db"
    manager = DatabaseManager(db_path=db_file)
    return TelemetryRepository(manager=manager)


def test_insert_and_retrieve_reading(test_repo):
    event_id = str(uuid.uuid4())
    reading = {
        "event_id": event_id,
        "device_id": "motor01",
        "timestamp": "2026-10-30T10:00:00Z",
        "source": "synthetic",
        "temperature_c": 35.5,
        "vibration_rms_g": 0.15,
        "current_a": 0.55,
        "rpm": 2150.0,
        "sensor_status": "ok",
    }
    inserted = test_repo.insert_sensor_reading(reading)
    assert inserted is True

    # Duplicate check
    inserted_dup = test_repo.insert_sensor_reading(reading)
    assert inserted_dup is False

    # Retrieve
    df = test_repo.get_recent_readings(device_id="motor01", limit=10)
    assert len(df) == 1
    assert df.iloc[0]["event_id"] == event_id
    assert df.iloc[0]["temperature_c"] == 35.5


def test_alert_lifecycle(test_repo):
    alert_id = test_repo.insert_alert(
        device_id="motor01",
        severity="CRITICAL",
        title="Motor Stall Detected",
        message="Current at 2.1A, RPM at 0.",
    )
    assert alert_id > 0

    alerts = test_repo.get_alerts(device_id="motor01", only_unacknowledged=True)
    assert len(alerts) == 1
    assert alerts.iloc[0]["acknowledged"] == 0

    # Acknowledge alert
    ok = test_repo.acknowledge_alert(alert_id, user="test_engineer")
    assert ok is True

    # Should no longer appear in unacknowledged list
    unack = test_repo.get_alerts(device_id="motor01", only_unacknowledged=True)
    assert len(unack) == 0

```

tests/test_ml.py

```python
"""
Unit Tests for Machine Learning Feature Extraction, Inference, and Hybrid Diagnosis.
"""

import pytest
import numpy as np
from ml.features import extract_single_reading_features, FEATURE_NAMES
from ml.predict import FaultPredictor
from diagnosis.hybrid_engine import HybridDiagnosticEngine


def test_feature_extraction():
    reading = {
        "temperature_c": 40.0,
        "vibration_rms_g": 0.20,
        "current_a": 0.80,
        "rpm": 1900.0,
    }
    history = [
        {"temperature_c": 39.5, "vibration_rms_g": 0.18, "current_a": 0.75, "rpm": 1950.0}
    ]
    feat = extract_single_reading_features(reading, history)
    assert feat.shape == (1, len(FEATURE_NAMES))
    assert feat[0, 0] == 40.0  # temperature
    assert round(feat[0, 4], 2) == 0.50  # temp_trend = 40.0 - 39.5


def test_ml_predictor_inference():
    predictor = FaultPredictor()
    assert predictor.is_loaded is True

    normal_reading = {
        "temperature_c": 36.0,
        "vibration_rms_g": 0.12,
        "current_a": 0.60,
        "rpm": 2150.0,
        "sensor_status": "ok",
    }
    pred_cls, conf, probs = predictor.predict(normal_reading)
    assert pred_cls == "Normal"
    assert conf >= 0.80
    assert "Normal" in probs


def test_hybrid_engine_consensus():
    engine = HybridDiagnosticEngine()
    reading = {
        "device_id": "motor01",
        "event_id": "evt-001",
        "timestamp": "2026-10-30T11:00:00Z",
        "temperature_c": 36.0,
        "vibration_rms_g": 0.12,
        "current_a": 0.60,
        "rpm": 2150.0,
        "sensor_status": "ok",
    }
    diag = engine.diagnose(reading)
    assert diag["hybrid_fault"] == "Normal"
    assert diag["hybrid_severity"] == "NORMAL"
    assert diag["conflict_detected"] is False
    assert diag["hybrid_confidence"] >= 0.90

```

tests/test_mqtt.py

```python
"""
Tests for MQTT Configuration, Client Abstraction, and Local Fallback.
"""

import time
import pytest
from config.settings import settings
from ingestion.mqtt_client import ManagedMQTTClient
from ingestion.data_validator import DataValidator


def test_effective_mqtt_config_fallback():
    """Verify fallback to local broker when HiveMQ credentials are blank."""
    cfg = settings.get_effective_mqtt_config()
    assert cfg["broker_type"] in ("local_fallback", "hivemq_cloud")
    assert "host" in cfg
    assert "port" in cfg
    assert "use_tls" in cfg


def test_data_validator_deduplication():
    """Verify duplicate event IDs are blocked."""
    validator = DataValidator(seen_cache_size=100)
    reading = {
        "event_id": "test-unique-uuid-12345",
        "device_id": "motor01",
        "timestamp": "2026-10-30T11:00:00Z",
        "source": "synthetic",
        "temperature_c": 38.0,
        "vibration_rms_g": 0.12,
        "current_a": 0.60,
        "rpm": 2100.0,
        "sensor_status": "ok",
    }
    is_valid, err = validator.validate_sensor_payload(reading)
    assert is_valid is True

    # Immediate duplicate
    is_dup, err_dup = validator.validate_sensor_payload(reading)
    assert is_dup is False
    assert "Duplicate" in err_dup


def test_local_mqtt_pub_sub_loopback():
    """Test publishing and subscribing on local MQTT broker (Mosquitto port 1883)."""
    received_messages = []

    def on_msg(topic, payload):
        received_messages.append((topic, payload))

    client = ManagedMQTTClient(
        client_id="test_sub_loopback",
        config=settings.get_effective_mqtt_config(),
        on_message_callback=on_msg,
    )

    if not client.connect(timeout=3):
        pytest.skip("Local MQTT broker (port 1883) not reachable in test environment.")

    test_topic = "iot/test/loopback/ping"
    client.subscribe(test_topic)
    time.sleep(0.2)

    payload = {"test": "ping_data", "value": 42}
    published = client.publish(test_topic, payload)
    assert published is True

    # Wait for loopback delivery
    for _ in range(20):
        if received_messages:
            break
```

tests/test_mqtt.py (continued)

```
72          time.sleep(0.1)
73
74      client.disconnect()
75      assert len(received_messages) > 0
76      assert received_messages[0][1]["test"] == "ping_data"
77
```

tests/test_rules.py

```
"""
Unit Tests for Knowledge-Driven Expert Rule Engine.
"""

import pytest
from diagnosis.rule_engine import RuleEngine


@pytest.fixture
def engine():
    return RuleEngine()


def test_rule_normal_operation(engine):
    reading = {
        "temperature_c": 35.0,
        "vibration_rms_g": 0.12,
        "current_a": 0.58,
        "rpm": 2150.0,
        "sensor_status": "ok",
    }
    result = engine.evaluate(reading)
    assert result["suspected_fault"] == "Normal"
    assert result["severity"] == "NORMAL"
    assert result["confidence"] >= 0.90
    assert "RULE_007_NORMAL_OPERATION" in result["activated_rules"]


def test_rule_motor_stall(engine):
    reading = {
        "temperature_c": 55.0,
        "vibration_rms_g": 0.05,
        "current_a": 1.95,
        "rpm": 50.0,
        "sensor_status": "ok",
    }
    result = engine.evaluate(reading)
    assert result["suspected_fault"] == "Motor Stall / Locked Rotor"
    assert result["severity"] == "CRITICAL"
    assert "RULE_002_MOTOR_STALL" in result["activated_rules"]


def test_rule_overheating(engine):
    reading = {
        "temperature_c": 72.0,
        "vibration_rms_g": 0.18,
        "current_a": 1.15,
        "rpm": 1850.0,
        "sensor_status": "ok",
    }
    result = engine.evaluate(reading)
    assert result["suspected_fault"] == "Motor Overheating"
    assert result["severity"] == "CRITICAL"
    assert "RULE_003_THERMAL_OVERHEATING" in result["activated_rules"]


def test_rule_mechanical_overload(engine):
    reading = {
        "temperature_c": 45.0,
        "vibration_rms_g": 0.22,
        "current_a": 1.25,
        "rpm": 1350.0,
        "sensor_status": "ok",
    }
    result = engine.evaluate(reading)
    assert result["suspected_fault"] == "Mechanical Overload"
    assert result["severity"] == "WARNING"


def test_rule_excessive_vibration(engine):
    reading = {
```

tests/test_rules.py (continued)

```
        "temperature_c": 38.0,
        "vibration_rms_g": 0.65,
        "current_a": 0.65,
        "rpm": 2100.0,
        "sensor_status": "ok",
    }
    result = engine.evaluate(reading)
    assert result["suspected_fault"] == "Excessive Vibration"
    assert result["severity"] == "WARNING"


def test_rule_sensor_anomaly(engine):
    reading = {
        "temperature_c": -99.0,
        "vibration_rms_g": 18.0,
        "current_a": -2.0,
        "rpm": 9999.0,
        "sensor_status": "error",
    }
    result = engine.evaluate(reading)
    assert result["suspected_fault"] == "Sensor Acquisition Fault"
    assert result["severity"] == "UNKNOWN / SENSOR ERROR"
    assert "RULE_001_SENSOR_ANOMALY" in result["activated_rules"]

```

tests/test_simulation.py

```python
"""
Unit Tests for Synthetic Generator and Physics Scenarios.
Verifies schema compliance, deterministic seeds, and scenario transitions.
"""

import pytest
from simulation.scenarios import MotorScenario, SCENARIO_PROFILES
from simulation.synthetic_generator import SyntheticGenerator
from ingestion.data_validator import DataValidator


def test_scenario_profiles_completeness():
    """Ensure all scenarios have baseline physical properties defined."""
    for scen in MotorScenario:
        assert scen in SCENARIO_PROFILES
        profile = SCENARIO_PROFILES[scen]
        assert "temp_base" in profile
        assert "vibe_base" in profile
        assert "current_base" in profile
        assert "rpm_base" in profile
        assert "ground_truth_label" in profile
        assert "severity" in profile


def test_generator_schema_compliance():
    """Verify that generated synthetic readings pass DataValidator."""
    gen = SyntheticGenerator(device_id="motor01", seed=42)
    validator = DataValidator()

    for _ in range(20):
        reading, gt = gen.generate_next_sample()
        is_valid, err = validator.validate_sensor_payload(reading)
        assert is_valid, f"Validation failed: {err}"
        assert reading["device_id"] == "motor01"
        assert reading["source"] == "synthetic"
        assert reading["sensor_status"] == "ok"
        assert 20.0 <= reading["temperature_c"] <= 60.0
        assert 0.01 <= reading["vibration_rms_g"] <= 1.0
        assert 0.2 <= reading["current_a"] <= 1.5
        assert 1500.0 <= reading["rpm"] <= 2600.0


def test_generator_reproducibility():
    """Verify reproducible streams with identical random seeds."""
    gen1 = SyntheticGenerator(device_id="motor01", seed=999)
    gen2 = SyntheticGenerator(device_id="motor01", seed=999)

    for _ in range(10):
        r1, _ = gen1.generate_next_sample()
        r2, _ = gen2.generate_next_sample()
        assert r1["temperature_c"] == r2["temperature_c"]
        assert r1["vibration_rms_g"] == r2["vibration_rms_g"]
        assert r1["current_a"] == r2["current_a"]
        assert r1["rpm"] == r2["rpm"]


def test_stall_scenario_physics():
    """Verify stall scenario produces surge current (>1.7A) and near-zero RPM (<200)."""
    gen = SyntheticGenerator(device_id="motor01", seed=123)
    gen.set_scenario(MotorScenario.MOTOR_STALL)

    for _ in range(5):
        reading, gt = gen.generate_next_sample()

    # After a few steps, stall dynamics settle
    assert reading["current_a"] >= 1.70
    assert reading["rpm"] <= 250.0
    assert gt["ground_truth_label"] == "Motor Stall"

```

Artifact inventory and file index

Binary artifacts retained in the ZIP, not rendered in this PDF:

ml/models/random_forest_model.joblib | 757,498 bytes
data/sqlite/fault_diagnosis_backup_20261031_095605.db | 634,880 bytes
data/sqlite/fault_diagnosis.db-wal | 4,165,352 bytes
data/sqlite/fault_diagnosis.db-shm | 32,768 bytes
data/sqlite/fault_diagnosis.db | 4,825,088 bytes

File index - starting pages

2 README.md
6 .env
7 .env.example
8 .gitignore
9 Dockerfile
10 IMPLEMENTATION_PLAN.md
13 PROJECT_STATUS.md
15 docker-compose.yml
16 node_red_flow.json
21 pyproject.toml
22 requirements.txt
23 Final_ARD_Code/Final_ARD_Code.ino
25 Final_ARD_Code/secrets.h
26 backend/auth/security.py
28 backend/main.py
31 config/devices.yaml
32 config/knowledge_base.yaml
34 config/settings.py
37 config/thresholds.yaml
38 dashboard/app.py
40 dashboard/components/charts.py
43 dashboard/pages/1_Overview.py
45 dashboard/pages/2_Live_Monitoring.py
47 dashboard/pages/3_Fault_Diagnosis.py
49 dashboard/pages/4_Historical_Analytics.py
51 dashboard/pages/5_Alerts.py
52 dashboard/pages/6_Settings.py
54 dashboard/pages/7_Model_Performance.py
56 dashboard/styles/theme.css
58 data/csv/sensor_telemetry.csv
60 data/node_red/fault_alerts.jsonl
61 data/node_red/mqtt_test_alerts.jsonl
62 data/synthetic/motor_training_data.csv
64 database/connection.py
65 database/migrations/001_initial_schema.sql
67 database/migrations/migrate.py
68 database/models.py
70 database/repository.py
74 diagnosis/hardware_rules.py
76 diagnosis/hybrid_engine.py
78 diagnosis/knowledge_base.py
80 diagnosis/recommendations.py
83 diagnosis/rule_engine.py
85 docs/API.md
88 docs/ARCHITECTURE.md
90 docs/HARDWARE_WIRING.md
92 docs/LAYER_1.md
93 docs/LAYER_2.md
94 docs/LAYER_3.md
96 docs/LAYER_4.md
97 docs/LAYER_5.md

Artifact inventory and file index (continued)

```
 99  docs/LAYER_6.md
100  docs/PROJECT_REPORT.md
105  docs/TESTING.md
107  docs/TROUBLESHOOTING.md
109  firmware/esp32_motor_monitor/README.md
111  firmware/esp32_motor_monitor/include/config.h
113  firmware/esp32_motor_monitor/include/local_secrets.example.h
114  firmware/esp32_motor_monitor/include/local_secrets.h
115  firmware/esp32_motor_monitor/platformio.ini
116  firmware/esp32_motor_monitor/src/main.cpp
123  ingestion/csv_logger.py
125  ingestion/data_validator.py
127  ingestion/mqtt_client.py
129  ingestion/mqtt_subscriber.py
135  ml/evaluate.py
137  ml/features.py
139  ml/generate_dataset.py
140  ml/models/confusion_matrix.json
142  ml/models/model_metrics.json
144  ml/predict.py
146  ml/train.py
149  nodered/README.md
150  nodered/package.json
151  scripts/init_admin.py
152  scripts/run_demo.sh
153  scripts/run_layer1.sh
154  scripts/run_layer2.sh
155  scripts/run_layer3.sh
156  scripts/run_layer4.sh
157  scripts/run_layer5.sh
158  scripts/run_layer6.sh
159  scripts/setup.sh
161  simulation/mqtt_publisher.py
164  simulation/scenarios.py
167  simulation/synthetic_generator.py
169  tests/test_alerts.py
170  tests/test_api.py
171  tests/test_database.py
172  tests/test_ml.py
173  tests/test_mqtt.py
175  tests/test_rules.py
177  tests/test_simulation.py
```